

Mobile Computing

simi

October 2022

Contents

1	Introduzione	11
1.1	Il contesto tecnologico	12
1.2	Il mobile computing	12
1.2.1	Data Management	13
1.2.2	Sistemi distribuiti	13
1.2.3	Reti	13
1.2.4	Human-Computer Interaction	13
1.2.5	Sistemi Operativi	13
1.3	Caratteristiche dei dispositivi mobili	14
1.3.1	Unità di calcolo	14
1.3.2	Comunicazione di rete	15
1.3.3	Rete cellulare	15
1.3.4	Reti locali	15
1.3.5	GNSS	16
1.3.6	Display	16
1.3.7	Altre periferiche per l'interazione con l'utente	16
1.3.8	Sensori	16
1.3.9	Approssimazione	18
1.3.10	Fotocamere	18
1.3.11	Sensori Virtuali	20
2	Sistema Operativo	21
2.1	Introduzione	22
2.1.1	IOS	22
2.1.2	Android	22
2.2	Caratteristiche dei SO per dispositivi mobili	23
2.2.1	Gestione della memoria	23
2.2.2	Ciclo di vita concettuale	24
2.2.3	SO tradizionale	25
2.2.4	So di dispositivi mobili	26
2.2.5	Riflessione	26
2.3	Astrazione Hardware e Software	27
2.3.1	Frammentazione Hardware	27
2.3.2	Frammentazione Software	27

2.3.3	Frammentazione HW e SW in IOS	28
2.3.4	Esperienza Prof	28
2.4	Gestione della sincronizzazione	29
2.4.1	Eventi	29
2.4.2	Main Thread	30
2.4.3	Problema di sincronizzazione	30
2.5	Gestione Background	32
2.5.1	IOS	32
2.5.2	Android	33
3	Reti e Architetture	35
3.1	Reti cellulari	36
3.1.1	Architettura	36
3.1.2	Celle	37
3.1.3	Utilizzo del canale	37
3.1.4	Handover	37
3.2	Le architetture	38
3.2.1	Connection-oriented	38
3.2.2	Come si progetta l'architettura di un sistema	38
3.2.3	Architettura three-tier per il web	38
3.2.4	Architettura three-tier verso applicazioni	39
3.2.5	Esempio login	40
3.2.6	Confronto	40
3.2.7	Sistemi web+app	41
3.2.8	La definizione del protocollo di comunicazione	41
3.2.9	Esplorazione API	42
3.3	Codifica delle informazioni	42
3.3.1	Codifica dei dati binari	44
3.3.2	Protocol Buffer	44
3.4	Notifiche Push	45
3.4.1	Fase setup	46
3.4.2	Fase di invio	47
3.4.3	Attacchi non possibili (o difficili) nel protocollo notifiche push	48
3.4.4	Notifiche push e locali	49
3.4.5	Esercizi	49
4	La posizione	51
4.1	Tempo	52
4.2	Approssimazione	53
4.2.1	Errore di posizionamento	53
4.3	Orientamento	54
4.4	Calcolo posizione con trilaterazione	54
4.4.1	Upper bound della distanza	55
4.4.2	Range di distanze	55
4.4.3	Gestione dei dati inesatti	56

4.5	Contesti	56
4.6	Tecniche per il calcolo della posizione outdoor	56
4.6.1	Global Navigation Satellite System (GNSS)	57
4.6.2	Posizione basata su rete cellulare	58
4.6.3	WIFI positioning	59
4.6.4	Approccio ibrido	60
4.7	Gestione dati di posizione	60
4.7.1	Geocoding e reverse-Geocoding	60
4.7.2	Geofetcing	60
4.7.3	Calcolo delle distanze	61
4.7.4	Trattamento dei dati spazio temporali	61
5	Analisi	65
5.1	Contesto d'uso di app per dispositivi mobili	66
5.2	Perchè si usano le app mobile	66
5.3	Durata delle sessioni d'uso	66
5.4	Interazione con l'utente	67
5.5	Funzionalità	67
5.6	Progettazione della user experience	68
5.7	Obiettivi dell'analisi	69
5.7.1	Comprensione del dominio applicativo	69
5.7.2	Gli attori	69
5.7.3	Comprendere i problemi	70
5.7.4	Quale metodologia di analisi?	71
5.8	User Centered Design	72
5.8.1	Prototipo	72
5.9	L'attitudine	72
6	Progettazione	75
6.1	Principi di progettazione	76
6.1.1	Pensa all'utente	76
6.1.2	Content Prioritiaztion	76
6.1.3	Less is more	77
6.1.4	Integrità estetica	77
6.1.5	Consistenza	77
6.1.6	Personalizzazione	78
6.1.7	Affordance	78
6.1.8	Metafore	78
6.1.9	Minimizzare l'input	79
6.1.10	Importanza della prima impressione	79
6.2	Progettazione delle interfacce	79
6.2.1	Dimensione dello schermo	79
6.2.2	Orientamento dello schermo	80
6.2.3	Layout a dimensioni assolute	80
6.2.4	Unità di misura delle dimensioni sullo schermo	80
6.2.5	Responsive design	81

6.2.6	Navigazione	82
6.2.7	Progettazione delle singole schermate	84
6.2.8	Prototipi	84
6.3	Progettare la struttura del codice	85
6.3.1	Pattern di progettazione (design pattern)	86
6.3.2	MVC: Model, View, Controller	86
6.3.3	MVP: Model, View, Presenter	87
6.3.4	Uso MVC	88
6.3.5	Vantaggi MVC	88
6.3.6	Limiti di MVC	88
6.3.7	MVVC: Model, View, ViewModel	89
7	Sviluppo Cross-Platform	91
7.1	Sviluppo cross-platform	93
7.1.1	Codice in comune C/C++	93
7.1.2	Sito web / web app	94
7.1.3	Hybrid App	95
7.1.4	Conversione del codice	96
7.1.5	React Native	97
7.1.6	Considerazioni generali sullo sviluppo cross-platform	97
7.1.7	Aneddoto	98
7.1.8	Aneddoto 2	98
8	Testing e Debugging	99
8.1	Dal problema alla procedura	100
8.2	Dalla procedura al codice	100
8.3	Esempio	101
8.4	Gestire progettazione e implementazione	103
8.5	Testing	103
8.5.1	Testing su dispositivi mobili	104
8.5.2	Testi in casi rappresentativi	104
8.5.3	Automatizzazione del testing	106
8.6	Debugging	107
8.6.1	Riprodurre il malfunzionamento	107
8.6.2	Logging	107
8.6.3	Debugger	108
8.6.4	Strategie di debugging	108
8.6.5	Errori comuni	109
8.7	Conclusioni	109
8.7.1	Attenzione a come fare il test	109
8.7.2	Esperienza	110

9	React Native	111
9.1	JavaScript	112
9.1.1	Caratteristiche di JS	112
9.1.2	Definizione di una funzione	112
9.1.3	Metodo map	113
9.2	Oggetti	113
9.2.1	Literal Notation	113
9.2.2	Metodi	113
9.2.3	Il concetto di this	113
9.2.4	Costrutto sintattico new	113
9.2.5	Prototipi	113
9.2.6	Definizione e istanza di una classe	114
9.2.7	Ereditarietà	114
9.2.8	Uso di this nelle classi	114
9.2.9	Wrapper	114
9.2.10	Binding	114
9.2.11	Wrapping vs Binding	114
9.2.12	Arrow function e binding	115
9.3	React	115
9.3.1	Component	115
9.3.2	JSX	116
9.3.3	Hello Word in react	117
9.3.4	Struttura di un componente	118
9.3.5	Lo stato dei componenti	118
9.3.6	Visualizzazione condizionale JSX	119
9.3.7	Visualizzazione iterativa JSX	119
9.3.8	Gestione degli eventi	121
9.3.9	Aggiornamento dello stato	122
9.3.10	Gestione degli eventi con parametri	123
9.4	Composizione di componenti	123
9.4.1	Gestione eventi	125
9.4.2	Single source of truth	126
9.4.3	Componenti controllate	127
9.4.4	Sincronizzazione di componenti multiple	127
9.5	React Native	127
9.5.1	Architettura del sistema	128
9.5.2	Come sono le performance	128
9.5.3	Uso di ReactNative	128
9.5.4	Core component	129
9.5.5	Uso delle liste	130
9.6	Ciclo di Vita di un'applicazione ReactNative	130
9.6.1	Ciclo vista componenti	130
9.7	Componenti funzionali e hook	132
9.7.1	useState	133
9.7.2	useEffect	133
9.8	Thread	134

9.8.1	Callback	135
9.8.2	Promise	136
9.8.3	Async	137
9.8.4	Strumenti di comunicazione in React	139
9.8.5	Fetch risposta	139
9.8.6	Fetch gestione risposta	139
9.8.7	Inviare dei dati	141
9.8.8	Estrazione del risultato	141
9.9	Organizzazione del codice	142
9.9.1	Context	142
9.9.2	Problema della navigazione, ReactNavigation	144
9.9.3	Organizzazione del communication controller	144
9.10	Gli stili	147
9.10.1	Dimensioni	148
9.10.2	Distribuzione del contenuto	150
9.10.3	Posizioni	150
9.11	Memorizzazione persistente	151
9.11.1	Memorizzazione coppie chiave-valore	151
9.11.2	Memorizzazione su DB	152
9.12	Posizione e mappe	155
9.12.1	Acquisizione della posizione	155
9.12.2	Acquisire la posizione	156
9.12.3	Mappe	156
9.13	Debug e test	158
9.13.1	Uso del debug da chrome	158
9.13.2	Unit Test	158
9.13.3	Testing interfaccia	159
10	Android	161
10.1	Activity	163
10.2	Context	164
10.3	Android Manifest	164
10.4	Passaggio tra Activity	164
10.5	Gestione degli eventi	165
10.6	View in Android	168
10.6.1	ViewGroup	169
10.6.2	Personalizzazione di View	169
10.6.3	Layout	170
10.6.4	Stili e temi	171
10.6.5	Creazione delle View	171
10.6.6	Creazione statica della navigazione	172
10.6.7	Risorse e codice	172
10.6.8	View Binding	172
10.7	Fragment	173
10.7.1	Ciclo di vita	173
10.7.2	Creazione del Fragment	174

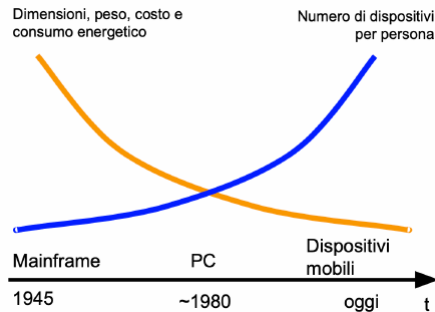
10.7.3	Mostrare un Fragment	174
10.7.4	Passaggio parametri	175
10.8	Model-View-Presenter	175
10.8.1	Memoria persistente	176
10.8.2	Singleton Model	177
10.8.3	View-Model	178
10.8.4	Struttura dell'applicazione	179
10.8.5	Organizzazione del model	181
10.9	Liste ed Adapter	181
10.9.1	Recicle View	181
10.9.2	Esempio	182
10.9.3	La gestione degli eventi	185
10.10	Comunicazione di rete	188
10.10.1	Volley	190
10.10.2	Retrofit	192
10.11	Testing in Android	193
10.12	Memorizzazione persistente	194
10.12.1	Shared Preferences	195
10.12.2	File	195
10.12.3	Database	195
10.12.4	Room	196
10.13	Programmazione multithread in Android	198
10.14	Posizione	200
10.14.1	Ricevere l'ultima posizione nota	201
10.14.2	Autorizzazione dell'utente	202
10.14.3	Uso delle mappe	204

Chapter 1

Introduzione

1.1 Il contesto tecnologico

Perchè studiamo il mobile computing? Partiamo dagli anni dopo la seconda guerra mondiale ed abbiamo uno dei primi calcolatori general purpose che era enorme, costava tantissimo e consumava altrettanto ed era condiviso tra decine di operatori (programmatore). Siamo arrivati ad uno dei computer più piccoli mai sviluppati (più piccolo di un chicco di riso) che si alimenta solo attraverso la luce del sole. Oltre a questi dispositivi di ricerca esistono anche computer molto piccoli, come l'echo loop che è uno smart ring che costa poco. Siamo arrivati a questo perchè all'inizio degli anni 80 abbiamo delle macchine più piccole con prezzi più abbordabili che permettono a tutti di avere una macchina (personal computer). Quindi se vogliamo riportare in termini qualitativi siamo partiti da macchine enormi e costose e man mano nel tempo abbiamo avuto dimensioni peso e costo che sono diminuiti ed il numero di dispositivi che è cresciuto.



1.2 Il mobile computing

Come mai fino ad adesso non ce n'è mai fregato niente del supporto su cui facciamo girare i nostri programmi ed invece adesso dobbiamo focalizzarci sui dispositivi mobili? Da un punto di vista teorico un dispositivo mobile possiamo astrarlo e considerarlo come un qualsiasi computer (macchina di von neumann) in quanto l'informatica è sempre la stessa, è una disciplina scientifica, indipendentemente dagli aspetti tecnologici. Il motivo è che i dispositivi mobili hanno introdotto una serie di problemi e di opportunità che hanno reso necessario ripensare alcune soluzioni tipiche dell'informatica e rivedere molte scelte che erano state prese in alcune discipline dell'informatica. Quindi nelle prime lezioni andremo a rivedere le principali discipline dell'informatica (reti, sistemi operativi, basi di dati...) per capire come sono state affrontate per i sistemi mobili. Di fatto la disciplina del mobile computing ha richiesto di rivedere quasi tutte le discipline dell'informatica, anche l'interazione uomo macchina che è un aspetto fondamentale del mobile computing.

Quindi il mobile computing è lo studio di varie problematiche in un contesto di mobilità durante la fruizione dei servizi. La mobilità degli utenti, il fatto che

gli utenti siano in mobilità non vuol dire solo che si sposta ma che è in contesti diversi, è quindi molto importante capire il contesto in cui l'utente sta usufruendo dei dispositivi (ad esempio un navigatore deve tenere conto che l'utente guida). C'è eterogeneità dei dispositivi mobili e delle interfacce utente, che è molto più marcata rispetto a quella dei dispositivi fissi. Siccome questi dispositivi mobili sono in mobilità non sono connessi via cavo né per l'alimentazione né nella connessione in rete, la realtà quindi è che non hanno una connessione costante, questa cosa ha un grande impatto su come progettiamo i protocolli di comunicazione. Inoltre i dispositivi mobili hanno la sensoristica avanzata.

1.2.1 Data Management

I dispositivi mobili richiedono il trattamento di dati particolari, ad esempio:

- Dati spazio-temporali:
 - Posizione: indoor ed outdoor
 - Tracce: sequenze di posizioni nel tempo
- Dati dei sensori

1.2.2 Sistemi distribuiti

I dispositivi mobili sono quasi sempre parte di un sistema distribuito. Molto raramente sono applicazioni stand alone e spesso possono includere l'accesso a servizi remoti (spesso in cloud) e la comunicazione con altri dispositivi, diretta oppure mediata dai servizi web.

1.2.3 Reti

Ci sono nuovi protocolli di comunicazioni nati apposta per i dispositivi mobili. Le connessioni non sono stabili e quindi sono stati inventati nuovi protocolli di comunicazione.

1.2.4 Human-Computer Interaction

I dispositivi mobili presentano strumenti di I/O sostanzialmente differenti rispetto ai dispositivi tradizionali, hanno quindi reso necessario ripensare completamente il paradigma di interazione con l'utente.

La user experience è fondamentale per il successo della nostra applicazione, e' necessario ripensare completamente le applicazioni in tale senso, fin dalla fase di analisi.

1.2.5 Sistemi Operativi

Problemi tipici del mobile computing da affrontare a livello di sistema operativo:

- Capacità computazionali limitate

- Risparmio energetico: ottimizzare l'OS per il risparmio energetico
- Migliorare l'accesso all'Hardware per i programmatori
- Privacy e sicurezza
- Vincoli real time: il sistema operativo deve rispondere all'utente entro un certo tempo

1.3 Caratteristiche dei dispositivi mobili

Dalla fine degli anni 90 iniziano a diffondersi i primi dispositivi usabili in mobilità, sono i PDA (personal digital assistant) ed i cellulari. In seguito i due dispositivi confluiscono in un unico device chiamato smartphone. Oggi abbiamo un insieme di dispositivi chiamati mobile (smartphone, tablet e dispositivi indossabili).

Le caratteristiche dei dispositivi mobili:

- Utilizzabili in mobilità
- Compatti e leggeri
- Hanno una batteria: che è una risorsa scarsa, perchè deve essere piccola per pesare poco. Noi dovremo ottimizzare le risorse per consumare poca batteria.
- Comunicazione wireless: non hanno un cavo
- Introducono nuove forme di interazione con l'utente

Tutte queste caratteristiche dei dispositivi mobili rendono necessario adottare nuove soluzioni hardware e software.

1.3.1 Unità di calcolo

La macchina di Von Neumann definisce un sistema in cui c'è un unità di elaborazione una memoria ed un Bus per fare comunicare i due. Dal punto di vista teorico lo smartphone usa la stessa architettura ma dal punto di vista tecnologico le componenti si sono evolute. I processori sono ottimizzati per garantire minori consumi e solitamente hanno minori performance, ad oggi in realtà un tablet figo di ultima generazione non ha un processore tanto diverso da quello di un computer ma non dobbiamo ragionare pensando solo ai dispositivi più nuovi, ma considerare che in uso ci sono anche dispositivi con basse performance. Oltre al processore ci sono anche i co-processori, che sono delle componenti hardware che elaborano le informazioni ma sono progettati per risolvere problemi specifici, la CPU è general purpose mentre i co-processori sono progettati per risolvere problemi specifici, ad esempio la GPU è una componente di calcolo specifica per un certo compito. Servono anche per risolvere alcuni problemi in modo più ottimale dal punto di vista del consumo energetico, esistono anche sui dispositivi

tradizionali, ma sui dispositivi mobili ce ne sono di più ed hanno un ruolo più fondamentale.

Esempio architettura I-Phone 11 pro, i dispositivi moderni hanno un'unica componente elettronica che contiene più funzionalità hardware:

- A13 Bionic: SOC (system on a chip): Un'unica componente elettronica che include più funzionalità Hardware
 - CPU: architettura ARM (Risc rispetto a x86 che è Cisc) a 64bit con 6 core: 2 a high-performance e 4 high-efficiency. 1 core è in grado di eseguire un flusso di operazione, quindi avendone di più posso eseguire più flussi di esecuzioni in parallelo.
 - Co-processor:
 - * GPU: a 4 core, unità di calcolo specifica per questioni di grafica
 - * Neural engine: 8 core, dedicato a problemi di machine learning
 - * Image coprocessor: utilizzato per la gestione delle immagini (prese dalla fotocamera)
 - * Secure Enclave coprocessor: per la gestione delle chiavi crittografiche
- M13: altro co-processore (esterno rispetto all'A13) per la gestione di dati in movimento

1.3.2 Comunicazione di rete

I dispositivi mobili sono stati progettati per essere sempre connessi in rete, ma è comune che un dispositivo possa sconnettersi (ad esempio quando siamo in galleria) e permettono di comunicare su rete cellulare voce e dati, mentre su rete wireless solo rete dati.

1.3.3 Rete cellulare

Avviene con varie antenne sparse sul territorio e con vari protocolli che si evolvono nel tempo (5G)

1.3.4 Reti locali

Abbiamo reti wifi, protocollo 802.11 con vari sotto protocolli, utilizzate per realizzare WLAN con distanze di circa 100m. Bluetooth per trasmissioni dati a corto raggio e sono usati per:

- Creazione personal area network: quando usiamo le cuffie bluetooth
- Far comunicare dispositivi diversi
- Far comunicare il mio dispositivo con un device specifico (beacon Bluetooth)

C'è l'NFC per la comunicazione a cortissimo raggio. L'ultra wide band (UWB) è un tipo di comunicazione che aiuta a capire la posizione dei vari oggetti.

1.3.5 GNSS

Esiste un'antenna specifica per ricevere dati dal satellite, GPS. Vedremo come facciamo a calcolare la posizione utilizzando il segnale satellitare

1.3.6 Display

Ha un ruolo diverso rispetto ai dispositivi tradizionali perché sui dispositivi fissi è un dispositivo di output mentre nei dispositivi mobili anche di input. Può avere varie caratteristiche come la risoluzione, la densità in pixel, l'aspect ratio (rapporto tra i lati), può utilizzare diverse tecnologie come LCD o OLED. Ma soprattutto può non avere una dimensione rettangolare.

Display touch

La componente di input ha due tecnologie:

- Resistiva: abbiamo due strati che conducono elettricità e nel momento in cui si vanno a toccare permettono di calcolare dove è avvenuto il tocco. E' in disuso. Ma ci sono alcune situazioni in cui lo schermo capacitivo non può essere utilizzato
- Capacitivi: hanno un solo livello, su cui scorre un flusso di elettroni, e vari sensori negli angoli che capiscono il flusso degli elettroni, nel momento in cui tocchiamo con il dito deviamo il flusso degli elettroni ed i sensori capiscono dove abbiamo toccato. Possono anche gestire il multi-touch.

I casi in cui serve uno schermo resistivo servono quando abbiamo un contesto in cui ad esempio un utente deve utilizzare dei guanti particolari

1.3.7 Altre periferiche per l'interazione con l'utente

La vibrazione è una periferica di output, serve per dare un'informazione all'utente. Si possono utilizzare le periferiche che vediamo come strumenti di input. Abbiamo un'interfaccia audio, il microfono input e l'altoparlante output, possiamo avere un'interfaccia vocale in cui l'utente si interfaccia con il dispositivo parlando.

1.3.8 Sensori

I dispositivi mobili hanno una serie di sensori, che in generale, convertono una quantità fisica in un segnale. Ad esempio quando facciamo una chiamata per evitare che l'utente tocchi con la guancia utilizziamo un sensore di prossimità

per spegnere lo schermo. Un sensore di luminosità serve per regolare automaticamente la luminosità dello schermo in base alla luce che c'è intorno. Il barometro serve per capire l'altezza in cui siamo. Ci sono tutta una serie di sensori biometrici che misurano dei tratti biologici dell'utente (impronta digitale)

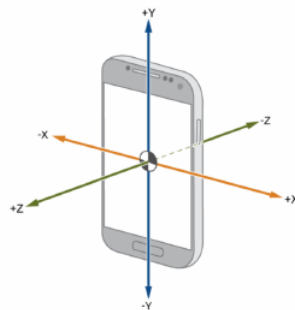
Sensori inerziali

Sono sensori che permettono di calcolare l'accelerazione e la rotazione. In realtà viene spesso citato anche il magnetometro, la bussola ci dice da che parte stiamo guardando, calcola la direzione rispetto al campo magnetico.

Accelerometro

Misura l'accelerazione lungo i tre assi del dispositivo, attenzione che si può utilizzare per due cose:

- Quanto o come si sta spostando il device. Bisogna fare attenzione perchè la forza di gravità è calcolata come un'accelerazione.
- Capire com'è orientato il device, perchè grazie al fatto che misura l'accelerazione di gravità sui vari assi capiamo com'è orientato il device. Attenzione che la forza di gravità è calcolata in base al device, non in base al mondo.



Un esempio di utilizzo è per un'applicazione che riconosce le strisce pedonali per le persone non vedenti. Vogliamo aiutare l'utente a tenere il device puntato con l'angolo giusto (non deve inquadrare né il cielo né i suoi piedi). La soluzione è tenere monitorato il valore dell'accelerometro sull'asse che passa attraverso lo schermo (in modo da calcolare l'accelerazione di gravità e capire dov'è direzionato il telefono), deve essere incluso tra due valori di soglia.

Giroscopio

Misura la velocità angolare sui tre assi. Siccome misura la velocità in quando il device è in stato di quiete è 0. Non sappiamo qual'è l'angolo di riferimento, cioè sappiamo a che velocità sta ruotando, ma non in che direzione, in pratica il sistema di riferimento è sempre il device.

Magnetometro

Misura l'angolo rispetto al nord. In ambienti urbani è poco affidabile ed è usato insieme al giroscopio per tenere traccia di dove sto guardando rispetto al nord (l'angolo rispetto al nord)

1.3.9 Approssimazione

Quando abbiamo le misurazioni dei sensori inerziali queste sono necessariamente discrete, ovvero sappiamo ogni tanto qual'è il valore di quel sensore, ma nel mondo reale è un'informazione continua. Inoltre il valore misurato non è sempre corretto, ci sarà sempre un valore di approssimazione. Questa approssimazione ci porta un problema molto comune quando usiamo sensori inerziali, che riscontriamo principalmente nel calcolo dello spostamento. Possiamo modellare partendo dall'accelerazione integrandola e trovando la velocità ed integrando di nuovo per trovare la posizione. Con sensori molto precisi funziona bene, ma con un device standard funziona molto male, perchè facendolo abbiamo un errore che si accumula nel tempo, tale per cui dopo pochissimo non sappiamo più dove si trova il device. Questo effetto di approssimazione che aumenta nel tempo si chiama DRIFT.

1.3.10 Fotocamere

La camera è importante perchè può essere utilizzata per applicazioni evolute, ad esempio adottando tecniche di computer vision per estrarre informazioni dal mondo esterno (esempio google maps che capisce dove siamo). Queste tecniche utilizzano spesso un'immagine detta di profondità, che è un'immagine in bianco e nero in cui il colore di un pixel dipende dalla distanza del punto rispetto al sensore, se vogliamo riconoscere un ambiente possiamo farlo molto meglio se conosciamo la distanza degli oggetti.



Stereo triangulation

Tecnica per stimare le distanze e per calcolare una depth image, è una tecnica che usa il nostro cervello, i nostri occhi prendono immagini diverse ed il nostro cervello è in grado di calcolare la distanza degli oggetti ricomponendo le immagini. In questo caso abbiamo più camere sul device, la cui posizione relativa è nota. Quindi se prendiamo le immagini risulteranno diverse quindi calcolando

le differenze delle immagini calcoliamo la distanza. Questa tecnica è usata dalle librerie di realtà aumentata di apple.

Motion parallax

E' usata quando si ha una camera sola, prendiamo a brevissima distanza di tempo due immagini dalla stessa camera e stimiamo quanto si è spostata la camera tra un'immagine e l'altra e a questo punto come prima calcolando le differenze tra le due immagini posso calcolare le distanze. Usato dalle tecniche AR core di Google per la realtà aumentata.

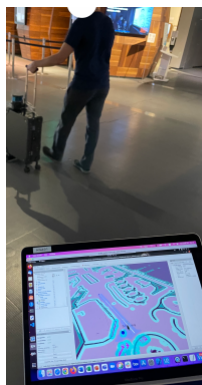
Structured Light

Esistono dei sensori appositi, su alcuni modelli di Iphone esiste una telecamera apposita. Supponiamo di essere in una stanza buia, sappiamo che di fronte a noi c'è un muro ma non sappiamo quanto è lontano ma abbiamo due puntatori laser, possiamo metterne uno ad un angolo noto rispetto all'altro, se i puntini sul muro sono vicini vuol dire che siamo vicini al muro. Costruiamo un triangolo quindi sappiamo calcolare la distanza. Questo hardware fa la stessa cosa ma lo fa nell'infrarosso.



Lidar - Liser Imaging Detector and Rading

Hardware per calcolare la distanza. Negli ultimissimi modelli è disponibile anche in alcune tipologie di smartphone. Trasmette un impulso laser e vede come viene ritrasmesso. Permette di avere ricostruzioni precise di ambienti 3D.



1.3.11 Sensori Virtuali

Oltre i sensori fisici abbiamo quelli virtuali, si comportano come quelli fisici ma sono delle librerie software che prendono i dati da sensori fisici elaborano i dati e dicono qualcosa di più. Ad esempio i dispositivi hanno un contapassi, che è un sensore virtuale, è il co-processore di movimento che sulla base dei dati dell'accelerometro e del barometro calcola come ci stiamo spostando.

Chapter 2

Sistema Operativo

Tutto quello che ci spiega oggi ci serve per programmare, non è solo un ripasso di sistemi operativi finì a se stessi e se non capiamo quello che dice in questo capitolo una volta che ci troveremo davanti ad un errore del codice non capiremo perchè succede.

2.1 Introduzione

I SO per i dispositivi mobili hanno sostanzialmente gli stessi obiettivi principali di un OS normale:

- Ottimizzare l'uso delle risorse
- Semplificare l'accesso alle risorse da parte del programmatore

Ma hanno delle peculiarità. Le risorse computazionali sono limitate soprattutto nei dispositivi indossabili e tra queste risorse si aggiunge l'utilizzo della batteria e poi hanno dei vincoli di real time, che possono esserci anche nei dispositivi tradizionali ma in quelli mobili data la scarsità delle risorse sono più difficili da gestire. Un esempio di vincolo è processare immediatamente una chiamata in arrivo.

2.1.1 IOS

E' un OS dei dispositivi apple, arrivato alla versione 16. Fino al 2019 esisteva una sola versione per Iphone ed Ipad, dopo le hanno separate, per Ipad c'è IpadOS di derivazione IOS. I linguaggi di sviluppo per la programmazione IOS sono:

- Objective C, che è un'estensione propria di C++
- SWIFT, molto più moderno che ha introdotto una serie di nozioni interessanti, come l'oggetto nullable, la gestione della memoria fatta in maniera diversa

2.1.2 Android

Nasce come piattaforma sviluppata da Google insieme ad un consorzio di produttori (Open Handset Alliance). Mentre IOS supporta solo architettura arm, android supporta anche x86 e supporta un gran numero di piattaforme hardware. I linguaggi di programmazione:

- Java: lo faremo noi
- Kotlin: un dialetto di Java

Fino alla versione 4.4 di Android Java veniva convertito in bytecode e poi compilato dalla java virtual machine ogni volta che veniva richiamato il codice. Dopo il codice viene compilato quando scarichiamo la applicazione (compilato in fase di installazione). E' possibile sviluppare in C e C++ tramite una libreria NDK.

Storia

Android era nato prima, sviluppato da un progetto indipendente. Google l'ha acquistato, quando Steve Jobs ha presentato il primo iPhone hanno fondato la Open Handset Alliance.

Android è [open source](#), ma in realtà la versione di Android che abbiamo sul cellulare sono degli adattamenti del sistema operativo originale di Google.

2.2 Caratteristiche dei SO per dispositivi mobili

Tutto quello che abbiamo imparato per i SO tradizionali continua a funzionare. Però vedremo alcune peculiarità dei SO per dispositivi mobili, ripetiamo che ci servono per programmare.

- Gestione della memoria principale
- Gestione del background
- Gestione della sincronizzazione
- Astrazione dell'hardware
- Supporto delle push notifications

2.2.1 Gestione della memoria

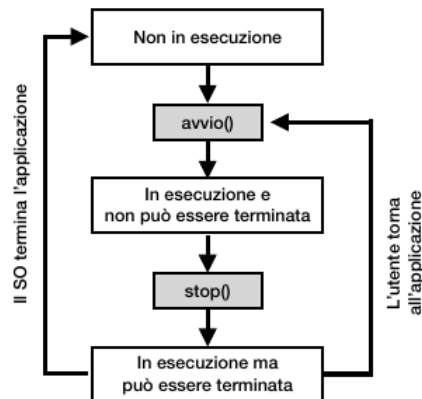
Come nei dispositivi tradizionali, anche i dispositivi mobili adottano tecniche di memoria virtuale. Quello che ci interessa è che ogni applicazione ha uno spazio di memoria virtuale dove può scrivere soltanto lei e per quanto le riguarda esiste solo quella memoria lì, è un vantaggio in termini di sicurezza, perché non vogliamo che un processo possa scrivere su memoria di altri. E poi non si deve preoccupare il processo di capire se c'è o meno memoria libera, è l'OS che rimappa la memoria virtuale in memoria fisica, tramite la MMU. La memoria viene usata in blocchi di dimensione fissa dette pagine, è una discretizzazione dello spazio. Quello che succede quando terminiamo la memoria principale nei dispositivi tradizionali è che il OS prende delle pagine dalla memoria principale e le copia nella memoria secondaria in modo da poter riassegnare le pagine che ha liberato, se poi qualcuno ha bisogno di quelle pagine vengono riportate in memoria, si chiama swap. Ci sono dei problemi perché accedere alla memoria secondaria è molto più lenta della principale.

I OS per dispositivi mobili utilizzano la memoria virtuale con un'unica sostanziale differenza, le pagine di memoria non vengono mai spostate su memoria secondaria. Cosa succede quando il sistema termina la memoria principale, il OS termina 1 o più processi per liberare la memoria. Il problema di questa cosa è che terminando un processo perdo anche i dati che stava elaborando. Però dalla nostra esperienza abbiamo notato che non è mai successo che spostando un'applicazione in background periamo quello che stavamo facendo, quindi c'è

un truccone. Il trucco è che c'è una **collaborazione**, in generale OS e applicazione per dispositivi mobili collaborano per nascondere all'utente che il processo è terminato, vuol dire che il OS quando la nostra applicazione viene messo in background sa che quando è così può essere terminato, quindi quando va in background salva tutte le informazioni che gli servono per quando torna in foreground ha tutto quello che gli serve per ripartire. Ma come fa un applicazione a capire quando va in background? non è un evento del codice è una cosa che fa l'utente. Allora i OS per dispositivi mobili definiscono delle funzioni che il programmatore della applicazione implementa e che il OS richiama. La definizione di alcuni metodi che il programmatore implementa ed il OS richiama si chiama ciclo di vita dell'applicazione.

2.2.2 Ciclo di vita concettuale

Quando l'applicazione non è in esecuzione non c'è neanche un processo, quindi la prima fase è abbastanza superflua. Quando viene avviata, c'è un processo, viene chiamato un metodo, `avvio()`, dall'OS ed il programmatore che può definire in modo specifico cosa fa l'applicazione quando parte. Poi l'app entra in uno stato in cui non può essere terminata (esiste anche lo stato in cui può essere terminata), tendenzialmente quando è in foreground, ovvero quando l'utente la sta usando. Ad un certo punto ad esempio l'utente mette l'app in background l'OS chiama un metodo `stop()` che dice che l'app potrebbe terminare da un momento all'altro e noi dobbiamo creare questo metodo in modo da salvare i dati che stavo usando, dopo il metodo `stop()` passa in uno stato in cui è in esecuzione e può essere terminata. A questo punto se il OS ha bisogno di memoria termina l'applicazione oppure se non lo termina, l'utente torna ad aprire l'applicazione, in entrambi i casi quando l'app riparte viene chiamato il metodo di `avvio()`.



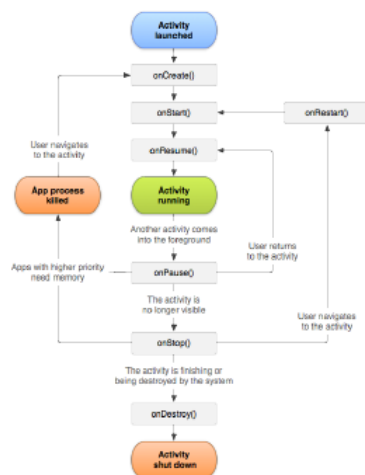
Android: ciclo di vita

Abbiamo lo stato dell'applicazione che non è ancora stata lanciata, quando non può essere terminata e quando è terminata. Inoltre esistono altri stati:

- Entra un'altra applicazione e mi copre solo parzialmente la mia, ad esempio messaggio che la batteria è quasi scarica, quindi richiamo solo il metodo `onPause()`
- L'applicazione va in background viene chiamato prima `onPause()` e poi `onStop()`

Questo è un ciclo di vita di un activity, che è una componente di android che rappresenta una schermata di un applicazione.

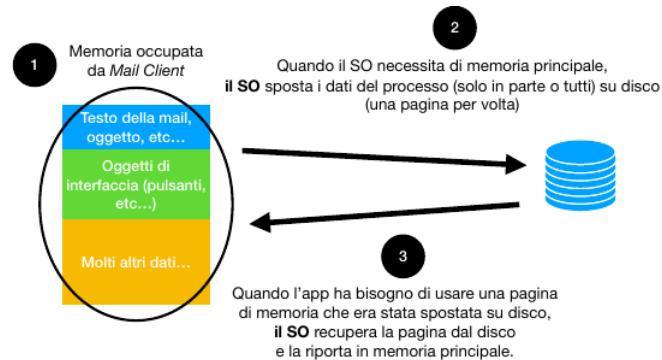
Il OS ci nasconde il fatto che il processo è stato terminato, perchè nel launcher ci sono le applicazioni usate di recente, non quelle attive in questo momento.



2.2.3 SO tradizionale

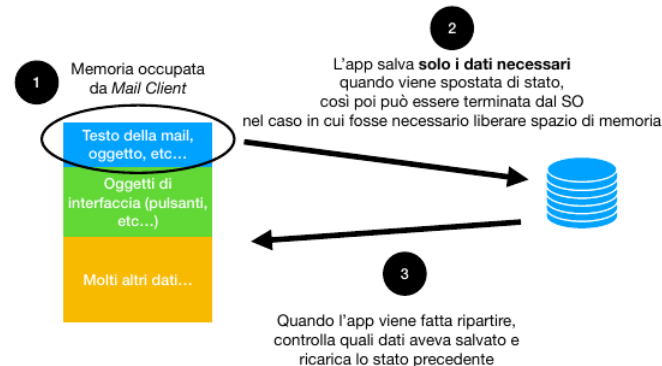
Abbiamo la memoria del mail client, ha il testo della mail e così via, inoltre ha tutta una parte di memoria che mi rappresenta gli oggetti di interfaccia (pulsanti..) e molti altri dati. Quando il SO necessita di memoria principale inizia a prendere delle pagine di questa memoria e le sposta in memoria secondaria, quando noi riapriamo il mail client riprende le pagine dalla memoria secondaria e le rimette in quella principale. Tra i dati in memoria l'unica cosa che dobbiamo salvare sono i dati che ha inserito l'utente, tutti gli altri dati (come l'interfaccia grafica) non servono, perchè sono tutte cose che sono già in memoria secondaria. Perchè l'OS per dispositivi tradizionali deve fare questo? perchè solo il programmatore sa cosa veramente serve in memoria principale. Però l'approccio dei dispositivi tradizionali è un gran vantaggio, perchè

il programmatore può fregarsene di quando la memoria finisce, perchè gestisce tutto il OS dando l'impressione al programmatore che la memoria virtuale sia infinita. Quindi questo vantaggio viene pagato dal fatto che rischiamo di avere un sistema poco efficiente.



2.2.4 So di dispositivi mobili

Nei sistemi mobili è il programmatore che gestisce i dati in memoria, quindi salva in memoria secondaria solo i dati che sa che gli serviranno, non quelli che ha già in memoria principale.



2.2.5 Riflessione

Questa collaborazione tra OS ed applicazioni ci permette di gestire la memoria in modo più performante, ovviamente questo ha un costo, i programmatori lavorano di più perchè va scritto del codice in più ed il problema legato a questo è che i programmatori fanno errori scrivendo codice.

C'è un altro vantaggio, con questo approccio il sistema operativo trova sempre spazio in memoria, perchè ogni volta che gli serve memoria principale semplicemente termina un'applicazione e rapidamente libera una grossa parte di

memoria per essere utilizzate per altri processi. Un contro invece è che nella gestione e pagine tipica dei dispositivi tradizionali se ci servono poche pagine posso liberare solo quelle, mentre nei dispositivi mobili può succedere che nonostante serva poca memoria devo comunque terminare un processo che poi deve ripartire e ricopiare un sacco di dati in memoria.

2.3 Astrazione Hardware e Software

Il concetto di astrazione è una parte molto importante negli OS, hardware abstraction layer è una componente dell'OS che fa l'astrazione dell'hardware, un file è un astrazione fatta dall'OS per gestire i dati. Inoltre scrivere su una periferica sarebbe un casino senza OS perchè dovresti conoscere perfettamente la periferica per usarla, anche questa è un'astrazione. Gli OS per sistemi tradizionali hanno molte periferiche con cui lavorare ma il problema è ancora più sentito per i dispositivi mobili ed è causato dal fenomeno della frammentazione:

- Frammentazione hardware: ho tanti possibili dispositivi diversi fisici, con caratteristiche molto diverse tra loro
- Frammentazione software: ci sono tante versioni dell'OS che possono offrire funzionalità diverse o offrirle in maniera diversa

2.3.1 Frammentazione Hardware

I dispositivi che supportano Android sono circa 40.000 e crescono ad una velocità di 5.000 all'anno, che vuol dire circa 10 dispositivi nuovi con android al giorno. Android esegue su smartphone, tablet, ereader, console dei giochi, macchine fotografiche, TV...

Rendiamoci conto quanto è difficile a livello dell'OS astrarre tutto questo hardware differente, non è che il OS riesca benissimo ad astrarre per tutti questi dispositivi diversi, in molti casi quindi il problema ricade sul programmatore (apri la telecamera posteriore, se c'è).

2.3.2 Frammentazione Software

L'altro problema è quello dato dalle diverse versioni software. Quando usiamo qualsiasi applicazione abbiamo ad un certo punto questa che interagisce con l'OS (scrivere sul local storage). Queste chiamate che si possono fare molte sono sempre simili, ma molte altre volte cambiano. Quindi le versioni del sistema operativo sono spesso associate ad un numero che ci dice con che versione delle API funziona, le API cambiano per correggere errori, per migliorare le prestazioni e per supportare nuovo hardware/servizi/librerie.

Software Fragmentation in Android

Quando creiamo un nuovo progetto (in android studio) indichiamo quali sono la massima e la minima versione del sistema operativo che l'applicazione supporta.

La massima ok è l'ultima, ma come scegliamo la minima? (notiamo che esiste una chiamata quando l'app esegue per capire qual'è la sua versione) un primo approccio è fare funzionare l'applicazione con tutte le versioni, il problema è che ci saranno un sacco di casi in cui dovremo usare delle funzionalità che esistono nella nuova versione ma non in quelle vecchie, un casino, devo scrivere un sacco di controlli. Posso mettere la minima uguale alla massima, ma il problema è che escludo un sacco di utenti. Bisogna mediare, tocca trovare una tabellina in cui ci dicono le varie versioni di android con le varie versioni delle API e caso per caso si sceglie un compromesso per non impazzire con il codice e non escludere tanti utenti. In ambito industriale inizialmente sviluppiamo prima in una versione che ci permette di buttare fuori l'applicazione in fretta e poi lo sbatti di supportare anche versioni meno recenti.

Questo problema su Android è terribilmente sentito, perchè esistono una marea di versioni di Android e API. Ci sono un sacco di sistemi diversi e l'aggiornamento dei dispositivi Android è molto lento, ad esempio ad oggi sono in vendita android con versioni 10 (3 versioni più vecchie), quindi questa coda è molto lunga.

ANDROID PLATFORM VERSION	API LEVEL	CUMULATIVE DISTRIBUTION
4.1 Jelly Bean	16	
4.2 Jelly Bean	17	99,9%
4.3 Jelly Bean	18	99,7%
4.4 KitKat	19	99,7%
5.0 Lollipop	21	98,8%
5.1 Lollipop	22	98,4%
6.0 Marshmallow	23	96,2%
7.0 Nougat	24	92,7%
7.1 Nougat	25	90,4%
8.0 Oreo	26	88,2%
8.1 Oreo	27	85,2%
9.0 Pie	28	77,3%
10. Q	29	62,8%
11. R	30	40,5%
12. S	31	13,5%

2.3.3 Frammentazione HW e SW in IOS

Frammentazione hardware in IOS è molto più limitata, ci saranno una ventina di dispositivi apple. Tecnicamente anche IOS ha tante versioni dell'OS come android, ma i device sono molto più aggiornati, quindi addirittura per IOS possiamo gestire solo 2 versioni o ancora più estremo è farlo solo per l'ultima versione.

2.3.4 Esperienza Prof

Applicazione che trasforma la quantità di luce in suono. Si usava la camera, ma non bastava perchè l'obiettivo può essere tanto o poco aperto. Su IOS fai una

chiamata all'OS che da un metadato dell'immagine che ha anche l'informazione che ci interessa. Per Android in cui molti avevano la versione 4, ma stava uscendo la 5. Tra la 4 e la 5 sono uscite delle nuove API per la gestione della camera (camera2), su camera 2 tutto funzionava. Su dispositivi 4 l'API camera restituiva il valore che ci interessava come stringa, quindi dobbiamo convertire la stringa in frazione. Il problema che in alcune lingue la stringa era in caratteri arabi, giapponesi o cinesi, quindi toccava tradurre la stringa per ogni lingua in cui faceva crashare l'applicazione. L'hardware abstraction layer non era fatto bene, non astraeva abbastanza l'hardware per il programmatore. Per risolvere questo problema in Android erano 3/400 righe di codice, mentre in IOS erano 3.

Osservazione

In università siamo abituati a scrivere dei prototipi che sono applicazioni che non affrontano molti problemi, tra cui quello della frammentazione.

Consigli: tutte le volte che facciamo un progetto universitario teniamo traccia di quanto tempo ci mettiamo, in modo da saper vendere il nostro tempo. Abbiamo in testa che per andare dall'idea iniziale al prototipo ci voglia 100 e dal prototipo al mercato ci voglia 1. No, per andare dal prototipo al prodotto finale ci vuole tanto tempo quando da 0 al prototipo ed è un lavoro noioso.

2.4 Gestione della sincronizzazione

Dobbiamo capire come gli OS mobile gestiscono la sincronizzazione, perchè sarà il problema che ci farà fare più errori quando andiamo a programmare.

I OS moderni sono multitask, vuol dire che possono eseguire più processi contemporaneamente, vuol dire che se abbiamo 1 CPU con 1 Core i processi si alternano nell'utilizzo della CPU simulando il fatto che eseguono in parallelo, l'OS alterna i processi. Se invece abbiamo tanti core possiamo veramente eseguire in parallelo. L'esecuzione in parallelo non avviene solo tra i processi (programma in esecuzione), possiamo avere più modi per eseguire in parallelo perchè all'interno di un processo abbiamo tanti thread, che sono flussi esecutivi indipendenti all'interno dello stesso processo (usano la stessa memoria del processo). Ogni thread esegue il suo flusso di esecuzione, il programmatore non sa più in che ordine il sistema operativo schedula i thread, quindi non possiamo mai sapere in un thread a che punto è arrivato l'altro, se ho bisogno di saperlo devo sincronizzare i due thread in modo che quando uno scrive non scriva sulla memoria di altro. Per fare questo si usano i semafori.

2.4.1 Eventi

Questo è terribilmente importante nella programmazione mobile, i programmi per i dispositivi mobili sono fortemente basati su eventi, un evento è qualcosa che avviene indipendentemente dal mio flusso di esecuzione. Ad esempio se un

utente clicca su un pulsante non sappiamo a che riga del codice siamo quando clicca. Così come non sappiamo quando l'OS ci da dei nuovi dati. Tutte le cose che avvengono in maniera asincrona rispetto al nostro codice sono eventi, possono essere eventi di comunicazione di rete, di sensori (ad esempio l'OS ci dice ogni tanto quando c'è una nuova posizione). Possiamo però creare del codice di gestione dell'evento.

2.4.2 Main Thread

Nei OS per i dispositivi mobili c'è UN solo thread che può gestire (rispondere) gli eventi e modificare l'interfaccia grafica, per questione di performance. Questo è detto Main Thread (thread principale).

Siccome c'è un solo thread che può modificare l'interfaccia grafica, se questo rimane bloccato a fare altro nessuno può modificare l'interfaccia quindi rimane tutto bloccato (finestra dell'app che non risponde).

Supponiamo di avere un'applicazione che quando l'utente tocca sul pulsante deve richiedere dati ad un server e quando risponde deve mostrare la scritta in una casella di testo. Abbiamo un modo per dire quando l'utente tocca su quel pulsante esegui quel codice.

```
//Codice eseguito quanto l'utente preme sul pulsante
result = sendRequest("myServer.com");
myTextField.setTest(result);
```

C'è una chiamata bloccante quella di rete al server, per terminare deve fare prima un operazione di I/O che è molto più lento rispetto ad operazioni classiche. Questo codice lo esegue il main thread, che se è bloccato perché sta aspettando I/O non può aggiornare l'interfaccia grafica, quindi l'utente non può fare nulla.

Le chiamate bloccanti sono di I/O o di CPU bound non possono essere eseguite dal main thread, che deve essere libero per fare aggiornamenti di interfaccia.

Quando devo fare una chiamata bloccante devo creare un altro thread secondario che faccia l'operazione di I/O o un operazione CPU bound, in modo tale che deleghiamo ad altri thread le operazioni bloccanti (lunghe) e teniamo il thread principale libero per rispondere ad eventi di interfaccia. Attenzione, problema di terminologia, in doc android sono detti background thread che si confonde con processo in background ma non centra una mazza.

2.4.3 Problema di sincronizzazione

Solitamente vogliamo fare qualche operazione sul thread secondario e quasi sempre vogliamo notificare l'utente (scarichiamo una roba e diciamo qualcosa all'utente) ma può farlo solo il thread principle, quindi ci vuole sincronizzazione, quando il thread secondario ha finito deve avvisare il thread principale che ha finito ed ora tocca a lui. Il codice concorrente è molto difficile da scrivere e debuggare, perché introduce un concetto di non determinismo, perché i thread

vengono schedulati in maniera diversa ed una cosa che prima funzionava ora non funziona più.

Esempio di errore: un thread secondario prova a modificare l'interfaccia grafica:

```
Crea un thread secondario che faccia questo {
    result = sendRequest("myServer.com");
    myTextField.setTest(result);
}
```

Esempio di errore: il main thread prova ad utilizzare il risultato della chiamata di rete prima che questo sia disponibile

```
Crea un thread secondario che faccia questo {
    result = sendRequest("myServer.com");
}
myTextField.setTest(result);
```

Occhio ci sono due flussi esecutivi diversi! il thread secondario è lento perchè sta facendo operazioni bloccanti.

La prima soluzione sono i costrutti di sincronizzazione di basso livello come wait, notify e semafori. Che fortunatamente nessuno usa.

Esistono quindi due soluzioni a livello piu' alto:

- Costrutti di sincronizzazione di alto livello introdotti per semplificare la gestione del codice nei casi tipici della gestione degli eventi, ad esempio un thread secondario può richiedere al thread principale di eseguire del codice. Ho due chiamate, una mi crea un altro thread ed un'altra che dice di fare eseguire un pezzo di codice al thread principale che a livello di OS gestisce le wait e notify
- Chiamate asincrone: metodi non bloccanti che gestiscono internamente la complessità della sincronizzazione, implementando la chiamata bloccante al loro interno

Esempio: uso di costrutti di sincronizzazione di alto livello:

```
Crea un thread secondario che faccia questo {
    result = sendRequest("myServer.com");
    Esegui questo codice nel thread principale{
        myTextField.setTest(result);
    }
}
```

Viene creato il thread secondario, noi usiamo una chiamata che dice quando il risultato della computazione del thread secondario è disponibile esegui quel pezzo di codice nel thread principale.

Esempio: Uso delle chiamate asincrone

```
//Definiamo prima questa funzione
func manageResult(result){
    myTextField.setText(result)
}

//Codice eseguito quando l'utente clicca sul pulsante
sendRequestAsync("myServer.com", manageResult)
```

Possiamo scrivere un metodo che all'interno giochi con i thread in questo modo e all'esterno esponi le cose più semplici. In questo caso creiamo prima una funzione che deve eseguire il main thread, e per usare quel metodo facciamo una chiamata asincrona che funziona aspettando che la chiamata request produca un risultato, questa chiamata non è bloccante per il main thread che non rimane bloccato lì aspettando, ma mentre aspetta il risultato può fare altro. Usa sempre i costrutti wait e notify ma noi non lo vediamo.

2.5 Gestione Background

E' l'unico argomento che si differenzia tra android e IOS a differenza di quello che abbiamo visto fino ad adesso che è uguale per entrambi i sistemi operativi.

Il background è quando un'applicazione esegue senza essere usata dall'utente. Un processo di background avrà accesso ad un po' meno risorse, lo scheduler se ci sono tanti processi che hanno bisogno di accedere alla CPU darà precedenza a quelli in foreground perchè sono quelli con cui l'utente deve interagire. Il programmatore di dispositivi tradizionale non programma per applicazioni in background, non gli cambia niente. Mentre per IOS ed Android bisogna farlo. Bisogna tenere presente che molti servizi necessitano farlo, ad esempio un riproduttore musicale, oppure un'app di messaggistica deve sempre ricevere i messaggi. Si esegue in background quello che è strettamente necessario, tanto che quello che richiedeva esecuzione in background nei dispositivi tradizionali nei dispositivi mobili hanno cambiato l'architettura per evitarlo.

Un classico esempio di situazione in cui nei dispositivi tradizionali in cui si eseguiva in background è quando dobbiamo ricevere un messaggio dal server, nei dispositivi mobili è stato inventato un protocollo in modo che il nostro dispositivo non debba sempre eseguire del codice per aspettare una risposta dal server.

2.5.1 IOS

Apple ha sempre avuto un approccio molto restrittivo rispetto al background a differenza di Android.

Le applicazioni in IOS non possono eseguire in background a meno di particolari azioni che sono in un elenco stilato da apple. Un esempio di queste

opzioni è quando un applicazione richiede una posizione, deve fare quello che deve in una piccola frazione in background e se non riesce viene terminata.

I controlli che fa apple sono serrati, si possono trovare dei modi per fregare, avevano dichiarato l'app come di navigazione in modo che il sistema operativo si risveglia e loro potevano fare i calcoli in background nonostante l'app non fosse di navigazione, c'è però un omino che controlla manualmente che l'app siano veramente quello che sono state dichiarate, li hanno fregati.

2.5.2 Android

Fino a non molto tempo fa si poteva eseguire tutto il codice che volevamo in background, la differenza con i dispositivi tradizionali è che si doveva dichiarare. Adesso è cambiato un po' l'approccio, ridefinisce l'idea di background dividendolo in due categorie:

- Applicazioni che hanno un effetto immediato sull'utente (ad esempio un riproduttore musicale), quindi è in background in termini tecnici, ma gli consente di eseguire in background tutto il codice che vuole.
- Le applicazioni che non hanno un effetto immediato sull'utente, ad esempio le applicazioni che memorizzano dei log, queste sono operazioni posticipabili che il sistema operativo mette a disposizione un servizio in cui diciamo al sistema operativo in determinate condizioni (quando batteria è carica, quando sei in wifi...) fai questa cosa

Chapter 3

Reti e Architetture

Molti dettagli di come sono fatte le reti per dispositivi mobili sono interessanti dal punto di vista teorico e concettuale, ma per programmare servono a poco. Ha quindi deciso di sacrificare la parte di reti e approfondire di più la parte di architetture.

3.1 Reti cellulari

Permettono ai dispositivi mobili di comunicare voce e dati. La comunicazione avviene tramite antenne, c'è un dispositivo del telefono che permette di comunicare con un antenna. L'idea è che c'è un'infrastruttura di tante antenne ma ci da l'idea che stiamo comunicando con una sola antenna, esistono varie antenne perchè la copertura di una singola antenna è limitata ma la distribuzione delle antenne nel territorio permette di avere una copertura su scala geografica.

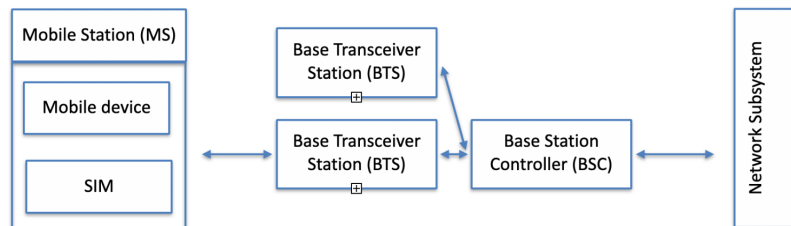
Le tecnologie sono molte (e poco interessanti), sono nate negli anni 80 e si sono evolute molto rapidamente come generazioni (ora c'è il 5G), più o meno 1 ogni 10 anni. Poi ci sono varie tecnologie specifiche all'interno di una generazione (GSM; GPRS), le varie tecnologie nelle varie generazioni condividono la stessa architettura, chiaramente l'hardware è cambiato negli anni.

3.1.1 Architettura

In ambito delle reti si parla di mobile station, si chiama così perchè si tende ad enfatizzare che è composta da due componenti:

- Telefono
- SIM: componente hardware che contiene tutta una serie di dati (chiavi crittografiche) che ci permettono di effettuare la comunicazione

La mobile station comunica in modalità wireless che è detta Base Transceiver Station (BTS) che è l'antenna, la BTS è composta dalle antenne più altro hardware per controllare il traffico. Ci sono tante BTS sparse per il territorio che a gruppi sono connesse ad un Base Station Controller (BSC) che gestisce diverse BTS (decine o centinaia). La BSC a sua volta comunica con il Network Subsystem.



3.1.2 Celle

Un concetto molto importante da capire è il concetto di cella. Il principio che vale è che più le due antenne si allontanano più debole è la potenza del segnale, sappiamo che oltre alla distanza ci sono altri fattori che influenzano la conversazione (ostacoli) che impediscono al segnale di propagarsi come atteso e fanno decadere la potenza del segnale. Quindi in linea di massima la potenza del segnale decresce con la distanza ma possono esserci molti altri ostacoli. Possono anche esserci casi in cui un certo ambiente rallenta la velocità con cui il segnale viene perso (in un corridoio il segnale rimbalza). Quando una mobile station è connessa ad una BTS percepisce una certa potenza del segnale e c'è uno spazio del territorio in cui un dispositivo mobile percepisce una certa antenna A con potenza più alta rispetto alle altre è la cella di A. Possiamo dividere il territorio in celle, ogni cella è uno spazio in cui un antenna è percepita più forte rispetto alle altre. Una cella è circa circolare, in realtà per questioni tecniche si tende a fare celle a spicchio di torta.

3.1.3 Utilizzo del canale

La comunicazione avviene via onde radio. Ci sono vari intervalli di frequenza delle varie tecnologie dove avviene la comunicazione. Le tecnologie usano più intervalli di frequenza. Ogni intervallo di frequenza è diviso in portanti, ad esempio con GSM abbiamo 248 portanti nell'intervallo attorno ai 900Mhz, dato che ci servono due reti per ogni utente (upload e download) possiamo avere collegati 124 utenti alla stessa cella. Quindi usiamo tecniche di FDM (frequency division multiplexing): suddivido una portante in tante sottofrequenze più piccole e TDM (time division multiplexing): è un modo per usare la stessa frequenza tra più utenti suddividendo la frequenza in blocchi temporali, si comunica uno alla volta per un intervallo di tempo stabilito. Grazie a questi trucchi il GSM parte dalle 248 portanti riesce ad avere 1000 canali (500 download e 500 upload). Quando abbiamo una comunicazione su rete cellulare il limite è la distanza dalla cella (la distanza dipende dalla tecnologia), ma l'altro limite è che non possiamo avere un numero di utenti infiniti in una stessa antenna perchè il numero di portanti è limitato.

3.1.4 Handover

C'è una parte del protocollo di comunicazione che ci interessa. Abbiamo detto che le mobile station sono connesse ad una cella, cosa succede quando da una cella passiamo ad un'altra cella? c'è una parte del protocollo che gestisce l'handover, ci permette che quando cambiamo cella la mobile station non si accorge che abbiamo cambiato cella, ad esempio l'indirizzo IP viene lasciato lo stesso in modo che l'utente possa continuare la comunicazione. Questo non succede quando cambiamo rete (da wifi a rete cellulare).

3.2 Le architetture

Le applicazioni per dispositivi mobili fanno quasi sempre parte di un sistema distribuito, quasi sempre la comunicazione avviene con un server, può succedere che comunichi anche con altri dispositivi mobili, ma c'è un server che fa da tramite. Ci sono tre fattori che dobbiamo tenere in considerazione quando progettiamo un protocollo e sappiamo che una delle componenti sarà un dispositivo mobile:

- Nonostante i dispositivi mobili siano stati progettati per una connessione persistente, in realtà non la hanno, spesso un dispositivo mobile non ha connessione
- Può cambiare la connessione, ci cambia l'indirizzo IP
- I dispositivi mobili vogliono eseguire in background il meno possibile, meglio non farlo proprio, dobbiamo tenere conto delle risorse dei dispositivi mobili quando progettiamo un protocollo di comunicazione per dispositivi mobili.

3.2.1 Connection-oriented

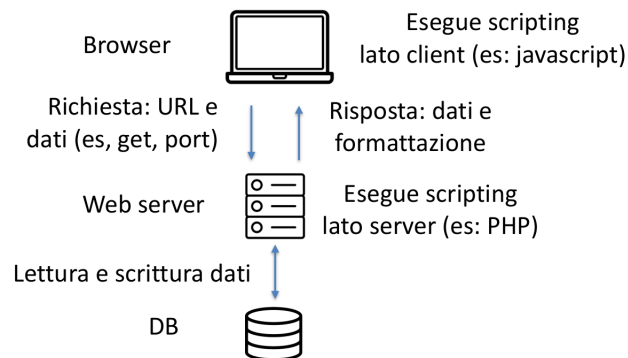
Un approccio molto comune era sviluppare protocolli connection oriented, ad esempio una connessione TCP è connection oriented significa che una volta aperta la connessione rimane aperta. In connessioni connection oriented per dispositivi tradizionali comunque consideriamo il caso in cui cade la connessione, ma è molto raro. Nei dispositivi mobili invece capita molto spesso, quindi forse è meglio usare protocolli connection less. Connection less significa che non tengo la connessione aperta, non tengo aperto il canale di comunicazione, un protocollo connection less è HTTP in cui un client fa una richiesta al server che risponde quando può.

3.2.2 Come si progetta l'architettura di un sistema

La prima cosa da fare è capire quali sono le componenti coinvolte, possono essere 2 ma spesso sono più di 2. Quali dati scambiano le componenti e quali sono le API che ogni componente espone verso gli altri, solitamente il client non espone nessuna API perchè è un casino per il server aprire connessioni con un client mobile perchè l'indirizzo IP e la porta cambiano.

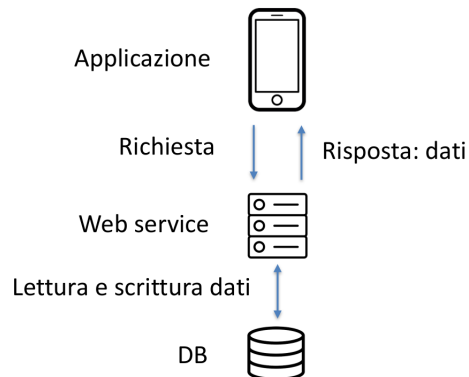
3.2.3 Architettura three-tier per il web

Abbiamo il browser, il server e la base di dati. Quando facciamo una richiesta al server comunichiamo con HTTP e indichiamo con l'IP del server qual'è la risorsa che vogliamo e possiamo anche passargli dati con il GET o il POST. Il server può eseguire codice PHP (che è una porcheria perchè stampa solo HTML) e può eventualmente fare richieste al DataBase.



3.2.4 Architettura three-tier verso applicazioni

Se togliamo il browser e ci mettiamo una applicazione mobile tutto funziona. Il server si chiama web service, noi mandiamo una richiesta HTTP indicando il nome della pagina... il server esegue dello script, ma non rispondiamo più con i dati e la loro formattazione (HTML + CSS), ma rispondiamo soltanto con i dati perchè come visualizzarli lo decide l'app perchè questa ha del codice.



3.2.5 Esempio login

Di fatto fare un web service è più semplice di un web server, perchè non abbiamo tutta la parte di formattazione dei dati.

```
//alcune righe omesse
$username = $_POST['username'];
$password = $_POST['password'] ;//ho semplificato questa riga per
l'esempio
$result = pg_query_params($dbConn, 'SELECT * FROM public.user WHERE
name = $1 AND password = $2', [$username, $password]);
if( pg_num_rows($result) == 1 ) { //se c'è un utente con uid e pwd..
    $id = generateSessionId(); //questo metodo è definito nel codice
    //alcune righe omesse
    echo $id; //← questo è quello che viene ritornato al client
} else { //username e password sono errate!
    http_response_code(400); //viene ritornato un errore al client
    die("INVALID CREDENTIALS");
}
//altre righe omesse
```

3.2.6 Confronto

Solitamente i dati vengono mandati tramite strutture dati complesse come JSON o XML. Quando sviluppiamo delle appa, è l'app stessa a decidere come mostrare l'informazione: richiede al web service solo i dati. Tipicamente codificati con XML o JSON.

```
<html>
<body>
  <h1>Uovo fritto</h1>
  <h3>Ingredienti:</h3>
  <ul>
    <li>1 Uovo</li>
    <li>Olio</li>
    <li>Sale</li>
  </ul>
  <h3>Procedura:</h3>
  <p>Scaldate un filo
    d'olio in una pentola.
    Quando è caldo aggiungete
    l'uovo, facendo attenzione
    a tenere integro il tuorlo.
    Quando l'albume diventa bianco
    aggiungete il sale e servite.
  </p>
</body>
</html>
```

Uovo fritto

Ingredienti:

- 1 Uovo
- Olio
- Sale

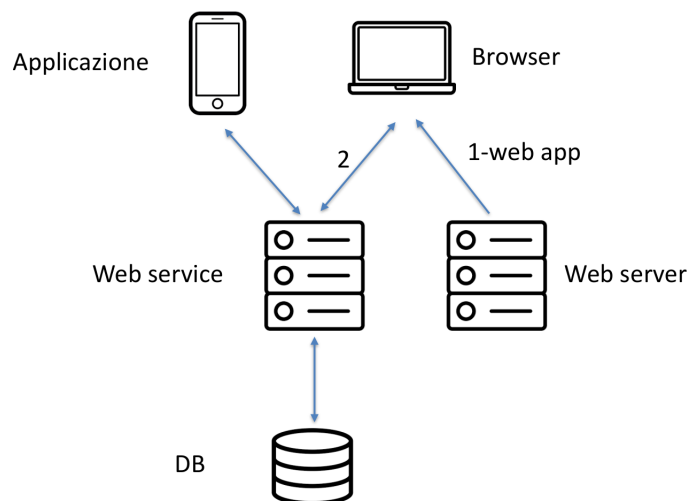
Procedura:

Scaldate un filo d'olio in una pentola. Quando è caldo aggiungete l'uovo, facendo attenzione a tenere integro il tuorlo. Quando l'albume diventa bianco aggiungete il sale e servite.



3.2.7 Sistemi web+app

Spesso abbiamo delle applicazioni che i sistemi che dobbiamo sviluppare hanno una versione in mobile ed anche in pagina web. Quello che viene fatto solitamente è fare un'unica comunicazione e scaricare l'intera applicazione (per evitare di continuare ad aprire e chiudere la connessione per ogni pagina), di fatto sto facendo una sola pagina web che mi rappresenta tutta l'applicazione, ho fatto una web app (sono tutte le pagine con cui l'utente deve interagire e poi uso JS per avere un comportamento più dinamico). Di fatto si ha sul browser un app che conosce tutto il comportamento della web app e che quando ha bisogno di avere i dati li chiede al web service. A questo punto il web service è lo stesso per l'applicazione browser e per l'applicazione.



3.2.8 La definizione del protocollo di comunicazione

Come lo implementiamo questo protocollo? quello che facevamo in base di dati il protocollo era per ogni pagina che devo mostrare ho una richiesta HTTP.

Ora non è detto che per ogni pagina ho una richiesta. Solitamente quello che consiglia se siamo noi a progettare app e web service partiamo dall'applicazione e per ogni pagina dell'app ci chiediamo che dati ci servono e da lì andiamo a raffinare. E' necessario definire le API che il server deve esporre al client. Le API devono permettere al client (per dispositivi mobili) di:

- Scambiare i dati con il server
- Ottimizzando il volume di dati scambiati, tenendo conto dei vincoli dei dispositivi mobili e permettendo una buona esperienza utente

Il protocollo è di tipo connection-less:

- Le chiamate avvengono con connessioni diverse
- Ma solitamente sono logicamente legate le une alle altre

Esempio: supponiamo di voler implementare Twitter in cui abbiamo una bacheca, abbiamo del codice sul client che deve fare richieste al server, dammi bacheca io sono id, il server ci risponde con faccina, nome e twit di una persona. Non è ottimizzato perchè se abbiamo tanti twit di una stessa persona ci rimanda x volte la stessa foto, quindi potremmo fare rispondere con una volta solo la foto di tutte le persone per non dover rimandare la foto ogni volta. Oppure potremmo richiedere la foto dell'utente la prima volta che lo vedo, in questo modo avremmo due API, una per richiedere la bacheca ed una per le immagini del profilo. Questa soluzione non funziona nel momento in cui l'utente aggiorna la foto, perchè il client continuerebbe a mostrare le foto vecchia, possiamo introdurre il concetto di versione, sul server ogni utente ha una foto associata al numero di versione, quando la aggiorna il numero di versione viene cambiato, quando facciamo una get mandiamo anche una picture version, in modo tale che il client possa verificare se ha l'immagine aggiornata. Abbiamo complicato un pochino il protocollo per ottimizzare il protocollo iniziale. Non ci ha spiegato questo perchè sia un protocollo importante, ma per capire cosa significa implementare un protocollo di comunicazione tra client e server.

3.2.9 Esplorazione API

Ci può capitare nella vita di progettare tutto il sistema (client e server) o ci capiterà di scrivere il client usando API scritte da altri, per quanto le API sono documentate malissimo. C'è un passaggio da fare prima di scrivere il codice client, prendiamo delle applicazione (postman) che ci permettono di dire quale pagina chiamare e quali parametri passare e ci fanno vedere la risposta, è comodo per capire come funzionano bene le API.

[esempio applicazione](#)

3.3 Codifica delle informazioni

Partiamo dal presupposto che le chiamate HTTP trasmettono testo semplice, spesso però vogliamo codificare questo testo semplice delle informazioni aggiun-

tive, non dobbiamo inventarcelo noi, usiamo le codifiche standard (JSON, XML, Protocol Buffer). Qual'è il motivo principale per cui non bisogna inventarsi codifiche ad hoc, perchè per JSON e XML esistono delle librerie che ci rendono più semplice la conversione di dati, tipo da JSON a classe, questa operazione si dice marshalling, l'inverso un-marshulling. Abbiamo tutti una serie di metodi che ci permettono di fare queste operazioni in modo facile. Ad esempio in Java esistono delle librerie, JSONOBJECT, che è una classe che mi rappresenta un oggetto JSON, quindi creiamo un oggetto JSONOBJECT e poi su quell'oggetto chiamiamo un metodo `.toString()` per trasformarlo in stringa.

```
public class Researcher {
    ...
    public String toJSONString() throws JSONException {
        JSONObject researcher = new JSONObject();
        researcher.put("name", this.getName());
        researcher.put("surname", this.getSurname());
        JSONArray publications = new JSONArray()
        for (Publication p : this.getPublications()) {
            JSONObject publication = new JSONObject();
            publication.put("title", p.getTitle());
            publication.put("year", p.getYear());
            publications.put(publication);
        }
        researcher.put("publishedPapers", publications);
        return researcher.toString();
    }
}
```

Nella deserializzazione ho un costruttore in modo da creare un'istanza di un oggetto a partire da un JSON.

```
public class Researcher {
    ...
    public Researcher(String jsonString) throws JSONException {
        this();
        JSONObject input = new JSONObject(jsonString);
        this.setName(input.getString("name"));
        this.setSurname(input.getString("surname"));
        JSONArray array = input.getJSONArray("publishedPapers");
        for (int i=0; i<array.length(); i++){
            JSONObject current = array.getJSONObject(i);
            String title = current.getString("title");
            int year = current.getInt("year");
            PublishedPaper pp = new PublishedPaper(title, year);
            this.getPublishedPapers().put(pp);
        }
    }
}
```

In realtà c'è un modo ancora più automatico (in JS è ancora più facile) in

Java c'è una libreria che si chiama GSON ([documentazione](#)) che di fatto fa a convertire in automatico un oggetto JSON in un suo equivalente oggetto della struttura dati. Abbiamo una classe con una serie di attributi ed un costruttore vuoto, dobbiamo creare un istanza dell'oggetto creare un istanza dell'oggetto GSON ed usare un metodo .toJson(sul nostro oggetto) che è in grado di vedere come è fatta la classe e creare un oggetto JSON con i dati che aveva l'oggetto creato in precedenza, ovviamente esiste anche il metodo in verso per deserializzare un oggetto JSON in una classe. La differenza tra oggetto JSON e di programmazione è che quello di programmazione ha stato e comportamento, mentre quello JSON ha solo stato (però possiamo creargli comportamento convertendo JSON a oggetto di programmazione).

```
class BagOfPrimitives {                                // ==> json is {"value1":1,"value2":"abc"}
    private int value1 = 1;
    private String value2 = "abc";                      // Deserialization
    BagOfPrimitives() {                                BagOfPrimitives obj2 = gson.fromJson(json,
        // no-args constructor                        BagOfPrimitives.class);
    }                                                    // ==> obj2 is just like obj
}

// Serialization
BagOfPrimitives obj = new BagOfPrimitives();
Gson gson = new Gson();
String json = gson.toJson(obj);
```

3.3.1 Codifica dei dati binari

XML e JSON sono delle codifiche in testo semplice, ma non tutti i dati che vogliamo mandarci sono in testo semplice (come un immagine), immaginiamo che vogliamo mettere in un tag JSON un dato binario può fare confusione, allora c'è un modo molto comune che si usa quali sempre per codificare dati binari, il BASE64, la cui idea è dividere la stringa in blocchi da 6 bit (64 valori possibili per rappresentali) ed associa a ciascuno di questi valori un carattere. Esistono numerose librerie per la conversione da/verso base64. Questo succede solo quando dobbiamo mandare dati binari, quando mandiamo testo semplice lo mandiamo così e basta.

Bit pattern	0	1	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0	1	1	0	1	1	1	0
Index	19					22					5					46								
Base64-encoded	T					W					F					u								

3.3.2 Protocol Buffer

Protocollo definito da GOOGLE per serializzare informazioni, risolve il problema di XML e JSON che sono molto verbosi (mandano un sacco di caratteri

che alla macchina non servono). (cita solo che c'è, se siamo curiosi andiamo a vederlo, [pagina web](#)).

3.4 Notifiche Push

E' un problema molto diffuso che ha portato alla creazione di un protocollo che è uno standard per i dispositivi mobili tanto che è stato portato anche come standard nei dispositivi tradizionali. Perché nasce? una delle cose che vogliamo evitare di far fare ad un client mobile è evitare di continuare a chiedere al server se c'è qualcosa che c'è, il problema delle chiamate asincrone che abbiamo visto prima. Ci sono una serie di situazioni in cui gestire tutto, ovvero nel caso in cui la comunicazione non parte dal client mobile ma dal server, come ad esempio la ricezione di un messaggio. Nei dispositivi tradizionali c'erano due soluzioni:

- Polling: il client continua a chiedere al server se c'è qualcosa per lui. Non va bene questa soluzione perchè dobbiamo continuare ad inviare messaggi consumando traffico di rete e risorse computazionali per mandare il messaggio e di conseguenza consuma batteria
- Quando un client si accende comunica con un server dicendo che lui è sveglio, in più il client apre una porta (server socket) dicendo al server di mandare i messaggi a quella porta. Il problema è che il dispositivo mobile non è un server, cambia l'indirizzo IP quando si sposta, ma comunque manda aggiornamenti al server per aggiornarlo sempre di qual'è l'indirizzo e la porta a cui deve mandare i messaggi. Fare questa cosa sui dispositivi mobili è più difficile, perchè comunque tenere una server socket in ascolto consuma risorse computazionali.
- L'altra soluzione è che si tiene una comunicazione aperta, teniamo una connessione TCP aperta. Comunque rimane il problema del cambio di rete e dell'esecuzione di codice in background.

In realtà c'è una soluzione che minimizza gli svantaggi di quello che abbiamo detto, ma che in mancanza di altro cerca di tenere in piedi quello che abbiamo detto alla meno peggio. Si sfrutta una caratteristica del protocollo HTTP, il server ha un tempo di elaborazione delle richieste, si chiama time out, un possibile soluzione che mitiga i problemi è: il client fa una specie di polling, manda una richiesta HTTP ed il server appositamente non risponde subito, quando scade il time out il client rimanda la richiesta. Il vantaggio è che la prima richiesta rimane appesa lato server, quindi nel momento in cui il server ha bisogno di comunicare risponde a quella richiesta. Rispetto al polling è meglio perchè faccio una richiesta in un lasso di tempo più lungo. Però richiede comunque di eseguire in background che deve comunque eseguire, questo abbiamo visto che non è possibile, soprattutto in dispositivi IOS.

C'è una soluzione che è ancora migliore, è lo standard per le notifiche PUSH. Ci sono due protocolli, uno di set up ed uno di invio. Quali sono le componenti:

- Utente
- OS sul device
- App che deve ricevere la notifica
- Push server: server esterno fatto da chi offre un servizio di notifiche push, come quello di Google o Apple...
- App server: server specifico del servizio, ad esempio il server di whatsapp

Noi scriviamo l'app che riceve la notifica e l'app del server. Tutto il resto ci è fornito.

C'è un altro oggetto da definire. Il device token che identifica la coppia applicazione-device. La caratteristica fondamentale del device token è che è un'informazione cifrata che non può essere modificata, la può rilasciare soltanto il push server e solo lui lo può creare, gli altri lo possono usare.

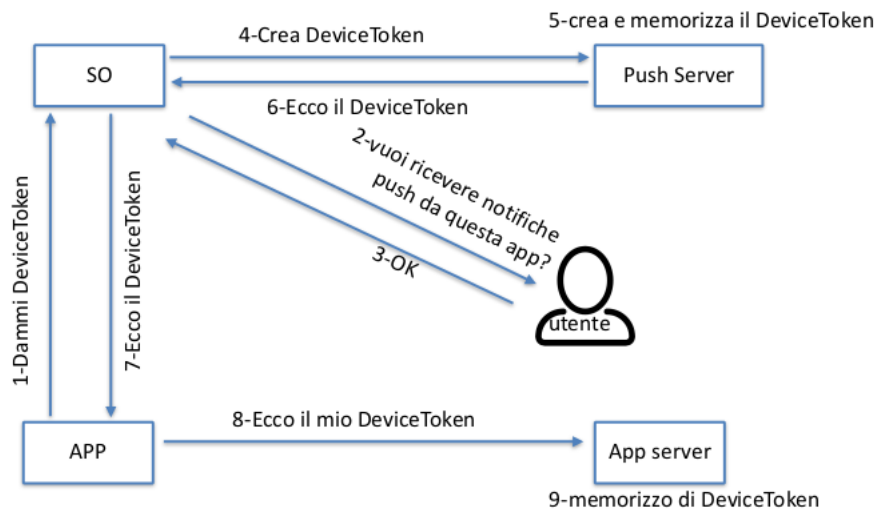
3.4.1 Fase setup

Abbiamo uno schema entità, che dati si scambiano ed un numero per capire l'ordine. Entità:

- OS per dispositivo mobile
- Applicazione
- Utente
- App server
- Push server

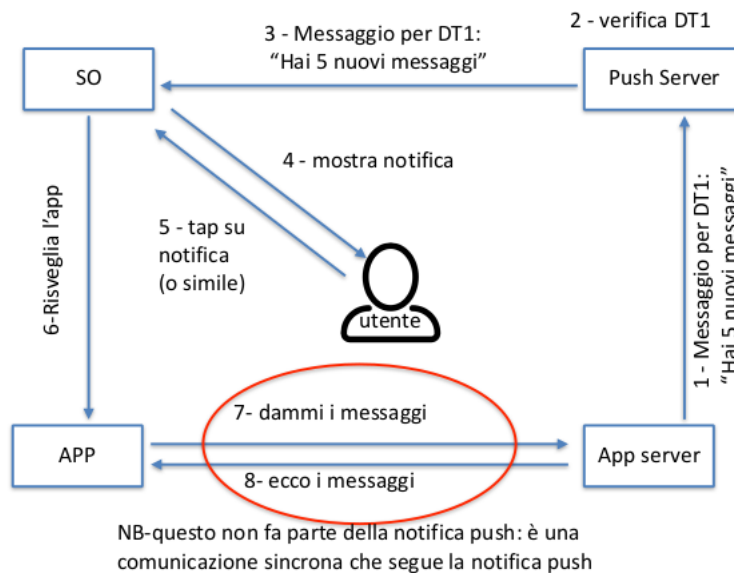
Ad un certo punto quando l'app viene fatta partire la prima volta, quando l'utente vuole usare un nuovo servizio, in modo sincrono rispetto all'applicazione la nostra app capisce che avrà bisogno in futuro di avere bisogno di notifiche push (generalmente fatto alla prima partenza dell'app). Quindi l'app fa una richiesta all'OS richiedendo il device token. Il OS come prima cosa chiede all'utente se vuole richiedere le notifiche push da questa app, se l'utente risponde ok il OS fa una richiesta al push server, che dal punto di vista del dispositivo mobile è una richiesta sincrona, chiedendo il device token per quel dispositivo e applicazione. Il push server crea il device token e lo memorizza, dopo lo restituisce al OS ed il OS a sua volta lo restituisce all'app. Dal punto di vista della nostra app abbiamo una semplice API, è banale.

Il device token che viene ritornato noi lo mandiamo al nostro app server. Vogliamo fare in modo che l'app server conosca il device token che rappresenta questa app in quel sistema operativo. A questo punto il device token rappresenta l'autorizzazione che ha l'app server di mandare messaggi all'app.



3.4.2 Fase di invio

La fase di set up viene fatta una volta e quella di invio N volte. Ricordiamo che l'app server deve mandare dei messaggi all'app ma non vogliamo farlo comunicare direttamente con l'app per evitare il problema dei messaggi asincroni.



L'app server comunica con il push server mandando il device token + un payload che è un piccolo set di dati che è quello che vediamo nell'anteprima dei messaggi, è molto piccolo. Possiamo fare questa chiamata perchè sono due server, è normale che ricevano chiamate asincrone. Quando arriva il messaggio

al push server come prima cosa verifica il device token, che abbiamo già detto che non può essere manomesso, inoltre verifica se quel device token è abilitato a ricevere messaggi e se l'ha fatto lui, se tutto ok comunica con il OS, che è una comunicazione asincrona per il OS, il vantaggio è che abbiamo centralizzato il servizio per ricevere messaggi perchè tante il OS riceve i messaggi push per tutte le app che esegue. Quindi il OS, anche eseguendo in background. Il sistema operativo mostra la notifica all'utente che quando tocca sulla notifica OS risveglia l'app e qua il protocollo del push notification termina, perchè l'app parte e fa in foreground che a questo punto può inviare richieste asincrone all'app server.

Questo concetto è collegato anche alla gestione della memoria, perchè anche se l'app è stata terminata dal OS perchè in background riusciamo a fare comunque arrivare notifiche push anche se in quel momento non esiste neanche un processo nel device.

3.4.3 Attacchi non possibili (o difficili) nel protocollo notifiche push

Perchè le notifiche push ci proteggono dallo spam? appunto perchè esiste questo protocollo che ci tutela, perchè un app server non può inviare notifiche push ad un client che non lo ha autorizzato, perchè non dispone del device token, le notifiche push passano da un server trusted che ci tutela da attacchi di spamming. Anche se otteniamo legittimamente un device token comunque l'utente può disabilitare le notifiche dell'applicazione, che in pratica significa che il OS comunica al push server di disabilitare il device token. Esiste un altro sistema di spamming, se qualcuno riesce ad entrare su un app server e rubare il device token, a questo punto questa persona potrebbe inviare dei messaggi a tutti gli utenti che avevano dato la autorizzazioni a quel server. Ma comunque ha vita breve, perchè non appena il gestore del app server comunica al push server il quale blocca tutti i device token.

Impersonificazione: un app server non può fare finta di essere un altro app server, perchè il device token è associato all'app, il push server se ne accorge. Questo aiuta a prevenire gli attacchi di phishing.

Messaggi indesiderati

Flooding: quando un ente manda una marea di messaggi ad un altro per bloccarlo. E' mitigato perchè il push server blocca l'invio dei messaggi all'OS se ne vede arrivare troppi.

Funziona perchè ci sono due principi:

- Il device token non è modificabile
- Tutto deve passare per un push server che è fidato

Questa questione della sicurezza ci spiega perchè questo protocollo è più complicato rispetto, ad esempio, al protocollo della mail

3.4.4 Notifiche push e locali

Un'altra cosa a cui fare attenzione è che da utenti potremmo confondere le notifiche push con le notifiche locali. Noi possiamo dire alla nostra app di fare arrivare una notifica all'utente, appaiono molto simili alle notifiche push ma in realtà il protocollo sottostante è molto diverso, perchè c'è un API che dice di comunicare una cosa all'utente.

3.4.5 Esercizi

Falli!

Chapter 4

La posizione

La posizione è un dato caratteristico dei dispositivi mobili, in quanto mobili sono in grado di capire la posizione in cui si trovano, si chiama location awareness. Abbiamo vari sistemi di riferimento:

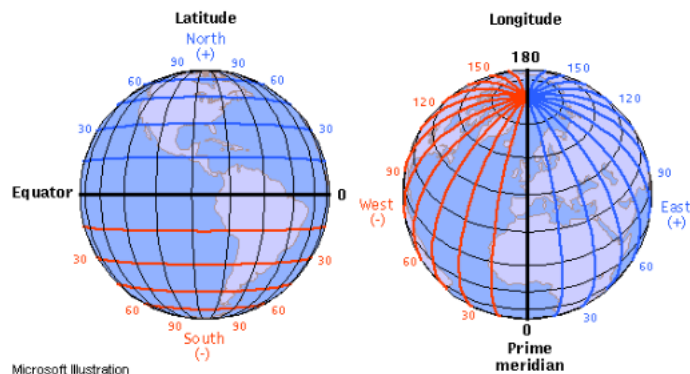
- Rispetto al mondo
- Rispetto ad un edificio

In quante posizioni vogliamo calcolare la posizione di un utente? a prima vista 2 (x,y), in realtà anche in applicazioni di tipo outdoor di navigazione spesso è interessante anche sapere l'altezza, quindi calcolare la posizione in 3 dimensioni. Ancora più importante è l'informazione dell'altezza negli ambienti indoor. Possiamo rappresentare la posizione rispetto a delle coordinate geografiche, locali e la posizione semantica (l'indirizzo a cui si trova l'utente). In realtà vedremo tra poco che le dimensioni sono più di 3, perchè molto spesso ci interessa sapere la posizione dell'utente e l'orientamento, da che parte è orientato l'utente.

I sistemi di riferimento geografico usano uno standard per la rappresentazione della posizione che è il WGS84, che divide il geoide in meridiani ed in paralleli. Noi possiamo numerare meridiani e paralleli con due numeri:

- il meridiano possiamo identificarlo con l'angolo della distanza dall'equatore (0 all'equatore e 90 ai poli)
- Per i paralleli dobbiamo inventarci un riferimento inventato (greenwich)

Si possono rappresentare in gradi e decimali. Esistono delle API come Google location API che ci danno una posizione in JSON.

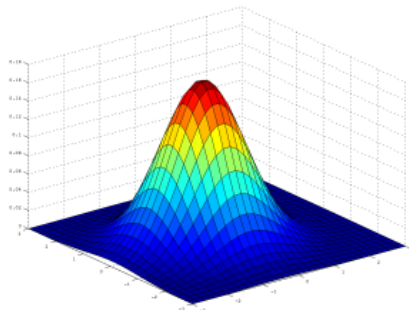


4.1 Tempo

Il tempo è un'altra informazione importante quando parliamo di dati spaziali, infatti si parla di dati spazio-temporali. Per i dispositivi mobili si parla di posizione spazio-temporale e si possono definire delle traiettorie. Una traiettoria è l'insieme di coppie posizione, timestamp.

4.2 Approssimazione

Volendo se volessimo in termini matematici rappresentare la posizione di un utente, premesso che non sappiamo mai che la posizione dell'utente è certa ma c'è un'incertezza, potremmo dire che dato lo spazio x e y la z potrebbe rappresentare la probabilità che l'utente si trovi in quello spazio. Notiamo che è una distribuzione lineare, quindi la somma delle probabilità deve dare 1 (o l'integrale se siamo nel continuo). La verità è che non abbiamo nessuno strumento tecnologico che ci da una roba del genere.



In termini più pratici l'approssimazione viene modellata come un'area in cui con molta probabilità si trova l'utente. Spesso prendiamo un'area (cerchio) e diciamo che con buona probabilità l'utente stà lì, ma non sappiamo internamente all'area la probabilità di dove sta l'utente.

Se volessimo approfondire dovremmo studiare due termini, la precisione è quanto diverse misurazioni della posizione sono vicine tra di loro. L'accuratezza è quanto la posizione è nella posizione vera.

4.2.1 Errore di posizionamento

Nel calcolo della posizione le due metriche di accuratezza e precisione sono scomode da utilizzare perché vanno valutate nella loro combinazione. Una misura più usabile è basata su questo concetto: una posizione calcolata ha una probabilità maggiore di essere trovata in un cerchio di raggio y rispetto alla posizione reale. La CEP mi indica il raggio di un cerchio centrato nella posizione corretta, tale per cui la metà delle posizioni calcolate sono dentro tale cerchio. Spesso si usa una misura 2DRMS che corrisponde ad un CEP con una probabilità del 98%, cioè mi dice che il 98% delle volte la posizione reale si trova entro il 50% del raggio del cerchio che dico.

Il livello di precisione richiesto dipende dal tipo dell'applicazione. In applicazioni indoor tipicamente la precisione deve essere maggiore, perché se l'errore è troppo grande le informazioni risulteranno totalmente inutili.

4.3 Orientamento

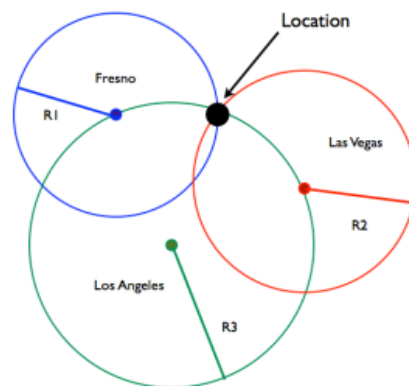
Spesso ci interessa anche l'orientamento dell'utente. Si misura in due modi:

- In quale direzione è direzionato il device (heading)
- In quale direzione si sta muovendo l'utente (course), ovviamente in questo caso ho bisogno di calcolare più spostamenti dell'utente



4.4 Calcolo posizione con trilaterazione

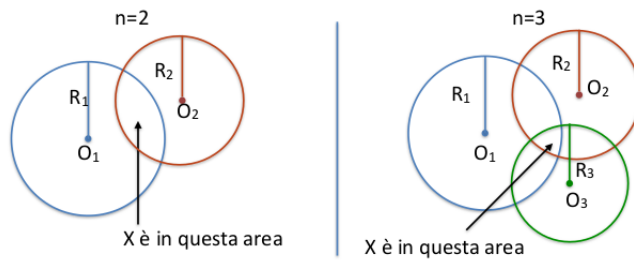
E' interessante introdurre il concetto di trilaterazione, è una tecnica per calcolare la posizione di un oggetto sapendo la distanza rispetto a degli oggetti di cui so la posizione. Trovo la distanza con un sistema di equazioni, se ho due circonferenze tangenti ho un solo punto in cui queste si intersecano, quindi ho bisogno di soli due punti. Ma togliendo casi strani il sistema dovrebbe avere una sola soluzione se ho tre circonferenze, quindi tre equazioni.



Nei casi pratici non è così facile, perchè non è detto che io abbia esattamente 3 di questi cerchi con cui calcolare la distanza, potrei averne di meno o di più. Se ne ho di meno potrei non sapere dove sta l'utente. Se ne ho di più avrò tante circonferenze che si intersecano nello stesso punto, non mi cambia niente rispetto a 3. Solitamente non so la distanza esatta, di solito posso sapere un upperbound della distanza o sapere che è compresa tra un minimo e un massimo. Ho sempre delle misurazioni soggette ad errore.

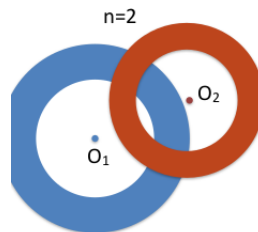
4.4.1 Upper bound della distanza

Se ho solo 1 upper bound della distanza so che l'utente starà nel cerchio, se ne ho di più l'utente starà nelle intersezioni. In questo caso mi interessa averne tanti, perchè più ne ho più posso restringere l'area in cui sta l'utente.



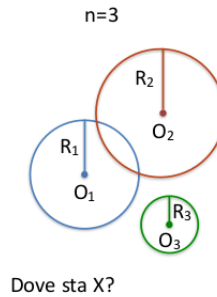
4.4.2 Range di distanze

Spesso succede che so che l'utente si trova in una distanza compresa tra un minimo ed un massimo, se ne ho 2 l'utente sta in quelle corone circolari che si sovrappongono.



4.4.3 Gestione dei dati inesatti

Notiamo che i nostri sistemi non sono precisi. Se matematicamente modellassimo questi cerchi come funzioni troveremmo 0 soluzioni, perchè sarebbe impossibile. Dobbiamo quindi modellare dei dati incerti. In questo esempio potremmo dire che l'utente si trova nell'intersezione dei cerchi O_1 e O_2 e spostato verso il cerchio O_3 .



4.5 Contesti

Spesso la posizione si calcola in modo indiretto: ad esempio l'utente interagisce con un oggetto di cui si sa la posizione. Esistono tecniche di calcolo della posizione outdoor, che sono uno standard e sono molto consolidate. Tecniche indoor invece non sono standardizzate, si possono utilizzare trasmissioni radio (wifi bluetooth), elementi visibili (tecniche visuali di calcolo dello spostamento), utilizzo di sistemi inerziali per il calcolo dello spostamento. Nessuna di queste tecniche è utilizzata da sola, ma la soluzione è ottenuta mettendo insieme varie tecniche.

4.6 Tecniche per il calcolo della posizione outdoor

Vedremo:

- Global Navigation Satellite System (GNSS)
- Rete Cellulare
- WIFI Network Based
- Soluzioni Ibride

4.6.1 Global Navigation Satellite System (GNSS)

La localizzazione satellitare è una delle più usate. Ottenuta grazie ad una serie di satelliti messi in orbita, una delle tecnologie più famose di questo tipo è il GPS. Le caratteristiche dei satelliti:

- Sono in orbita
- Seguono delle traiettorie prevedibili
- Conoscono l'ora in modo molto preciso, hanno al loro interno un orologio atomico
- Mentre sono in orbita continuano a mandare un segnale trasmettendo il proprio ID e l'informazione temporale del momento corrente su quel satellite

A terra abbiamo un dispositivo, lo smartphone, che ha un'antenna in grado di ricevere il segnale dal satellite, siccome possiamo stimare la velocità di propagazione del segnale nell'atmosfera (dato che sappiamo l'ora a cui è partito e l'ora in cui l'abbiamo ricevuto), date queste informazioni posso derivare la distanza, se lo faccio con più satelliti posso fare la multilaterazione e capire la posizione.

In realtà è più complicato di così perchè i nostri device non hanno un orologio atomico e quindi non possono calcolare il delta temporale e stimare la distanza da un satellite. In uno spazio tridimensionale in cui abbiamo anche il tempo se abbiamo 4 satelliti riusciamo a calcolare sia la posizione che il tempo in maniera esatta.

Quali sono le due fonti principali di approssimazione:

- Le interferenze: le condizioni meteo e gli edifici.
- I satelliti seguono delle orbite predefinite però non sono al millimetro sull'orbita, ma potrebbero essere fuori orbita e bisogna correggerla

Esistono tanti sistemi ed i device li usano tutti per raffinare la posizione.

International GNSS Constellations				
NAME	COUNTRY OF ORIGIN	FULLY OPERATIONAL	NUMBER OF SATELLITES	CARRIER FREQUENCIES
GPS	USA	1993	31	L1/L2/L5
GLONASS	Russia	1995	24+	G1/G2
Galileo	Europe	2020	30 (22 current)	E1/E5a/E5b
BeiDou	China	2020	30 (28 current)	B1/B2
QZSS	Japan	2024	7 (4 current)	L1/L2/L5

Galileo

Ci sono vari servizi, come il calcolo della posizione e servizi di emergenza. Tipicamente questi servizi hanno un servizio pubblico a cui tutti possono accedere ed uno privato e cifrato in cui solo alcuni utenti possono accedere per evitare attacchi:

- Spoofing: in cui intenzionalmente mando un segnale sbagliato in modo che il dispositivo calcoli la posizione sbagliata
- Jamming: mando tante informazioni in modo che coprano la banda e non mi arrivi il segnale

Differential GPS (D-GPS)

Sistema che permette di correggere l'errore dovuto alle interferenze dell'atmosfera e ad una non precisa posizione dei satelliti rispetto a quanto previsto. Se ho un ricevitore in una posizione fissa posso capire se la posizione del satellite è sbagliata oppure ci sono delle interferenze, posso farlo perchè se è fissa so quale dovrebbero essere le posizioni corrette.

Assisted GPS (A-GPS)

L'altro problema che emergeva era il calcolo del primo fix di posizione, perchè era molto lungo all'inizio avere la prima posizione. L'assisted GPS risolve questo problema, quando il dispositivo mobile si muove nell'ambiente è connesso ad un'antenna di una rete cellulare, le quali mantengono un elenco di satelliti in vista, comunicano questi satelliti in vista al dispositivo mobile che sa quindi quali satelliti deve cercare perchè probabilmente saranno in vista anche per lui, quindi sa fare il calcolo della posizione iniziale molto velocemente.

4.6.2 Posizione basata su rete cellulare

Quando utilizziamo la rete cellulare solo per il fatto che stiamo comunicando con un'antenna sappiamo dove ci troviamo, se sappiamo l'ID dell'antenna a cui siamo connessi e sappiamo dov'è la cella corrispondente sappiamo più o meno dove siamo. L'approssimazione dipende dalla dimensione della cella, con il 5G abbiamo celle piccole ma comunque rimane una posizione grossolana. Possiamo migliorare il calcolo della posizione:

- Utilizzare il protocollo stesso della rete cellulare, il GSM utilizza tecniche di time division, già questo protocollo ha una tecnica per stimare la distanza del device dall'antenna per evitare collisioni. Però se riesco a stimare la distanza rispetto all'antenna so che sono nella cella e so anche che sono ad una certa distanza dall'antenna. Di fatto fa già parte del protocollo GSM (quindi è comodo da usare) ma comunque la precisione rimane abbastanza bassa.

Enhanced observed time difference

Sono state sviluppate altre tecniche che si basano sull'idea di guardare le antenne attorno a quella a cui sono connesso, se riesco a stimare la distanza dall'antenna a cui sono connesso ed in qualche modo anche la distanza dalle celle attorno posso fare la tecnica della triangolazione. Guardiamo sempre quanto tempo ci mette il segnale a viaggiare per poter calcolare la distanza. Questo migliora la precisione, è nell'ordine dei 50 metri. I problemi sono: il device mobile deve poter ricevere il segnale da diverse antenne e non sempre è possibile, la tecnica è lenta (alcuni secondi), in linea di principio la potenza del segnale diminuisce in maniera lineare con la distanza nella realtà ci sono anche gli ostacoli che la fanno decadere, questo crea dei problemi al tempo di propagazione del segnale che rende falsata il calcolo della distanza.

4.6.3 WIFI positioning

L'intuizione potrebbe essere questa: se con la rete cellulare posso in qualche modo utilizzare tecniche di trilaterazione rispetto a delle antenne, con il wifi ho un sacco di antenne in più, posso calcolare la distanza rispetto alle antenne con la potenza del segnale e poi calcolo la posizione usando la trilaterazione. Per fare questo ci serve calcolare :

- La distanza del device dagli Acces Point
- La posizione degli Acces Point

Distanza da Access Point

Il protocollo 802.11 (alla base del WIFI) prevede una fase di discovery, quindi abbiamo gli access point che comunicano a tutti quelli che stanno attorno il proprio IP, anche ai device che non sono connessi a quella rete.

Così come vale per la rete cellulare, per le reti WIFI abbiamo il problema che la potenza del segnale è approssimativamente proporzionale alla distanza, ma sappiamo che non è esattamente proporzionale perchè posso avere anche delle interferenze.

L'altro problema è che gli Access Point sono numerosi e cambiano in fretta. Quindi introduciamo un concetto che serve per rispondere a questo problema, il crowdsourcing: quando raccolgo dei dati tramite gli utenti di un certo servizio, può essere esplicito in cui l'utente consapevolmente condivide dei dati oppure implicito, ad esempio quando usiamo un servizio di localizzazione sul cellulare collaboriamo alla costruzione di un DB con la posizione degli access point.

Quando andiamo in giro per la città usando un qualunque sistema di navigazione che richiede l'utilizzo di un sistema satellitare stiamo calcolando la posizione in maniera abbastanza precisa e contemporaneamente il nostro OS percepisce quali sono le antenne WIFI nei paraggi e la potenza del segnale per ognuna di queste antenne, queste informazioni sono inviate ad un server (location service) che ricevendo queste informazioni da tanti utenti è in grado di

costruirsi una mappa con le posizioni degli access point. Questo serve perchè quando non abbiamo attiva l'antenna GPS possiamo usare il WIFI per calcolare la posizione. Ma in realtà non è una soluzione solo WIFI, è ibrida perchè comunque ci serve una connessione GPS. Allora in questo caso ci torna utile l'informazione della rete cellulare per raffinare la posizione.

La prima adozione di approccio ibrido è stato fatto da Skyhook, adesso sia Apple che Google hanno il loro location service.

4.6.4 Approccio ibrido

Questo approccio ibrido si fa con 3 tecnologie, quando calcoliamo la posizione il OS comunica con un location service e gli manda tutti i dati che ha disposizione, il quale usa questi dati per aggiornare la mappa e quando abbiamo bisogno di calcolare la posizione possiamo mandargli alcuni dati parziali e lui se riesce ci raffina la posizione.

4.7 Gestione dati di posizione

Quando il device mobile ha calcolato la posizione può processarla lui oppure darla ad un server. In entrambi i casi si possono adottare tecniche per gestire i dati di posizione.

4.7.1 Geocoding e reverse-Geocoding

Con Geocoding si intende la conversione tra indirizzo esplicito numerico e indirizzo satellitare. Il geocoding converte la posizione semantica in coordinate spaziali. L'operazione inversa è il reverse-Geocoding. Questo lo fa il location service.

4.7.2 Geofetcing

E' tipicamente offerta dal OS. A volte può capitare che scriviamo un applicazione e vogliamo fare qualcosa quando l'utente entra o esce da una determinata area che può essere o circolare o rettangolare.



4.7.3 Calcolo delle distanze

Attenzione perchè se pensiamo di calcolare la distanza tra due punti con il teorema di Pitagora ricordiamo che non vale per piani in 3D ma solamente in 2D. C'è una formula di Haversine che assume che la terra sia sferica ma calcola la distanza tra due punti sulla superficie terrestre. Abbiamo il raggio della terra, due punti definiti con latitudine e longitudine.

Calcolo distanze su rete stradale

La distanza su rete stradale la usiamo spesso tramite API dei map service. Come fa Google Maps a calcolare questo, la rete stradale è modellata come un grafo rappresenta ad ogni intersezione un nodo (un incrocio) e gli archi sono pesati con la distanza tra i due nodi. Come peso dell'arco possiamo usare quello che vogliamo, metri, tempo di percorrenza in macchina...

Quali tecniche si usano per calcolare la distanza, il problema algoritmico è short paths problem, di cui questo [algoritmo](#) dobbiamo fare un approfondimento obbligatorio.

4.7.4 Trattamento dei dati spazio temporali

L'altra questione che si pone quando vogliamo trattare dati spazio temporali è che vogliamo farlo in maniera efficiente. E' difficile farlo, perchè indicizzare dati che sono una coppia di indice. Gli indici in un database servono per ricercare delle istanze, tramite l'indice riesco a tenere ordinate le informazioni e riesco a fare la ricerca in tempo logaritmico e non lineare.

In caso di dati spazio-temporali, non posso fare un indice per la posizione ed uno per il tempo, perchè i dati mi servono insieme.

Quindi a volte devo calcolare alcuni servizi e farlo in modo efficiente non è facile, esempi di servizi:

- Quanto ci metto ad arrivare al lavoro con le condizioni di traffico attuali?
- In caso di incidente, l'automobile segnala automaticamente la posizione ai soccorsi piu' vicini
- Dov'è il benzinaio piu' vicino?
- Quali sono i ristoranti nel raggio di 2Km?
- Quale dei miei amici è nelle vicinanze?
- Per quante persone questo negozio è il piu' vicino?

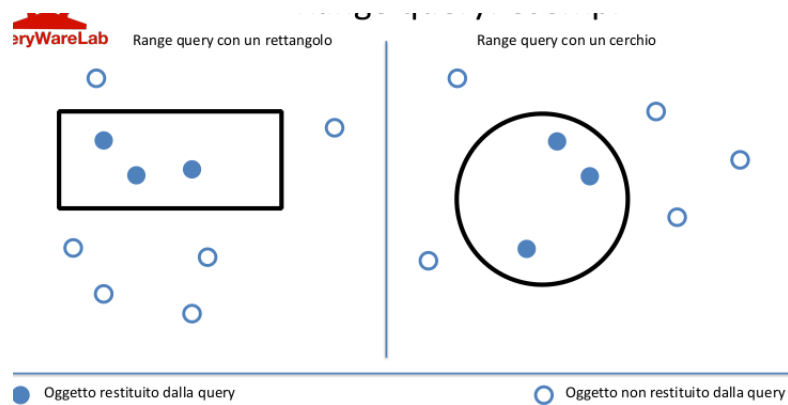
Tutti questi problemi sono risolti da database che implementano degli attributi spaziotemporali che ci permettono di fare delle query facili su quegli attributi. Per PostgreSQL c'è PostGIS che gestisce dati spaziali come punto, segmento, circonferenze. Esistono anche degli operatori specifici che possiamo

usare per calcolare query particolari con dati spazio temporali. Inoltre abbiamo funzioni particolari.

Poi ci sono 3 query spaziali che sono implementati nei DBMS spaziali, che servono perchè se riusciamo a mappare il problema su queste query tutto diventa facile.

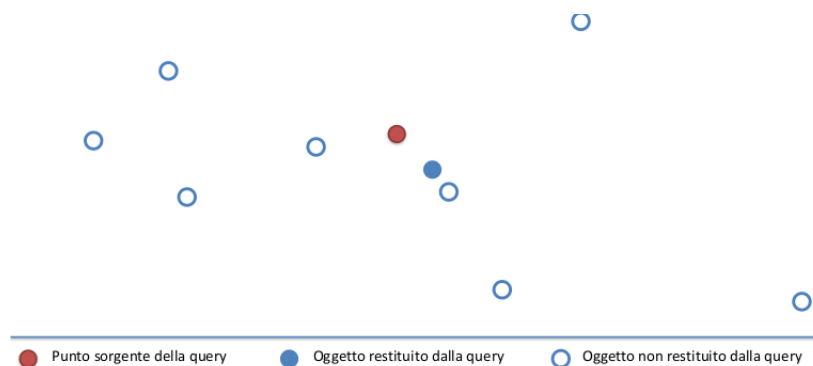
Range query

Ritorna gli oggetti all'interno di un'area, che tendenzialmente è un cerchio o un rettangolo.

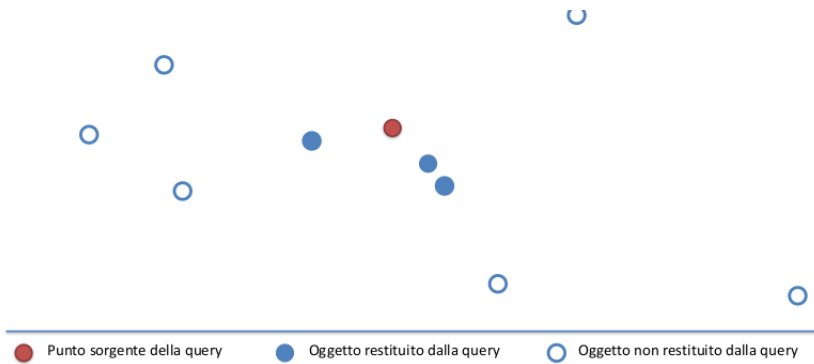


Nearest Neighbors (NN)

Dato un insieme di punti ed una posizione mi ritorna il punto dell'insieme che è il più vicino tra quelli selezionati.



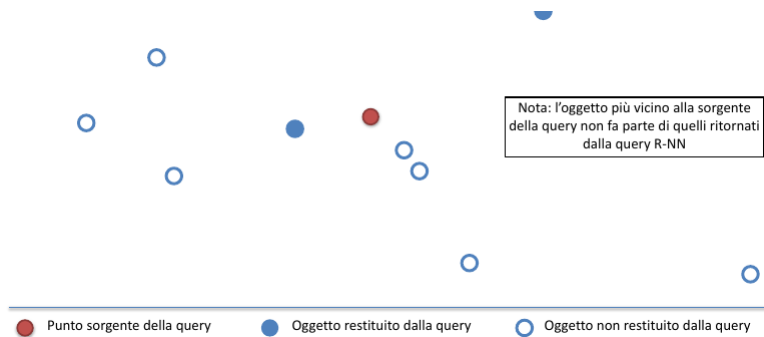
Esiste anche un'estensione che ritorna i K elementi più vicini alla posizione specificata.



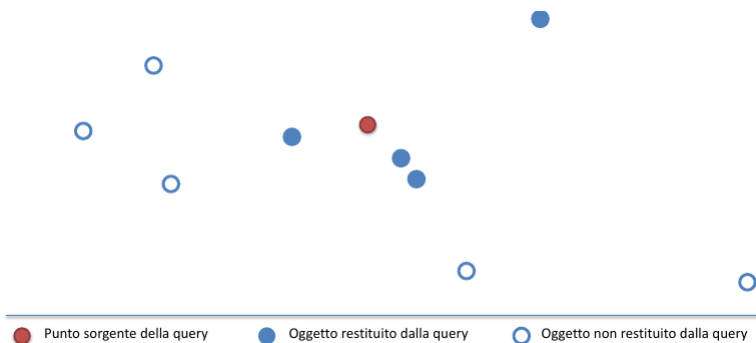
Reverse Nearest Neigh (RNN)

Ricordiamoci questo esempio: pensiamo al caso in cui voglio costruire un supermercato, ci interessa sapere quante case avrebbero quel supermercato come quello più vicino. Non è quel supermercato quale case ha più vicino ma le case che hanno quel super vicino.

Dato un insieme di punti P e un punto q di P , una reverse nearest Neighbors che avrebbero il punto q come il punto più vicino.



Esiste anche in questo caso l'estensione a K , ad esempio 2.



Qual'è la complessità computazionale

La Range e Nearest è lineare. La RNN ha una complessità quadratica. Esistono degli indici spazio temporali che permettono di fare queste query con complessità minore. Questi indici si chiamano [R-Trees](#), in cui abbiamo uno spazio (albero) e man mano che scendiamo nell'albero copriamo un area sempre più piccola in modo tale che se vogliamo fare una Range Query possiamo scorrere nell'albero andando rapidamente a restringere lo spazio e lì dentro possiamo scorrere.

Chapter 5

Analisi

Le prossime lezioni riguardano le quattro fasi principali dello sviluppo software. La parte di User Experience è importante perchè il successo dell'applicazione non dipende dall'ottimizzazione algoritmica ma è determinata appunto dalla User Experience. Facciamo un esempio motivante per tutto il corso di laurea, una piccola società con una ventina di dipendenti è stata comprata per 19 miliardi di dollari, era WhatsApp che è stata comprata da Facebook. Quando è stata comprata esistevano anche altre app di messaggistica, ma perchè WhatsApp è stata valutata così tanto e le altre 0? la fase di registrazione nelle altre applicazioni era complicata ed anche la fase della aggiunta degli amici era complicata, WhatsApp usa il numero di telefono, di fatto non facciamo la registrazione. Dietro WhatsApp non c'è nessun algoritmo complicato, hanno vinto perchè hanno cambiato la User Experience. Riuscire a progettare una User Experience vincente è la prima cosa che permette che la nostra applicazione abbia successo.

5.1 Contesto d'uso di app per dispositivi mobili

L'analisi è la fase di sviluppo del software che noi studiamo di meno, perchè tipicamente quando facciamo un progetto ci vengono dati i requisiti del software. Ma spesso quando dobbiamo lavorare su un'applicazione dobbiamo metterci noi a fare l'analisi.

Quali sono le differenze rispetto ai dispositivi tradizionali:

- Contesto: l'interazione con l'utente può avvenire mentre l'utente è concentrato a fare altro. Ad esempio il navigatore funziona mentre l'utente sta guidando.
- L'utente potrebbe essere in mobilità
- L'utente potrebbe non guardare lo schermo

5.2 Perchè si usano le app mobile

Le applicazioni si usano perchè risolvono problemi. Se affrontiamo l'analisi da questo punto di vista siamo sulla buona strada per trovare una soluzione. Esempio: il prof è stato chiamato dall'ufficio marketing di EXPO perchè volevano fare un'app il cui obiettivo era quello di dare visibilità agli sponsor, ma non funziona così, perchè se l'app è stata pensata per gli utenti deve risolvere un problema per l'utente.

Inoltre i dispositivi mobili sono sempre accesi e pronti all'uso

5.3 Durata delle sessioni d'uso

Tipicamente nei dispositivi tradizionali abbiamo una sessione d'uso che dura un'oretta. Mentre nei dispositivi mobili abbiamo sessioni d'uso estremamente

brevi. Va considerato. C'è un videogioco che ha progettato sessioni d'uso brevissime, angry birds, perchè il contesto di dispositivo mobile ci porta a giocare mentre ad esempio stiamo aspettando l'autobus.

Category	Apps	Avg. usage	Exemplary Apps
unknown	4,823	36.37 sec	
Finance	307	37.01 sec	Mint.com Personal Finance, Bank of America, Google Finance, iStockManager
Travel	782	48.72 sec	Google Maps, Yelp, Wikia
Communication	881	46.92 sec	Google Mail, Handcent SMS, K-9 Mail
Productivity	1,062	61.49 sec	Calendar, Evernote, GTasks
Shopping	326	61.71 sec	Market, Barcode Scanner, Craigslist
Social	538	62.69 sec	Facebook for Android, Twitter, TweetDeck
Sports	385	65.98 sec	Zillow Fantasy Football '10, ESPN ScoreCenter, NFL Mobile
News	784	68.11 sec	NewsRabbit, reddit is fun, BBC News
Settings	1	68.71 sec	Default Settings App
Browser	10	74.01 sec	Default browser, Skyfire browser, Dolphin browser
Entertainment	84	76.90 sec	IMDb Movies & TV, TV Guide Mobile, PhotoFania
Multimedia	130	82.79 sec	Pandora Radio, Music, Camera
Comics	3,242	91.33 sec	DailyStrip, KoolView, Dilbert Mobile
Games	2,822	134.25 sec	Angry Birds, Wordfeud FREE, Solitaire
Health	424	153.80 sec	CardioTrainer, Sleep Bot Tracker Log, Baby ESP
Lifestyle	956	167.77 sec	DailyHoroscope, Gentle Alarm, Epicurious Recipe
Reference	764	176.28 sec	Kindle for Android, Aldiko Book Reader, Audible
Tools	3,004	206.26 sec	AppBrain App Market, Apps Organizer, Google Goggles
Themes	1,061	258.28 sec	Zone Home, Fingerprint Screenshot, HomeChange
Libraries & Demos	240	274.23 sec	Google Services Framework, default Updater, Motorola Updater, Bubbles Demo, Ride Logger Demo, ES Task Manager

Le sessioni hanno durata minore negli orari in cui gli utenti sono intenti in altre attività

5.4 Interazione con l'utente

Le peculiarità dei dispositivi mobili ci rendono necessario ripensare molti aspetti tipici delle applicazioni tradizionali:

- Tempi di apprendimento: se abbiamo bisogno di spiegare all'utente come si usa l'applicazione l'abbiamo progettata male. Se proprio dobbiamo farlo in un tempo brevissimo ed in modo interattivo.
- Tempi di set/up e registrazione al servizio: niente schermata di registrazione! ci ammazza qualunque servizio. O fare registrazione implicita o farlo attraverso servizi di terze parti (Oauth). Ma comunque facciamo fare la registrazione dopo che facciamo usare l'app.
- Tempi di utilizzo: non tutte le applicazioni tendono a limitare i tempi di utilizzo quelle che guadagnano dalle pubblicità tendono a massimizzare i tempi.

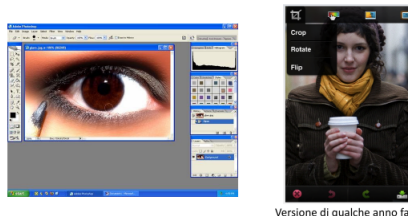
L'utente il testo non lo legge, non scriviamo niente.

5.5 Funzionalità

Uno degli obiettivi della fase di analisi è definire le funzionalità dell'applicazione. Il nostro approccio allo sviluppo dell'applicazione deve essere quello di rilasciarla con poche funzionalità, dobbiamo individuare il minimo sindacale delle funzionalità che l'app ha bisogno (3 al massimo) per essere caratterizzante e rilasciarla così. Perchè così l'app viene rilasciata subito e utilizzata da subito e quando

la rilasciamo ci rendiamo conto se gli utenti usano quell'applicazione o no. E' a questo punto che inizia il lavoro, perchè analizzando chi sono gli utenti e quali sono le loro esigenze che capiamo cosa serve e man mano aggiungiamo funzionalità.

Tipicamente nei dispositivi mobile le applicazioni hanno poche funzionalità, esempio di photoshop. Ci sono delle eccezioni, iniziano ad esserci applicazioni per dispositivi mobili ad uso professionale che quindi hanno tante funzionalità e lunghi tempi di apprendimento.



Versione di qualche anno fa

5.6 Progettazione della user experience

Riguardiamo dei concetti chiave della user experience. La user experience è ciò che una persona prova quando usa un prodotto, è un'esperienza prettamente soggettiva. Include vari aspetti come esperienza nell'uso, utilità, semplicità d'uso. Spesso confondiamo user experience con usabilità che è quanto il sistema è semplice da utilizzare ed è UN elemento della user experience. Comprende anche quanto un utente si diverte ad usare l'applicazione, quanto è gratificato. E spesso la confondiamo con la progettazione delle interfacce utente, spesso la UX passa attraverso un insieme di fattori (come presento le informazioni, in che ordine le presento). Durante il corso di interazione di interazione uomo macchina abbiamo studiato l'emotional design.



Definizione UX: un'app deve avere:

- Le funzionalità richieste
- Funzionare come previsto
- Essere usabile dall'utente (senza richiedere sforzo eccessivo), lo sforzo può essere di apprendimento ma anche di uso
- Deve dare soddisfazione all'utente: non è una cosa molto definita, dipende dall'applicazione cosa vogliamo misurare

Per misurare l'usabilità si usa il SUS (System Usability Test) in cui si sottopone ad un test l'utente e ci sono delle domande a cui deve rispondere, si applica una formula al punteggio ottenuto e sulla base del risultato della formula si capisce se l'applicazione è usabile o meno.

5.7 Obiettivi dell'analisi

- Comprendere il dominio applicativo: nella nostra carriera andremo a sviluppare applicazioni in domini che non conosciamo affatto. Per fare una buona applicazione, che risolva un problema, è importante conoscere il dominio in cui dovrà operare.
- Comprendere gli attori coinvolti e le loro necessità: gli utenti e gli stakeholder
- Comprendere il problema: se non comprendiamo il problema non possiamo trovare una soluzione, spesso non è l'utente che ci dice qual'è il suo problema, ma siamo noi che parlando con l'utente dobbiamo definire bene il problema.
- Definire i requisiti funzionali e non funzionali
- Quanto valore potrebbe avere il sistema per l'utente? Quanto vale quell'applicazione in termini di soldi oppure quanta pubblicità è disposto a guardare l'utente

5.7.1 Comprensione del dominio applicativo

Ci troveremo a creare sistemi in domini che non conosciamo. Serve comprendere questi contesti. Si parte cercando di interagire con tutti gli stakeholder. Può essere molto utile studiare i sistemi e processi attuali perchè molto spesso i sistemi attuali sono analogici e noi vogliamo digitalizzarli. Secondo l'esperienza del prof la vera difficoltà di fare un'applicazione è fare la parte di analisi (lui parla dei medici, perchè sono molto esperti del loro dominio e fanno difficoltà a spiegarti le loro scelte ed i loro problemi). Molto spesso gli utenti ti coinvolgono per un problema ma poi ti accorgi che i problemi su cui veramente intervenire sono altri.

5.7.2 Gli attori

Un altro passaggio semplice da individuare ma cruciale è capire chi sono gli attori del sistema. Spesso noi abbiamo in testa come attore del sistema solo l'utente finale (ad esempio in un sistema che mostra il menù di un ristorante noi pensiamo che l'utente sia solo il cliente, ma ci sono anche i camerieri, il proprietario...). Quando abbiamo un sistema in cui i contenuti sono creati da un addetto come attore sicuramente c'è anche lui. Spesso in quasi tutti i sistemi è più complicato di come l'abbiamo pensata, ad esempio possono essere coinvolti utenti business, i moderatori, un amministratore di sistema...

Conosci gli utenti

Capire cosa vogliono fare gli utenti è una cosa difficile, perchè anche loro spesso non lo sanno. Potremmo progettare un applicazione avendo un'idea di come gli utenti dovrebbero utilizzarla ed alla fine non la usano come avevamo pensato. Dobbiamo fare l'analisi dell'applicazione avendo in mente già di rimetterla in discussione. Quando sviluppiamo l'applicazione dobbiamo inserire del codice apposta per capire cosa fanno gli utenti che serve per capire che funzionalità usano gli utenti (come google analytics).

Esempio

iMove è un'applicazione sviluppata da loro che serve a persone con disabilità visive come supporto per muoversi in città. Per progettare hanno parlato con gli utenti. Funziona così: l'utente fa partire un'applicazione, quest'app quando cammini ti dice periodicamente a che indirizzo sei e cosa c'è nei paraggi. L'utente può impostare ogni quanto essere avvisato. Hanno rilasciato l'applicazione e poi raccolto dei log di cosa facevano gli utenti, realizzando un sistema che permetteva di tenere traccia di cosa ha fatto un utente. Hanno scoperto che sono un terzo dell'applicazione usava l'app per come l'avevano pensata, vuol dire che nell'analisi aveva portato ad escludere 2 terzi degli utenti. Un terzo usa l'app aprendola al volo per sapere l'indirizzo a cui è e poi la chiude, l'altro terzo usa solo la funzione che restituisce la lista dei negozi nelle vicinanze e poi la chiude.

Si sono accorti del problema e hanno rifatto la fase di analisi, ridefinendo i problemi e ri-progettando il sistema.

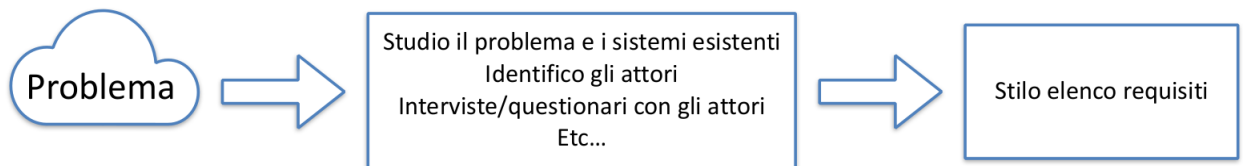
5.7.3 Comprendere i problemi

Ci sono una serie di metodologie per definire come fare emergere argomenti importanti, come ad esempio usare dei questionari per fare emergere cosa interesserebbe di più. Esistono anche interviste semi-strutturate in cui abbiamo una linea guida per svolgere l'intervista ma poi possiamo cambiarla in base a come gli intervistati rispondono. Esistono anche tutta una serie di metodologie su come andare a analizzare le interviste ed i questionari. L'obiettivo finale dell'analisi è definire i requisiti, che possono essere:

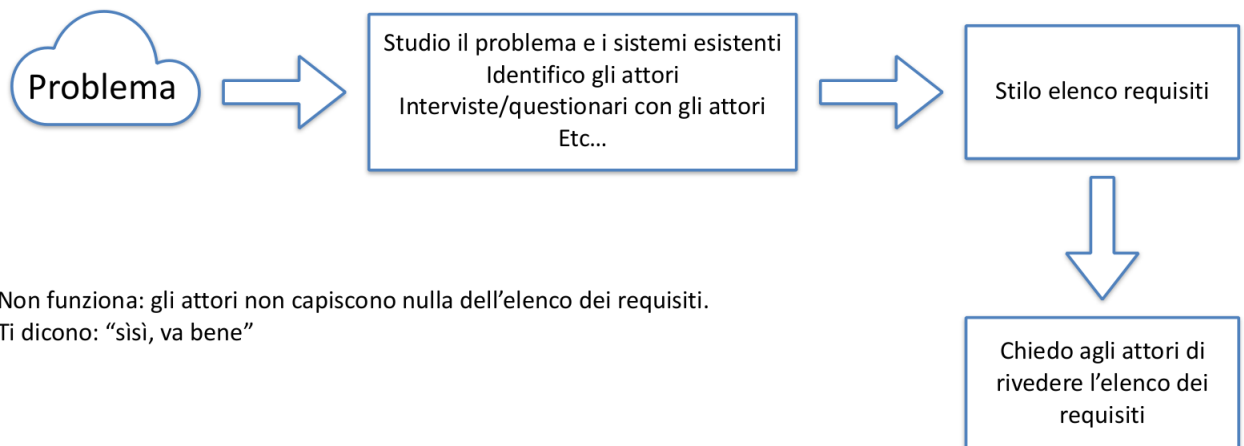
- Funzionali: cosa deve fare il sistema, ad esempio deve postare un contenuto testuale o deve permettere di mettere un "like" ad un post di un altro utente
- Non funzionali: quali caratteristiche deve avere il sistema, ad esempio per chi deve essere accessibile oppure il sistema deve incentivare il medico a completare il lavoro di annotazione

5.7.4 Quale metodologia di analisi?

L'approccio base è partire dal problema, capire cosa fanno gli utenti, che problemi hanno e stilare l'elenco delle funzionalità.



Questo nella maggioranza dei casi non funziona, perchè non abbiamo un riscontro dagli utenti, potrebbe essere che le funzionalità che abbiamo implementato non sono quelle effettivamente utili agli utenti. Fare vedere agli utenti l'elenco dei requisiti che ho trovato non è una cosa utile perchè non lo sanno se serve o no.

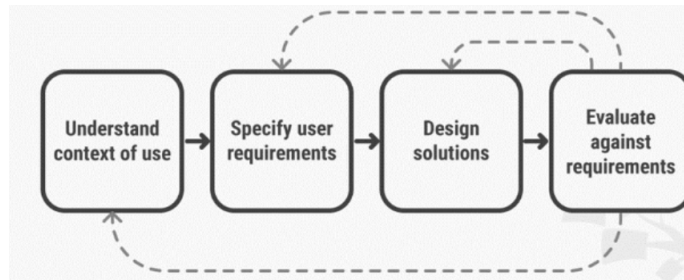


Non funziona: gli attori non capiscono nulla dell'elenco dei requisiti.
Ti dicono: "sì, va bene"

L'approccio funzionale è fare vedere agli utenti come funzionerà il sistema, dobbiamo passare per la fase di progettazione, quindi l'idea è fare dei prototipi e in maniera graduale farli vedere all'utente. Perchè non possiamo sviluppare il sistema, perdendo un sacco di tempo, per poi sentirci dire dagli utenti che il sistema non va bene.

5.8 User Centered Design

A dispetto del nome, non è una metodologia esclusivamente per la fase di progettazione (design), ma per l'intera applicazione. Ci si focalizza sull'utente e sulle sue esigenze in ogni fase dello sviluppo. E' un processo iterativo.



5.8.1 Prototipo

In questa visione c'è una cosa che è fondamentale e spesso sbagliamo a concepire, il concetto di prototipo. E' ingenuo pensare di costruire il sistema senza commettere errori, dobbiamo pensare di fare pesare poco gli errori che commettiamo, non possiamo pensare di toglierli completamente dall'applicazione. Riusciamo a fare pesare poco gli errori arrivando subito alla fase di valutazione da parte degli utenti, attraverso la realizzazione di prototipi, che potrebbero anche non essere funzionanti, ad esempio facendo uno schizzo su carta della schermata. L'obiettivo del prototipo è capire se stiamo procedendo bene con la comprensione del problema e la progettazione del sistema. Devono costare poco, perchè se abbiamo sbagliato lo dovremo buttare.

Wizard of Oz

E' una tecnica per la prototipazione, creiamo un sistema che apparentemente funzioni come quello che vorremmo realizzare ma che funziona solo in casi particolari e non implementa veramente le funzioni, ma sono simulate da qualcuno.

5.9 L'attitudine

Il programmatore inesperto ha l'attitudine di scrivere subito codice e di avere l'idea che la progettazione che abbiamo in testa è quella giusta, è una cosa sbagliata. La cosa da fare è mettersi con un foglio/lavagna e mettersi a disegnare le interfacce e le interazioni. Una soluzione per capire i problemi di analisi è fare vedere le schermate a qualcuno che non è un informatico.

Anche se non possiamo fare vedere il prototipo all'utente perchè non abbiamo tempo, quantomeno ricordiamoci di guardare le schermate con gli occhi dell'utente (esempio app IO).

Nello sviluppo di un sistema complesso dobbiamo sviluppare una funzionalità alla volta e testare subito il mercato per capire se gli utenti serve. Fail Fast, la maggior parte delle startup falliscono, meglio fallire in fretta piuttosto che aver perso un sacco di tempo

Chapter 6

Progettazione

Per quanto abbiamo detto che non dovrebbe esserci una netta separazione tra la fase di analisi e di progettazione, am distinguiamo quello che è l'obiettivo della progettazione, ovvero come vogliamo fare quello che abbiamo identificato nella fase di analisi. L'aspetto centrale è la progettazione della user experience, non è ovviamente l'unica cosa da progettare, ma come abbiamo visto in precedenza tutti gli altri protocolli (reti, sistemi operativi...) vengono adattati per la user experience. Cosa possiamo produrre:

- Documentazione tecnica:
 - Architettura del sistema: possiamo definire quali sono le componenti coinvolte e le API che espongono verso le altre componenti
 - Progettazione della struttura interna del codice. Vedremo alcuni pattern di programmazione
- Schema di navigazione tra le schermate
- Progetto della GUI (interfaccia) delle schermate

Prima progettiamo lo schema di navigazione e successivamente le singole schermate. Per la progettazione della GUI delle schermate dobbiamo anche andare ad indicare dei comportamenti che non sono grafici

6.1 Principi di progettazione

Vediamo una serie di principi/idee generali/consigli/best practice di progettazione che ci possono aiutare a migliorare la User Experience. E' chiaro che è una parte molto aleatoria, non è un teorema matematico ma può aiutare ad evitare molti errori.

6.1.1 Pensa all'utente

Se posticipiamo la progettazione a quando programiamo siamo distratti da mille cose tecniche a cui l'utente non interessa nulla. Vediamo un esempio tratto dall'esperienza del prof, avevano derivato la progettazione della User Experience dalla progettazione tecnica SBAGLIATO!. Avevano quest'app che serviva a delle persone non vedenti per distinguere il contenuto di due barattoli, l'app scansava il codice a barre e mostrava un elenco dei provider che hanno le informazioni sui prodotti e selezionato il provider mostrava la lista di prodotti associati. Ovviamente all'utente non interessa nulla del provider.

6.1.2 Content Prioritization

L'idea è di dare all'utente prima i contenuti e le funzionalità più importanti permettendo di accedere ai contenuti e le funzionalità meno importanti in un secondo momento. Dobbiamo capire prima cosa interessa all'utente.

6.1.3 Less is more

Quando lavoriamo alla nostra interfaccia grafica dobbiamo capire se possiamo togliere qualcosa, perchè meno contenuto diamo all'utente più semplice è l'interazione tra l'utente e l'app. Un chiaro esempio è quello che ha fatto Google con la barra di ricerca e basta. Un altro esempio del prof è l'app che fa un suono in base all'intensità della luce, quest'app non fa niente, si accende e non puoi interagirci. Gli utenti si concentravano sul fatto che era immediata e senza fronzoli.



6.1.4 Integrità estetica

Le nostre applicazioni devono riflettere, attraverso la grafica, la natura dell'applicazione stessa. Ad esempio un applicazione per la produttività dovrebbe avere un aspetto serio, semplice, lineare. Mentre un applicazione ludica dovrebbe avere maggiore spazio alla grafica ricercata, divertente.

6.1.5 Consistenza

- Consistenza esterna: consistenza rispetto alle altre applicazioni, spesso alcuni per fare fare qualcosa all'utente si inventano qualcosa di strano, ad esempio per aggiornare la pagina si scorre dall'alto verso il basso. Ci sono tutta una serie di strumenti con cui l'utente è già abituato ad interagire che non ha senso proporre qualcosa di diverso. Quando individuiamo un problema di interazione, guardiamo cosa fanno le applicazioni più note per vedere cosa fanno
- Consistenza interna: l'applicazione è consistente con se stessa? Servono gli stessi termini, la stessa grafica e gli stessi comportamenti per indicare gli stessi oggetti e task

6.1.6 Personalizzazione

Vogliamo cambiare il modo in cui l'utente ha fatto qualche cosa, possiamo farlo se la nostra applicazione ha dei requisiti particolari che lo impongono.

6.1.7 Affordance

E' la caratteristica di un oggetto di suggerire a un individuo la possibilità di utilizzo. Un esempio è quello di windows phone in cui in alcune schermate c'era un pezzo a destra come se fosse tagliato, che suggeriva il fatto che potevamo scorrere per vedere cosa c'è dietro.



6.1.8 Metafore

Da quando esiste il desktop, c'è una scrivania sulla quale ci sono delle cartelle e dentro ci metto i file. Una rognia nelle applicazioni è la localizzazione, adattare un'applicazione ad una cultura ed una lingua diversa. Se troviamo delle metafore che vanno bene in tutto il mondo queste possono aiutarci a portare l'applicazione che è valida anche per altre culture.



Sergio Mascetti - Mobile Computing



6.1.9 Minimizzare l'input

Forse noi ce ne rendiamo conto meno perchè i dispositivi mobili sono già maturi. Bisogna permettere all'utente di fare il meno sforzo possibile, per esempio dobbiamo evitare che l'utente inserisca del testo.

Questo non vale solo per il testo ma per tutte le componenti dell'interfaccia, ad esempio le dimensioni, oggetti di interfaccia troppo piccoli sono difficili da visualizzare. Apple definisce delle linee guida la dimensione minima degli elementi di interfaccia. Anche le scritte devono essere della dimensione appropriata affinché tutti le possano leggere.

C'è tutto un filone di ricerca che dice qual'è la fatica di andare a selezionare le varie parti dello schermo quando utilizzo il device con una sola mano.



6.1.10 Importanza della prima impressione

Gli utenti perdono pochissimo tempo per decidere se un applicazione è utile o meno. Per esempio se non è ovvio dobbiamo chiarire cosa fa l'app, con un breve esempio. Cerchiamo di posticipare la parte di registrazioni e login, se proprio dobbiamo farla, facciamola con servizio di terze parti. Posticipiamo anche la parte di richiesta di permessi, ad esempio se all'inizio non serve la posizione non chiediamo i permessi subito, ma quando servono veramente.

6.2 Progettazione delle interfacce

Le interfacce grafiche hanno un ruolo importante nella progettazione della User Experience. Come abbiamo già visto l'interazione con il dispositivo mobile è cambiata, soprattutto è data dal touchscreen, ma abbiamo anche i sensori come l'accelerometro, il giroscopio oppure la fotocamera.

L'uso del touchscreen ha portato ad una serie di gesti standard. Non dimentichiamo che nelle nostre applicazioni possiamo usare dei gesti che vadano oltre al tocco sul pulsante, possiamo anche generare dei gesti in più rispetto a quelli standard per gestire casi particolari. Esempio TypeInBraille in cui hanno usato dei gesti ad hoc per fare scrivere in braille.

6.2.1 Dimensione dello schermo

Nei dispositivi tradizionali le dimensioni dello schermo possono variare in maniera anche importante, ma tipicamente il layout delle interfacce grafiche cambia mod-

eratamente. Nei dispositivi mobili variano, ma nel caso di questi dispositivi si progetta un'interfaccia grafica ad hoc.

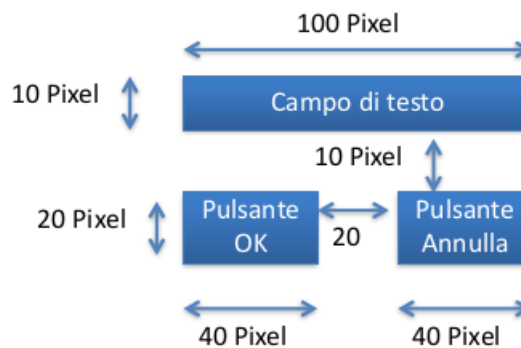
6.2.2 Orientamento dello schermo

Nei dispositivi mobili siamo abituati al fatto che lo schermo può essere ruotato.



6.2.3 Layout a dimensioni assolute

Questo modo di progettazione non si adatta alla dimensione dello schermo. Inoltre se ho uno schermo con una definizione molto alta (definizione = densità di pixel per pollice) e definisco un oggetto di interfaccia tramite pixel quella roba sarà larga pochissimo in centimetri.



6.2.4 Unità di misura delle dimensioni sullo schermo

Supponiamo che vogliamo definire un oggetto grafico in termini della sua dimensione in centimetri, oppure dire che non deve essere più piccolo di tot. Il prob-

lema di questa cosa è che se cambiamo la risoluzione del monitor quell'oggetto diventa piccolo. Hanno introdotto delle misure per ovviare a questi problemi:

- Density Independent pixel, DIP: 1 pixel con una densità di 160dpi. Di fatto un DIP è una unità di misura di lunghezza che corrisponde ad 1/160 di pollice. Cambia la dimensione di un oggetto in base alla risoluzione dello schermo.
- Scale Independent Pixel: stessa cosa di un DIP ma per le scritte

6.2.5 Responsive design

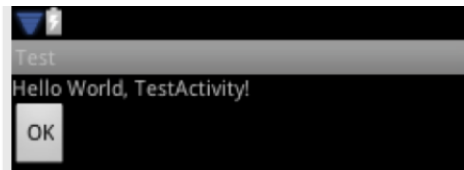
E' una tecnica di design finalizzata a creare interfacce che si adattano a schermi con diversa:

- Dimensione
- Risoluzione
- Densità di pixel

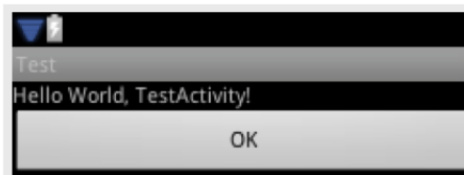
I layout "liquidi"

L'idea alla base è quella di non definire le dimensioni assolute degli oggetti, ma si definiscono in base alle dimensioni di altri oggetti (fammi questo pulsante largo quanto lo schermo, o come quanto l'oggetto che lo contiene).

```
android:layout_width =  
"wrap_content"  
android:layout_height=  
"wrap_content"
```



```
android:layout_width =  
"match_parent"  
android:layout_height=  
"wrap_content"
```



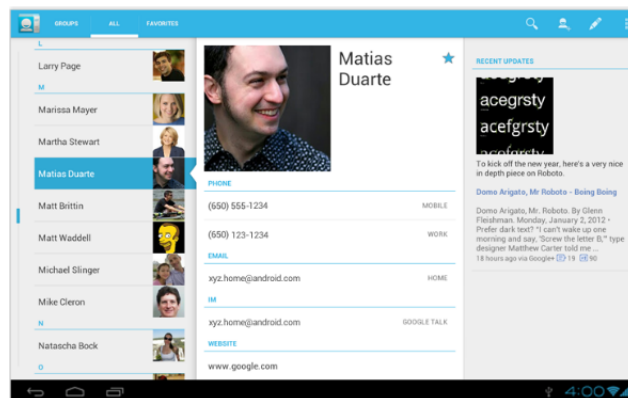
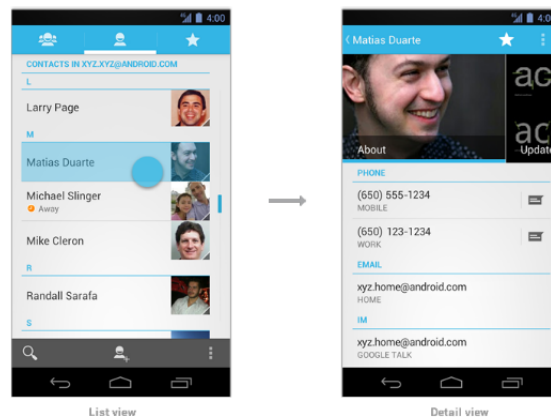
Ri-definizione delle schermate

La presentazione delle interfacce per dispositivi mobili deve avere due ragionamenti:

- Cercare quanto più possibile di avere un solo codice che funzioni bene per tutti i dispositivi

- A volte il primo punto è impossibile perchè a volte devo cambiare la UX per schermi diversi.

Per esempio in android possiamo definire dei layout che vengono automaticamente utilizzati quando valgono certe condizioni.



6.2.6 Navigazione

La navigazione tra le schermate può avvenire in due modi:

- per task
- in modo sequenziale

Lo schema di navigazione definisce come l'utente si muoverà tra le varie schermate. Non è necessario definire il dettaglio grafico di ogni schermata, basta indicare le funzionalità. Le funzionalità più importanti/frequenti devono essere presentate il prima possibile all'utente. La prima schermata mostrata all'utente può variare: primo avvio/avvii successivi.

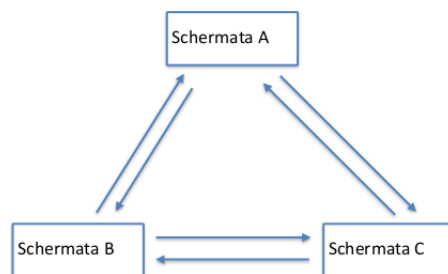
Sequenziale

Da una schermata mi sposto ad un'altra e posso tornare indietro. Si chiama sequenziale perchè una sequenza di schermate e posso andare avanti ed indietro.



Per task

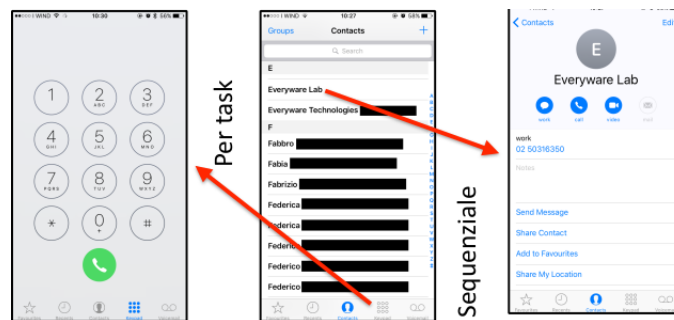
Perette di muoversi da un task ad un altro in qualunque momento, come i menù. La navigazione sequenziale viene implementata principalmente con la barra di navigazione. Invece esistono vari strumenti di interfaccia per implementare la navigazione per task.





Navigazione combinata

La navigazione avviene quasi sempre concatenando sequenziale e per task.



Vedere la documentazione di [Android](#) e [IOS](#)

6.2.7 Progettazione delle singole schermate

Dobbiamo definire i singoli elementi di interfaccia. Quali elementi di interfaccia utilizziamo? volendo potremmo definire solo oggetti di interfaccia nuovi, tipicamente vogliamo comporre l'interfaccia come composizione di elementi già esistenti. Possiamo utilizzare:

- [Android design](#)
- [Human interface guideline](#)

Volendo potremmo utilizzare nuovi elementi di interfaccia, solo che è abbastanza raro.

6.2.8 Prototipi

Prima di sviluppare l'interfaccia facciamo dei prototipi. Prima su carta, non affoghiamo la nostra creatività in strumenti tecnici. Un altro modo molto rapido

è usare delle slide. In realtà una volta che abbiamo fatto una progettazione su carta c'è un app, POP, che permette di creare un prototipo facendo delle foto del disegno su carta e dicendo schiacciando un pulsante vai di là.

Lo step successivo è uno strumento di disegno, possiamo farlo in due modi:

- Autocad, photoshop
- Applicazioni apposite di progettazione:
 - Balsamiq
 - Figma

L'ultima cosa da fare è usare i tool di sviluppo, ad esempio in Android Studio c'è un tool in cui prendi i pulsanti e li metti nell'applicazione.

6.3 Progettare la struttura del codice

Quando scriviamo applicazioni da poche centinaia di righe di codice ci serve progettare la struttura del codice? tendenzialmente no, perchè ce lo ricordiamo, possiamo ricordarci tutte le funzioni che abbiamo scritto e cosa fanno. Quando scriviamo un programma molto più lungo non possiamo ricordarci tutto, abbiamo bisogno di schematizzarlo. Questo schema serve per non perderci nel codice, inoltre se lavoriamo in gruppi è molto comodo avere uno schema di quello che stiamo facendo per comunicare con gli altri componenti del gruppo. L'altro motivo è la manutenzione del software, quando facciamo un'applicazione nei casi reali dobbiamo nel tempo metterci le mani, se non abbiamo una struttura è un casino. Inoltre molto spesso si usano dei modi per risolvere i problemi che sono comuni, se riusciamo a rimappare un problema (o un suo sottoproblema) su un modo di risolverlo che è comune è molto comodo. Per fare questo usiamo un pattern ed abbiamo un modo già ragionato e veloce per risolvere un problema. Ricapitolando, ci serve definire delle componenti logiche grazie alle quali è più semplice:

- Risolvere il problema, adottando la stessa struttura già usata da altri in casi simili in precedenza
- Distribuire il lavoro tra vari membri del team di lavoro
- Valutare la correttezza delle singole componenti
- Correggere eventuali errori
- Mantenere il codice nel tempo

Perchè ci serve progettare la struttura del codice, un altro aspetto è la verificabilità del codice, se abbiamo organizzato bene il codice abbiamo alcuni strumenti che permettono di verificare se il codice funziona.

6.3.1 Pattern di progettazione (design pattern)

E' molto facile quando si parla di progettazione che si parli di concetti astratti. Vedremo dei pattern di progettazione, quando parliamo di pattern indichiamo un problema che si ripete in diversi progetti con una sua soluzione standard. Dobbiamo distinguere tra due famiglie di pattern:

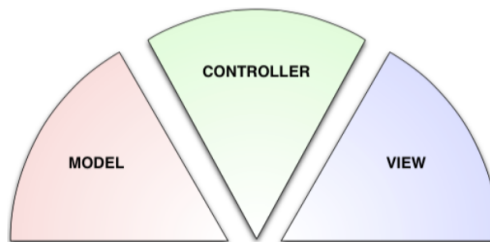
- Puntuali: risolvono problemi abbastanza puntuali del codice (observer, è molto specifico e va adattato al contesto)
- Architetturali: non ci dicono come risolvere un problema, ma come risolvere un'applicazione (ad esempio MVC)

Noi vediamo quelli architetturali, perchè la progettazione di applicazioni con interfacce fa ampio uso di pattern architetturali.

6.3.2 MVC: Model, View, Controller

Fino a 2/3 anni fa era IL pattern architetturale. E' un pattern architetturale che suddivide la nostra applicazione in 3 componenti, ognuna con una funzionalità specifica e che comunicano tra loro mediante interfacce:

- Model
- View
- Controller



Model

Serve per rappresentare e memorizzare le strutture dati specifiche dell'applicazione. Definisce le operazioni sui dati ai quali le altre componenti possono accedere. Spesso si occupa di:

- Creare oggetti dalla memoria persistente (in realtà anche in memoria secondaria va bene)
- Salvare gli oggetti
- Serializzare e de-serializzare i dati

Un esempio è in un client per un social network avere la classe di un model "friend" che rappresenta i dati di un contatto.

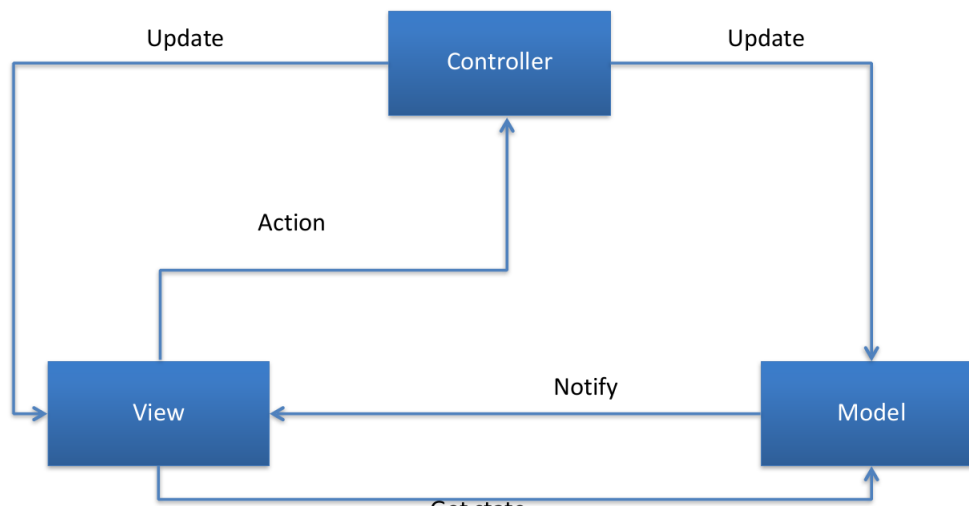
View

E' la componente che visualizza i dati, è l'interfaccia con cui interagisce l'utente, esempi di classi che compongono la view possono essere pulsanti, liste, label che si possono utilizzare così come sono proposte oppure estenderle. Tipicamente la view ci richiede pochissime righe di codice, perchè utilizziamo delle componenti già create da altri (usiamo un pulsante scritto da altri). Volendo si può usare la parte di view per creare un oggetto di interfaccia personalizzato.

Controller

Fa da intermediario tra il view ed il model. Gestisce il ciclo di vita dell'applicazione (partenza, background, terminazione..) e gli eventi generati dall'utente tramite la view (utente schiaccia il pulsante) e modifica di conseguenza il model.

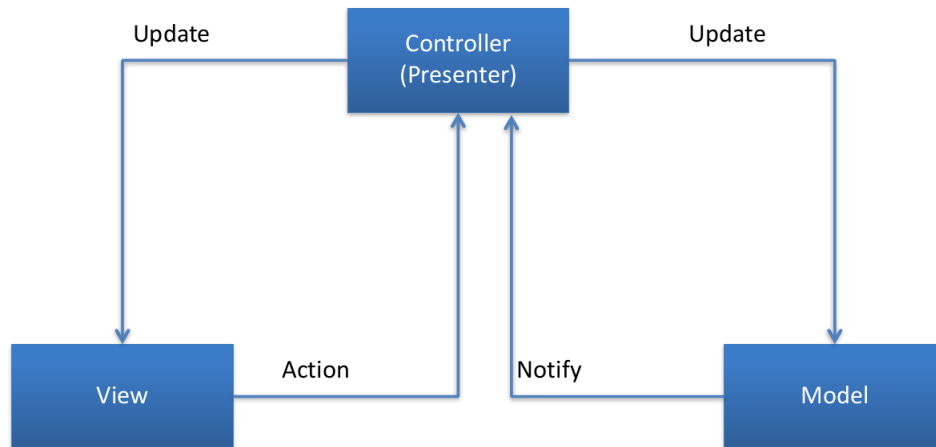
Struttura originale di MVC



Uno dei limiti è che il flusso di informazione varia parecchio, non è così scontato qual'è la comunicazione? perchè tutti possono comunicare con tutti.

6.3.3 MVP: Model, View, Presenter

E' una variante del MVC, apple lo chiama MVC. La differenza è che il Model e la View non possono comunicare tra di loro, ma devono passare per il controller/presenter.



6.3.4 Uso MVC

Bisogna favorire la separazione tra le componenti, nessuna sa come sono implementate le altre. Le componenti comunicano tra loro solo tramite delle interfacce.

6.3.5 Vantaggi MVC

Sono stati utilizzati per tanto tempo come pattern architetturale. Da una struttura al codice, che si può ritrovare in tutte le applicazioni con GUI. E' utile per comunicare con i colleghi. Migliora:

- La leggibilità
- La riusabilità: possiamo riusare le view cambiando il dominio applicativo oppure la struttura dati rimane uguale (una classe)
- La mantenibilità
- La testabilità

6.3.6 Limiti di MVC

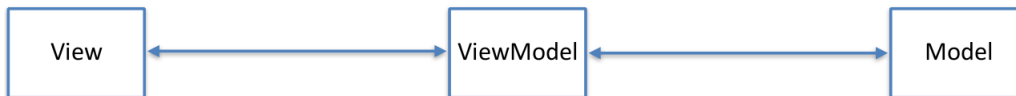
Come abbiamo detto MVC prevede una separazione netta tra le tre componenti che però raramente si riesce a realizzare. In realtà quasi sempre si ricorre a componenti combinate, sia in Android che in IOS c'erano delle classi che erano componenti combinati di view e controller, ad esempio la classe view-controller è un controller delegato alla gestione di una schermata che presenta la view e gestisce gli eventi.

Inoltre un altro limite, che il prof ha notato nella sua esperienza, è che MVC fa fatica ad adattarsi ad applicazioni con più schermate. Il problema è la gestione di tante view.

6.3.7 MVVC: Model, View, ViewModel

Negli ultimi anni c'è un altro pattern di progettazione che sta soppiantando MVC. Si è diffuso con windows phone. L'idea è concettualmente simile ad MVP, suddivide i compiti in maniera diversa. L'idea è che la view mostra i dati e gestisce gli eventi di interfaccia (ha aggiunto la gestione degli eventi alla view di MVC), il model rappresenta i dati, il viewModel prepara i dati che devono essere mostrati dalla view. L'idea è che il viewModel mi ritorna i dati già formattati in modo che devo solo mostrarli. Abbiamo semplificato la view ed abbiamo diviso le operazioni.

In MVC si arriva spesso a scrivere componenti di ViewController la cui complessità cresce molto rapidamente perchè gestisce anche la business logic, cioè come gestire i dati del model prima di essere mostrati. Spessi ci si trova a mescolare codice di gestione degli eventi e dell'interfaccia con codice di preparazione dei dati da essere mostrati. In MVVM per ogni evento la View richiama un metodo (spesso uno solo) di ViewModel per prendere i dati già pronti per essere mostrati. Sostanzialmente il ViewModel prepara i dati così da semplificare (molto) il codice della view.



Chapter 7

Sviluppo Cross-Platform

Android ed IOS hanno molti aspetti concettuali in comune. Tuttavia le applicazioni si scrivono in linguaggi nativi differenti:

- Android usa Java o Kotlin
- iOS usa Objective C oppure SWIFT

. Per sviluppare un'applicazione commerciale dobbiamo scriverla due volte? Oppure scriverla solo per un sistema? L'analisi di mercato è molto complicata e non ci interessa nel dettaglio. Possiamo fare delle osservazioni di massima, sappiamo che i dispositivi Android sono più diffusi in termini di numero di dispositivi venduti e numero di dispositivi attivi. Sappiamo però che c'è una percentuale maggiore di dispositivi Android di fascia bassa ed una minore attitudine degli utenti all'acquisto di app. Le revenue derivanti dalla vendita delle app sono più alte su iOS in assoluto ma molto più alte per iOS considerando la media per device. In generale (salvo casi molto particolari) se noi rinunciassimo a sviluppare per una delle due piattaforme come minimo perdiamo un terzo del mercato e tipicamente non è accettabile. Casi particolari potrebbero essere:

- Prototipi
- Situazioni di distribuzione controllata: app richiesta da un cliente che la fa usare ai suoi impiegati che hanno tutti lo stesso tipo di dispositivo

L'alternativa allo sviluppare un'applicazione per una sola piattaforma, che è la soluzione più comune, è sviluppare due volte. Però sviluppare due volte è una grande rognà:

- I linguaggi di programmazione sono diversi
- Le chiamate di SO sono spesso simili concettualmente, ma tecnicamente differenti
- In alcuni casi (background) ci sono chiamate di SO concettualmente differenti

Dunque raddoppiano i costi di:

- Implementazione
- Testing
- Manutenzione

In realtà l'analisi e la progettazione tra i due sistemi sono molto simili, anche se possono esserci problemi di consistenza.

7.1 Sviluppo cross-platform

Noi vorremmo sviluppare l'applicazione, o una parte di essa, una sola volta per entrambe le piattaforme. Questa soluzione prende il nome di sviluppo cross platform. Ci sono diverse soluzioni:

- Codice in comune
- Web app
- Hybrid app
- Conversione del codice
 - Con interfacce native
 - Senza interfacce native

7.1.1 Codice in comune C/C++

Scriviamo codice C/C++ che poi viene eseguito su IOS perchè Objective C estende questi linguaggi e su Android tramite librerie Java (JNI) che permettono di eseguire codice C/C++ all'interno di codice Java.

Vantaggi:

- Codice viene scritto una sola volta
- Le performance sono elevate
- Si possono usare le librerie esistenti di C/C++

Svantaggi:

- Non posso interagire direttamente con il SO: ho bisogno di un bridge in linguaggio nativo. Incluso per le funzioni di interfaccia grafica

Si fa ma per piccole parti di applicazioni, ad esempio se dobbiamo usare librerie particolari che esistono solo in C/C++. Ma non è adatto per creare applicazioni complete.

7.1.2 Sito web / web app

Al posto che creare un'app, creiamo un sito o una web app, non affrontiamo proprio il problema di sviluppare un app, però molte volte risolve il problema. Cosa intendiamo con web app, tecnicamente è un sito web, il quale tipicamente eroga contenuto anche in modalità statica, una web app si focalizza sul comportamento, quindi utilizza massicciamente JavaScript per fare cose e tipicamente è single page, noi chiediamo la pagina al server che ci risponde con tutto l'html delle varie schermate e con JS nascondiamo le varie componenti.



	Focus	Pagine	Script lato client
Web Site	Contenuto	Multiple pages	Possibile
Web app	Interazione	Single Page	Necessario

Table 7.1: Differenze tra sito e web app

Pro:

- La web app può funzionare anche su dispositivi tradizionali
- Le tecnologie sono ampiamente diffuse, stabili, note a molti programmatori e ben documentate

Svantaggi:

- Problemi di user experience
- Accesso al SO (es: periferiche, background, notifiche push, ...)
- Performance minori
- Connessione necessaria per caricare la pagina
- L'app realizzata non è disponibile sullo store: per molte persone la presenza sullo store è una necessità

Cerchiamo di avere un app con questi problemi un po' mitigati.

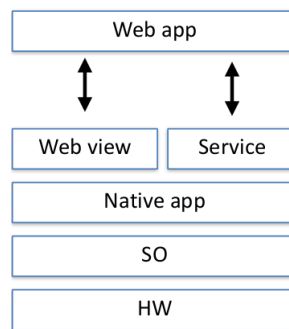
7.1.3 Hybrid App

La web app (scritta in HTML, CSS e JS) viene eseguita all'interno di un'app nativa che contiene una web view. Web view è un componente grafico nativo in grado di mostrare contenuto web. In realtà questo approccio lo si può automatizzare, ci sono delle librerie in cui gli diamo il codice HTML/CSS/JS ed in automatico ci creano progetto android ed IOS con dentro la web view e dentro il codice web.

Apache Cordova

Uno di questi tool/framework per la creazione di app nativa da web app si chiama Apache Cordova. Genera i due progetti che sono delle web app che girano dentro una web view. Le applicazioni sono impacchettate per l'uso come le applicazioni native quindi si possono pubblicare sul market. Esistono tutti dei trucchi che permettono di accedere ad alcune chiamate dell'OS, è una cosa importante perchè come visto prima uno dei grandi limiti di usare una web app è non poter accedere al OS del device.

Qual'è lo schema: abbiamo l'applicazione nativa, sopra la web view ed una serie di servizi che ci permettono di accedere al sistema operativo.



In Apache Cordova esistono dei plugin che vengono distribuiti insieme al sistema, inoltre si possono sviluppare anche plugin aggiuntivi. Per farlo però bisogna scrivere codice nativo Android e iOS ed una parte in JS con cui la web app interagisce per usare il plugin. In teoria permettono di accedere a qualunque funzionalità del OS, ma di fatto aggiungono un livello di complessità, obbligano il programmatore a scrivere codice nativo (che non vorremmo fare dato che stiamo usando app ibride per evitare di sviluppare due volte). Inoltre alcune volte l'interazione è complessa, se non impossibile (realtà aumentata).

Il problema rimane la User Experience. Con le app ibride abbiamo risolto tutti i problemi delle web app, tranne il problema della User Experience e delle



basse performance.



Facebook app, versione ibrida



Facebook app, versione nativa

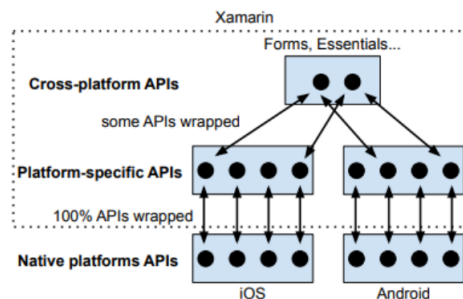
7.1.4 Conversione del codice

Per risolvere il problema delle performance e della user experience, vogliamo fare in modo che il codice esegua come se fosse scritto in nativo, però vogliamo scriverlo una sola volta. L'idea è di scrivere il programma in un linguaggio ed usare uno strumento automatico che converta il codice nelle due piattaforme.

Xamarin

Lo fa Xamarin, fa scrivere l'applicazione in C#. La cosa importante è che scrivendo l'applicazione in questo linguaggio abbiamo la possibilità di usare il 100% delle chiamate API dei vari dispositivi. Xamarin definisce due livelli di astrazione:

- Livello 1: definisce in C# il 100% delle chiamate API, ma di fatto tutte le interazioni con il sistema operativo le devi dividere in due (se sono su android fai questa chiamata se sei in IOS fai questa). Questo livello di astrazione si paga caro, ad esempio gli errori sono più difficili da debuggare.
- Xamarin Forms, è un secondo livello di astrazione, in cui tutte le volte che ho una funzionalità simile per android ed IOS la faccio una volta sola ed un livello sottostante me la traduce, automaticamente, in base al dispositivo in cui sono.



Il vantaggio ovviamente è che una volta che compilo il programma ho un'applicazione nativa. Vantaggi:

- Il programmatore può conoscere un solo linguaggio di programmazione
- Molte parti del codice si scrivono una volta sola
- Alte performance quando eseguito
- Garantisce un User Experience come quella nativa

Svantaggi:

- introducono un livello di astrazione in un ambiente tecnologico in rapida evoluzione e non ancora del tutto consolidato

Unity

E' un ambiente di sviluppo (motore grafico) per lo sviluppo di giochi. Si sviluppa in C# e JS, poi converte il codice in nativo per IOS e Android. In Unity le interfacce non sono native (perchè nei giochi non le usiamo).

Flutter

Si scrive in Dart (di Google), poi il codice viene compilato in Android e iOS.

Flutter come Unity non genera interfacce native, però in flutter sono fatte così bene che anche per un utente esperto è difficile da capire che non lo sono. Il fatto che Unity e Flutter genera interfacce non native porta il fatto che per il OS tutte le interfacce sono delle immagini, non sa cosa c'è dentro, dal punto di vista dell'accessibilità è un casino.

7.1.5 React Native

E' una via di mezzo tra un sistema ibrido ed un sistema a conversione del codice. Il programmatore scrive applicazioni in HTML CSS JS, poi il codice viene convertito solo in parte. Il codice di interfaccia viene convertito in codice nativo, ma se ci pensiamo è facile da convertire questo codice, se inserisco un button prenderò il button nativo. Il resto del codice viene interpretato da JS come per tutte le applicazioni ibride. Si è scoperto che facendo così i problemi di User Experience si risolvono, perchè sono pochissimi i casi in cui il collo di bottiglia è l'algoritmo JS ma è quasi sempre l'interfaccia grafica a rallentare il sistema. Quindi se usiamo l'interfaccia grafica nativa mi risolve il problema di User Experience. Quindi ad esempio il sistema di pulgin visto prima dovremo utilizzarlo anche in react native.

7.1.6 Considerazioni generali sullo sviluppo cross-platform

Le tecnologie stanno diventando mature ma ancora non è che le grosse aziende si fidano troppo, quindi lavorano ancora in nativo. Via via un po' di aziende decidono di rifare l'applicazione con tecnologie cross-platform. Il vantaggio di scrivere in react è che abbiamo già la web app.

7.1.7 Aneddoto

Stava facendo un lavoro e stava facendo uno studio di quanto gli ambienti di sviluppo supportano lo screen reader, se vogliamo scrivere un'applicazione che legga le cose dobbiamo usare delle api native. Ed hanno notato che queste librerie in xamarin non erano disponibili a livello 2 di astrazione, quindi hanno provato a scrivere dei plugin per il livello 1 da richiamare dal livello 2 e non sono riusciti, neanche un programmatore Microsoft (che ci ha messo qualche giorno). Questo per fare capire che non è vero che non funziona tutto benissimo.

7.1.8 Aneddoto 2

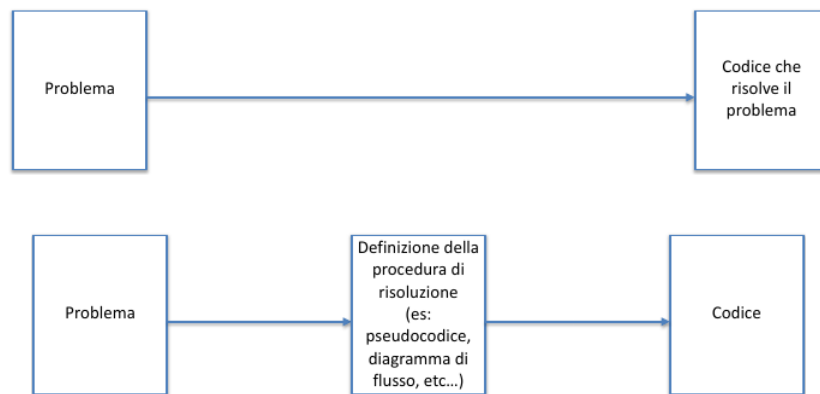
Fino a qualche tempo fa Expo non c'era e dovevamo usare react native senza. Era successa una situazione tipo che il programmatore aveva sviluppato l'applicazione nel suo sistema e poi ci hanno messo 2 giorni per ricompilarla su sistemi diversi. Perché passando da un sistema all'altro era cambiata la libreria. Questo perché React si basa su un sacco di librerie poco documentate e poco stabili.

Chapter 8

Testing e Debugging

Nella prima parte di questo capitolo vedremo alcuni consigli per migliorare nella programmazione.

Molti programmatori inesperti faticano a risolvere problemi di programmazione perchè cercano, dato un problema, di trovare direttamente la soluzione, cioè a scrivere codice. In realtà questo passaggio non è diretto, c'è un passaggio dall'idea alla progettazione della soluzione e poi un passaggio dalla definizione della procedura al codice. Quando un problema è complesso, separare esplicitamente i due passaggi può aiutare molto.



8.1 Dal problema alla procedura

Per alcuni problemi semplici un programmatore si immagina la procedura senza bisogno di formalizzarla. Anche in questo caso dovremmo fermarci, scrivere lo pseudocodice ed andare avanti. E' chiaro che se ci da' un problema difficilissimo lo facciamo, ma dobbiamo farlo anche per problemi medi. Come facciamo se non riusciamo a risolvere il problema? come prima cosa facciamo degli esempi per capire come si risolve. Come si fa se non riusciamo a trovare la procedura? probabilmente non stiamo capendo il problema oppure non capiamo come risolverlo.

Data la formulazione di un problema potrebbe essere difficile capire come si risolve. Facciamo un esempio di quel problema e risolviamolo a mano. Poi domandiamoci quali passaggi abbiamo svolto per risolvere il problema.

Quando abbiamo un'idea di procedura per risolvere il problema, scriviamo lo pseudocodice. Poi eseguiamo a mano la procedura con un insieme di dati di esempio e vediamo se funziona. In questo modo arriviamo ad avere la definizione della procedura.

8.2 Dalla procedura al codice

Spesso può essere complicato passare dalla procedura al codice, soprattutto se abbiamo poca esperienza con la tecnologia. Spesso abbiamo l'idea di scri-

vere grossi pezzi di codice per poi provarlo, sbagliato! Quando scriviamo un grosso blocco di codice non funziona quasi mai e non sappiamo qual'è il problema. L'idea è di non provare neanche a scrivere un grosso blocco di codice, ma una riga per volta lo proviamo, cerchiamo di strutturare la funzionalità da implementare un passettino alla volta per verificare se le cose funzionano. Un programmatore è esperto se riesce a smontare un grosso problema in tanti piccoli passaggi per verificare se funzionano. Il problema è che non tutte le parti del codice producono un effetto visibile (ad esempio il model), in Java però possiamo usare il metodo `toString` su un oggetto per vedere lo stato dell'oggetto. Scrivere delle classi di testing sono fondamentali, soprattutto nel model.

Ricordiamoci che il tempo di sviluppo è in buona parte dedicato a correggere gli errori, ridurre il numero di errori e il tempo necessario per correggerli è l'obiettivo principale se vogliamo programmare bene.

Un altro errore comune è provare il codice senza sapere cosa aspettarsi, altrimenti testare un pezzo alla volta non serve a niente. Questa parte di provare ogni volta sui dispositivi mobili è lento, ci vogliono minuti.

Quando verifichiamo dobbiamo mettere in discussione quello che stiamo facendo, dobbiamo fare dei challenge alla nostra applicazione. Per questo lavorare in 2 al progetto è una buona idea, 1 lo scrive l'altro lo testa, in modo che quello che testa cerca gli errori che ha fatto l'altro.

Questo approccio è ancora più importante nel mobile computing, perchè spesso dobbiamo interagire con componenti di libreria e di OS.

In generale non è errato copiare delle piccole parti di codice da esempi online, ma dobbiamo capire cosa fa il codice, altrimenti facciamo quasi sempre errori. L'approccio giusto è commentare tutto quel pezzo di codice, commentarlo tutto e decommentarlo una riga alla volta, così capiamo cosa fa.

8.3 Esempio

Vediamo un esempio esagerato, studiamo un problema che risulta essere facile, ma supponiamo che questa tecnologia che non conosciamo. Abbiamo un array che contiene un certo numero di parole e vogliamo scrivere un metodo che dice quante volte occorre una parola nell'array

```
//codice
```

```
ArrayList<String> wordsArray = ...
```

```
//altro codice
```

```
int a = wordCount(wordsArray, "ciao");
```

```
//stampa del valore di a
```

Possiamo usare un passaggio intermedio, creiamo una procedura:

```
input: array di parole e una parola
output: il numero di volte che la parola occorre nell'array
Procedura:
-inizializzo un contatore a zero
-Scorro tutte le parole dell'array
  -Per ogni parola che scorro, controllo se uguale a quella in input
    in caso affermativo, incremento il contatore
-Ritorno il contatore
```

Adesso dobbiamo passare dalla procedura al codice. Passo 1: creo un metodo che ritorna sempre lo stesso valore. Verifico di sapere scrivere la definizione del metodo stesso e provo il codice:

```
private int wordsCount(
    ArrayList<String> words, String target){
    return 0;
}
```

Passo 2: inizializzo una variabile e la ritorno, per capire se so inizializzare una variabile:

```
private int wordsCount(
    ArrayList<String> words, String target){
    int counter = 0;
    return counter;
}
```

Passo 3: verifico di sapere scrivere un codice che scorre tutti gli elementi. Contandoli. Modifico l'array di input per fare poche con zero, uno o tanti elementi.

```
private int wordsCount(
    ArrayList<String> words, String target){
    int counter = 0;
    for(String word : words) {
        counter++;
    }
    return counter;
}
```

Passo 4: verifico di saper controllare se la prima parola dell'array è quella cercata. Provo solo l'if per verificare se l'uguaglianza funziona.

```
private int wordsCount(
    ArrayList<String> words, String target){
    int counter = 0;
    String word = words.get(0);
    if(word.equalsIgnoreCase(target))
```

```
        counter++;  
    return counter;  
}
```

Passo 5: possono mettere assieme i passi precedenti. Prima di metterlo in produzione dobbiamo provarlo con vari elementi dell'array

```
private int wordsCount(  
    ArrayList<String> words, String target){  
    int counter = 0;  
    for (String word : words) {  
        if (word.equalsIgnoreCase(target))  
            counter++;  
    }  
    return counter;  
}
```

8.4 Gestire progettazione e implementazione

Specifichiamo ora quale rapporto c'è tra progettazione ed implementazione, sappiamo teoricamente che prima si progetta la struttura del codice e poi si implementa. Nella pratica facciamo fatica, perchè per progettare la struttura delle classi dobbiamo avere una certa esperienza, che non abbiamo. Quindi quello che facciamo è non progettare ed implementare direttamente. Questo approccio è sbagliato, prima di tutto sforziamoci di dire grossolanamente come sarà strutturato il codice, questo ci aiuta a scrivere codice più ordinato. Quando implementiamo ci rendiamo conto che è diverso dalla progettazione, non c'è niente di male ma modifichiamo la progettazione (diagramma delle classi). Alla fine abbiamo una progettazione che descrive il codice e evitiamo di dimenticarci quello che abbiamo fatto.

A volte quando scriviamo il codice implementiamo una funzionalità, e così via mentre scriviamo impariamo delle cose e ci accorgiamo che una cosa che abbiamo fatto prima si poteva fare meglio, in questo caso rifacciamole meglio, perchè la struttura deve essere solida.

8.5 Testing

Esistono vari tipi di testing:

- Funzionalità: functional testing, l'applicazione fa quello che dovrebbe?
- Usabilità: usability testing, l'utente riesce ad usare l'app come previsto?
- Performance: performance testing, l'applicazione ha le performance desiderate?

- Sicurezza: security/privacy testing, l'app è soggetta ad attacchi di sicurezza o privacy?

Noi ci focalizzeremo su test di funzionalità.

Lo scopo del testing è trovare i problemi, quando facciamo un testing e non troviamo i problemi ci sono due possibilità:

- Il codice funziona correttamente
- Non abbiamo fatto bene il testing

8.5.1 Testing su dispositivi mobili

Perchè è particolarmente difficile fare testing su dispositivi mobili? abbiamo detto che i dispositivi sono altamente frammentati (sia hardware che software) quindi in teoria se volessimo testare in maniera esaustiva la nostra applicazione dovremmo testarla su ogni dispositivo e su ogni sistema operativo supportato. Già qua capiamo che non sarebbe possibile. In realtà il problema è ancora più complicato perchè abbiamo vari contesti di utilizzo (connessione a internet non stabile, i server sono lenti, l'utente si sposta, l'orientamento dello schermo). Inoltre il codice su dispositivi mobili è basato su eventi, può esserci un caso in cui l'ordine degli eventi può non essere deterministico (mi arriva prima il dato dall'accelerometro o il dato di posizione?) di fatto si pongono dei problemi di concorrenza. Inoltre abbiamo dei problemi di concorrenza perchè la nostra applicazione è multi-thread e l'ordine in cui i thread vengono schedulati non è deterministico, quindi può essere che una parte di codice viene eseguita prima ed una dopo. In realtà per come organizzeremo il codice noi non avremo veramente problemi di concorrenza perchè non eseguiamo veramente le cose in parallelo. Questa concorrenza introduce un non determinismo, perchè il fatto che l'applicazione funzioni una volta non significa che sia corretta, cioè potrebbe presentare degli errori in futuro, anche se eseguita nello stesso modo.

Di fatto è impossibile provare tutte le combinazioni per i dispositivi mobili. Quello che facciamo è scegliere un insieme di casi rappresentativi in cui testare l'applicazione. Dobbiamo trovare una quantità piccola di queste combinazioni per cui è possibile fare i test, non sto avendo la garanzia formale che l'app funzioni sempre, ma ci serve per trovare degli errori.

8.5.2 Testi in casi rappresentativi

Come facciamo a capire quali sono i test rappresentativi? un po' dipende dall'esperienza, se stiamo facendo un app per tablet ed una per smartphone faremo test per tablet e smartphone. Inoltre dobbiamo coprire tutte le parti di codice che abbiamo scritto (tutti i rami degli if). Cerchiamo di avere un paio di dispositivi fisici su cui testare l'applicazione. Come minimo dobbiamo provarla sulla versione dell'OS più vecchia supportata e sulla più recente. Inoltre dobbiamo svolgere i test in assenza di problemi e in presenza dei problemi più comuni:

- mancanza/rallentamento della connessione
- non funzionamento/rallentamento dei servizi esterni
- posizione utente non disponibile
- login e logout e login con password errata, l'ha guardata come cosa durante il progetto

Questi eventi vanno gestiti sempre, perchè è la norma che succedano nei dispositivi mobili.

Bisogna verificare il funzionamento di tutto il codice, se ho dei costrutti di selezione devo provare tutti i possibili percorsi, se ho dei costrutti iterativi devo provare con zero iterazioni con 1 o più iterazioni.

Almeno periodicamente dobbiamo testare l'applicazione in tutti una serie di casi anomali:

- Assenza o rallentamento della connessione Internet
- Mancato funzionamento o rallentamento dei servizi web utilizzati
- Chiusura e riapertura dell'app, sia nel caso in cui l'app venga terminata, sia nel caso in cui non lo sia
- Chiamata in arrivo (o altri interrupt)
- Cambio di posizione (se pertinente all'applicazione)
- I casi in cui l'utente non concede dei permessi, come ad esempio non concede il permesso all'uso della posizione o uso con diverse lingue di sistema
- Primo avvio e avvii successivi
- Utente commette errori, ad esempio nome utente e password errati

8.5.3 Automatizzazione del testing

Svolgere a mano il test dell'applicazione è lungo e noioso. E' possibile automatizzare i test. Possiamo scrivere del codice che verifica la correttezza di altre parti del codice, lo unit testing, l'idea è scrivere una parte di codice il cui scopo è verificare se del codice funziona. Per esempio possiamo creare un codice di test che crei un'istanza di una classe persona, chiami il metodo setName e chiamando getName il nome ritornato sia quello del set.

I test automatizzati hanno una chiamata assert, se l'assert è corretto allora il test ha successo, altrimenti fallisce. Quindi possiamo avere una serie di test, se tutti hanno successo ok, se uno fallisce dobbiamo correggere un errore. Noi abbiamo un modo integrato negli ambienti di sviluppo che ci permette di verificare tutti i test.

```
public class UnitTestSampleJava {
    private static final String FAKE_STRING = "HELLO_WORLD";
    private Context context = ApplicationProvider.getApplicationContext();

    @Test
    public void readStringFromContext_LocalizedString() {
        // Given a Context object retrieved from Robolectric...
        ClassUnderTest myObjectUnderTest = new ClassUnderTest(context);

        // ...when the string is returned from the object under test...
        String result = myObjectUnderTest.getHelloWorldString();

        // ...then the result should be the expected one.
        assertEquals(result, FAKE_STRING);
    }
}
```

E' anche possibile verificare il funzionamento delle interfacce, devo in qualche modo simulare il comportamento dell'utente.

```
@RunWith(RobolectricTestRunner.class)
public class MyActivityTest {

    @Test
    public void clickingButton_shouldChangeResultsViewText() throws Exception {
        MyActivity activity =
            Robolectric.setupActivity(MyActivity.class);

        Button button = (Button) activity.findViewById(R.id.button);
        TextView results = (TextView)
            activity.findViewById(R.id.results);

        button.performClick();
        assertEquals(results.getText().toString(),
            "Robolectric Rocks!");
    }
}
```

Esempio (tratto da [robolectric](#))

Simulo un'azione dell'utente

Verifico che il risultato
sia quello atteso

In università non scriviamo codice di testing, perchè facciamo progetti che

non devono essere mantenuti, scrivere il testing potrebbe non valere la pena in questi casi. Quindi il codice di testing è comodo se dobbiamo rieseguire quel codice tante volte. E' molto vantaggioso il testing automatizzato quando ho progetti che devono essere mantenuti nel tempo. Molti team di sviluppo prima di fare il commit su git devi passare il test di unità, altrimenti non mandare neanche il codice. La controparte è che in altri contesti potrebbe non avere senso, ad esempio se abbiamo 1 mese per fare l'applicazione per una startup da presentare a degli investitori non perderemo tempo a scrivere unit test, nel caso li faremo dopo. Sintetizzando nei progetti commerciali ha senso, nei prototipi no.

Spesso si fanno tanti test per provare singole parti, meno test per verificare la loro integrazione e pochi test per valutare l'applicazione nella sua globalità. Questa cosa si chiama approccio a piramide. (noi vedremo i test di unità in Android, usando la libreria Espresso).

8.6 Debugging

Il debugging è la procedura attraverso la quale si identifica e risolve un bug. Ci sono tre passaggi:

- Riprodurre il malfunzionamento
- Trovare il bug
- Risolvere il bug

8.6.1 Riprodurre il malfunzionamento

Il primo passaggio per fare debugging è riuscire a riprodurre l'errore, sembra ovvio ma non lo è. E' molto meglio debuggare un problema che si verifica sempre rispetto ad uno che si verifica ogni tanto. Per riprodurre il malfunzionamento a volte è necessario modificare il codice in modo tale che quel bug sia sempre verificato. Spesso debuggando inseriamo e modifichiamo il codice introducendo delle cose che non vogliamo, facciamoci un commento TODO per ricordarci che va tolta.

Quando facciamo il test manuale succede spesso che per riprodurre una certa funzionalità devo fare una serie di passaggi, ad esempio l'app va in crash quando il nome è più lungo di 100 caratteri, ci creeremo una variabile settando lo username, ma facciamoci un commento per ricordarci di toglierlo.

8.6.2 Logging

Il logging è una parte del codice dell'applicazione finalizzata a stampare messaggi per il programmatore. Molto utile in applicazioni con interfaccia, per capire come l'applicazione gestisce gli eventi. Non aspettiamo di avere un errore per avere un messaggio di logging, ci sono dei casi in cui di base scriviamo un

log associato ad un evento. Incidentalmente questi messaggi servono anche da documentazione del codice, lui sostiene che in codice semplice (tipo applicazioni mobili) dobbiamo scrivere commenti solo se stiamo facendo casino. Se usiamo nomi di variabili e metodi chiari il nostro codice è autoesplicativo.

In Android ci sono dei modi per filtrare i messaggi di log, perchè potremmo averne tanti:

- Livelli di importanza
- Processo che li ha generati
- Tag: nei messaggi di log possiamo aggiungere un etichetta che può essere la funzionalità che stiamo debuggando o il nome della classe, così possiamo filtrare per classe o funzionalità

8.6.3 Debugger

E' uno strumento disponibile negli ide che permette di eseguire l'app in modalità debugging, vuol dire che posso fermare l'app in corrispondenza di alcuni break-point che sceglie il programmatore. Quando ho interrotto l'esecuzione posso:

- Vedere i valori delle variabili, spesso il codice non funziona perchè una variabile non ha il valore che ci aspettiamo
- Eseguire il codice riga dopo riga, per tenere traccia di come cambiano le variabili
- Cambiare il valore delle variabili o chiamare dei metodi

Attenzione che quando siamo in modalità debugging e l'app è ferma abbiamo vari modi per proseguire l'esecuzione, guardiamoli.

8.6.4 Strategie di debugging

- Piccoli passi: se scriviamo il codice riga per riga e lo proviamo, solitamente se si verifica un errore è nell'ultima riga che abbiamo fatto. Non è sempre detto.
- Tecnica "wolf fence": la similitudine che viene fatta è: dobbiamo trovare un lupo in Canada, mettiamo una staccionata (log) che divide il Canada in due, vediamo dove sta il lupo(bug), se sta a destra mettiamo un'altra staccionata che divide la metà del Canada ancora in due. Quindi mettiamo una stampa (log) e vediamo se l'errore è prima o dopo, iteriamo fino a quando non abbiamo trovato uno spazio sufficientemente piccolo dove sta l'errore. Immaginiamo di avere un crash, se mettiamo un log e viene stampato prima del crash significa che l'errore è dopo, quindi mettiamo il messaggio dopo.

- "Torna indietro e prova": se ho un bug e non so dove avviene posso rimuovere del codice (commentandolo) fino a quando non trovo una versione funzionante. A questo punto scommento riga per riga in modo da capire dove sta l'errore. Questo è particolarmente comodo quando copiamo del codice.
- Semplifica il codice: spesso copiamo del codice scritto da altri, solitamente noi non scriviamo metodi con metodi innestati. La soluzione se troviamo codice innestato è scomporlo partendo dai metodi più interni

Esempio semplificazione codice:

```
nameTextView.setText(db.getUserByID(idTextView.getText()).etName());
```

```
String userID = idTextView.getText();
User user = db.getUserByID(userID);
String name = user.getName();
nameTextView.setText(name);
```

8.6.5 Errori comuni

Un errore comune è vedere che c'è un errore e non leggere il messaggio di errore, ci sono dei trucchi per leggere i messaggi di errore, non dobbiamo leggerlo tutto, ma basta leggere:

- La causa di errore (null pointer exception)
- La riga del codice che ha generato l'errore. Potrebbe essere che un errore nel codice ha generato un errore in un codice di sistema o libreria, dunque lo stacktrace è molto lungo.

Un altro errore comune è testare l'app senza sapere cosa aspettarsi, dobbiamo avere bene in chiaro in testa cosa ci aspettiamo che l'app faccia.

8.7 Conclusioni

Questo è l'atteggiamento più sbagliato che possiamo avere in fase di debugging. Quando abbiamo un app che non fa quello che dovrebbe (o crasha) non dobbiamo guardare il monitor senza sapere cosa fare. E' del tutto normale avere dei bug. Bisogna iniziare ad agire per risolvere il bug, usando le tecniche suggerite. E' difficile che rileggendo il codice capiamo che errore abbiamo, dobbiamo fare delle cose.

8.7.1 Attenzione a come fare il test

Fare sempre le stesse operazioni durante le prove delle applicazioni non è una buona strategia. Così facendo si testano sempre le stesse parti del codice, non altre.

8.7.2 Esperienza

Quando troviamo il motivo del bug, fermiamoci a pensare il processo metodologico che mi ha portato a trovarlo. E' importante perchè possiamo riutilizzare quel metodo in altri contesti.

Chapter 9

React Native

9.1 JavaScript

Cominciamo con una lezione di introduzione a JavaScript, vedremo solo alcuni concetti che useremo in maniera massiva durante la parte di sviluppo in React. Usiamo JS perchè ReactNative è basato su React che è basato su una serie di librerie proprie e HTML/CSS/JS.

Javascript è un linguaggio di programmazione che nasce nel 1995, è nato perchè si voleva dare un po' di comportamento alle applicazioni web lato server. Man mano JS si è evoluto implementando funzionalità aggiunte, da JS possiamo modificare il DOM, ad esempio schiacciando su un pulsante modifichiamo un elemento del DOM e l'utente vede una cosa differente. Vedremo usando React che ci permette di fare applicazioni sofisticate scrivendo molto poco codice. In realtà negli ultimi anni JS si usa anche lato server, il motivo è sociale, è un linguaggio comune e molto utilizzato inoltre JS permette di scrivere molto velocemente, ha tante librerie e poi essendo basato su eventi risulta molto comodo anche lato client, dove l'evento è la richiesta di un client. Questo è per motivare che stiamo usando una tecnologia molto utile nel mondo del lavoro.

Ricordiamo che JavaScript non centra niente con Java. ECMAScript è lo standard che definisce delle varie versioni di JavaScript, la sintassi è definita da ECMAScript mentre l'implementazione del linguaggio è JS.

9.1.1 Caratteristiche di JS

La caratteristica fondamentale è che è un linguaggio debolmente tipato, significa che il tipo di una variabile è definito a runtime. In realtà i casi in cui è utile cambiare il tipo di una variabile sono molto rari, è molto più comune farlo per sbaglio, l'errore lo avremo a tempo di esecuzione, è scomodo. C'è un compromesso, più il linguaggio di programmazione è tollerante però tutela meno il programmatore rispetto agli errori.

Un'altra caratteristica di JS è che è basato sugli oggetti, occhio che Java è Object Oriented (orientato sugli oggetti), un linguaggio object oriented ha tutte le funzionalità, un linguaggio basato sugli oggetti ha alcune funzionalità.

Originariamente JS era un linguaggio interpretato (eseguito riga per riga) nelle ultime versioni usa un approccio just in time, significa che se alcuni parti di codice vengono usate spesso quelle parti vengono compilate.

9.1.2 Definizione di una funzione

Ci sono 3 modi per definire una funzione:

- Function declaration: ho una parola riservata function, definisco gli attributi e non specifico il valore di ritorno
- Function expression: posso avere una variabile associata ad una funzione
- Arrow notation: è molto simile alla function expression, posso associare ad una variabile una funzione definendola in questa maniera. C'è una differenza sintattica importante che vedremo più avanti.

Anche il valore di ritorno non è definito.

9.1.3 Metodo map

E' un metodo utilizzato molto nella programmazione funzionale. Se ho un array e chiamo il metodo mi restituisce un'altro array in cui elemento per elemento è applicata la funzione map. Guardiamo anche il metodo reduce. La differenza con il forEach è che il primo non ritorna una nuova variabile, mentre il map sì.

9.2 Oggetti

In JS la gestione degli oggetti è sintatticamente e concettualmente diversa rispetto a Java. A partire dalla versione 6 di ECMAScript sono stati introdotti nuovi metodi per gestire gli oggetti simile a Java, ma è solo zucchero sintattico, il modo in cui sotto gli oggetti sono implementati è completamente diverso rispetto a Java. In JS gli oggetti sono array associativi, in Java l'oggetto è l'istanza di una classe. In JS un oggetto è un insieme di coppie chiave-valore.

9.2.1 Literal Notation

Sto dicendo che una variabile è associata ad un oggetto, quello tra graffe.

Siccome è un array posso accedere agli elementi sia con la dot notation, è valida anche la sintassi tra quadre, che ci specifica che è un array. Ci sono alcuni casi in cui abbiamo il valore di una chiave che è una stringa e vogliamo farci qualcosa, quindi siamo obbligato ad usare la bracket notation.

9.2.2 Metodi

I valori dell'array associativo possono essere funzioni, questo equivale concettualmente a creare metodi in Java associati all'oggetto.

9.2.3 Il concetto di this

In JS this è un casino, proprio perchè è più tollerante. This è una keyword che fa riferimento allo scope in cui il codice è eseguito.

Possiamo definire una funzione che costruisce un oggetto

9.2.4 Costrutto sintattico new

Possiamo definire alcune funzioni, dette costruttori, che vengono richiamate con la parola riservata new che implicitamente costruiscono un oggetto e ci accedono con il this. Per convenzione la funzione la scrivo con la prima lettera maiuscola.

9.2.5 Prototipi

Per capire come funziona veramente JS ad oggetti dobbiamo studiare i prototipi.

9.2.6 Definizione e istanza di una classe

Possiamo definire una classi in modo molto simile a quanto avviene in Java. Si usa la parola riservata `constructor` per definire il costruttore.

Quando voglio andare a creare un istanza di quella classe uso la parola riservata `new`, il nome della classe ed i parametri del costruttore.

9.2.7 Ereditarietà

La parola chiave `extends` mi permette di estendere una classe. Nell'esempio usiamo il vecchio costruttore di `Persone` con `super` nel costruttore. Inoltre facciamo un `override` di `print`, che era un metodo di `Person`.

9.2.8 Uso di `this` nelle classi

In JavaScript è possibile fare in modo che un metodo di una classe punti al metodo in un'altra classe.

la funzione `bFunction` della classe `B` è un riferimento alla funzione `aFunction` della classe `A`. E' come se avessimo dato ad un attributo di `B` la funzione di `A`. A questo punto quel `this` non è chiaro a cosa fa riferimento. La soluzione sintattica di JavaScript è che il `this` fa riferimento allo scope, ovvero all'oggetto su cui è chiamato il metodo. Non è neanche detto che l'oggetto `B` abbia un valore string, quindi mi darebbe `undefined` quando richiamo il metodo.

9.2.9 Wrapper

Definisco `a` come una funzione che, quando richiamata, invoca la funzione `p.sayHi()`.

Posso usare una versione più compatta con la arrow notation.

9.2.10 Binding

Le funzioni hanno un metodo `bind` che permette di specificare quale sia il contesto nel quale devono essere richiamate. Il metodo `bind` ritorna una funzione. Diciamo che `sayHi` è una funzione, quando qualcuno me la copia chiamamela sull'istanza dell'oggetto, va messo nella definizione della classe.

9.2.11 Wrapping vs Binding

La cosa si complica ancora se cambio l'oggetto sul quale è definita la funzione. Con il wrapping cambia il risultato: in questo caso cambiando `p` cambia anche `a()`.

Usando il binding abbiamo il comportamento opposto, la funzione rimane sempre collegata all'oggetto originario:

9.2.12 Arrow function e binding

Se si definisce un metodo usando l'arrow notation, si ottiene il comportamento desiderato: `this` si riferisce sempre all'oggetto nel quale definiamo il metodo. Come per il `bind` quando l'oggetto cambia, il comportamento rimane lo stesso.

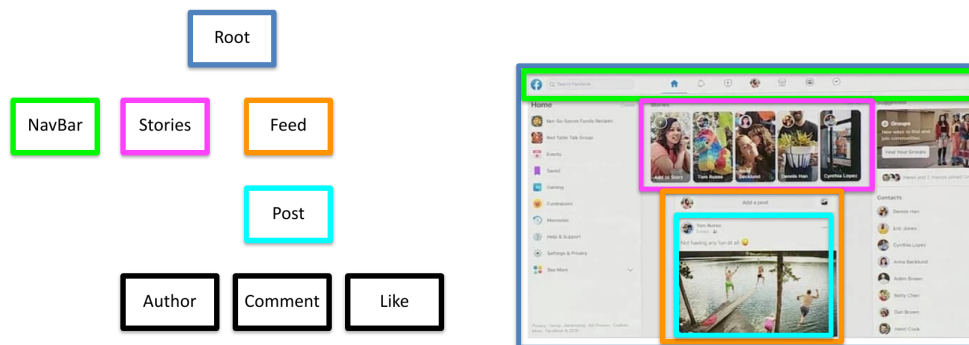
9.3 React

E' una libreria JS per realizzare interfacce grafiche sviluppata da Facebook a partire dal 2011. Uno dei tanti limiti di fare applicazioni web con HTML + JS è che JS produce codice HTML, un casino per debuggare. Per questo è stato introdotto react. React ha molto influenzato lo sviluppo di applicazioni web, perchè ti porta a pensare le interfacce grafiche in modo diverso.

Partiamo studiando react perchè poi useremo react native per fare le applicazioni.

9.3.1 Component

Component è un concetto fondamentale di React. E' una parte dell'interfaccia grafica indipendente, isolata e riutilizzabile. Ci sono vari componenti che possiamo mettere insieme. I componenti sono organizzati secondo una gerarchia ad albero. La radice è chiamata "root component" che rappresenta l'intera applicazione e contiene altre componenti.



E' un vantaggio perchè possiamo dare a team diversi lo sviluppo di componenti diversi, questo isolamento porta dei vantaggi in termini di sviluppo, test e manutenzione.

Un componente è una classe JavaScript con:

- Uno stato (state): è una variabile
- Un metodo render: mostra il contenuto della componente sulla base dello stato della componente

L'approccio in react è: immaginiamo un metodo che disegna l'interfaccia in base allo state. Come viene cambiato state quando disegniamo non interessa. Tecnicamente render ritorna un `reactElement`, che è un oggetto JS che rappresenta un elemento del DOM, non è un vero elemento del DOM ma lo rappresenta. Quindi di fatto abbiamo un albero di elementi che rappresentano elementi del DOM, si chiama virtual DOM, è un DOM rappresentato da oggetti JS. React tiene in memoria questo virtual DOM, quando lo stato cambia React richiama il metodo render, che aggiorna il virtual DOM poi confronta questo con il DOM reale e cambia quello che c'è da cambiare. Questo è fatto per questioni di performance.

9.3.2 JSX

JavaScript Syntax Extension, è una tecnologia tanto usata in React. E' un'estensione della sintassi JS. Permette di definire oggetti JavaScript che rappresentano tag HTML e componenti in modo sintatticamente simile a quello che avviene per HTML. Estendiamo la sintassi di JS dato che cambiamo il linguaggio abbiamo bisogno di un compilatore che compili la sintassi estesa. Noi vogliamo definire come parte del linguaggio dei costrutti che assomiglino molto ad HTML, che ci permetteranno di scrivere del codice.

Supponiamo di voler definire una serie di oggetti della libreria React che rappresentano un DIV con dentro un h1 e dentro un p. In JSX è molto più comodo farlo, il compilatore Babel converte la sintassi JSX in JS.

<pre>const a = <div> <h1>ciao</h1> <p>questo è un esempio di link </p> </div></pre> Nota: questa non è una stringa	<pre>1 "use strict"; 2 3 const a = /*#__PURE__*/ React.createElement(4 "div", 5 null, 6 /*#__PURE__*/ React.createElement("h1", null, "ciao"), 7 /*#__PURE__*/ React.createElement(8 "p", 9 null, 10 "questo \xE8 un esempio di", 11 /*#__PURE__*/ React.createElement(12 "a", 13 { 14 href: "www.google.com" 15 }, 16 "link" 17) 18) 19);</pre>
--	---

Attenzione che JSX appare molto simile ad HTML, ma NON lo è. Ci sono delle differenze sintattiche, ad esempio l'attributo CSS Class non lo possiamo utilizzare in quanto in JSX è una parola riservata, dovremo utilizzare Class-Name.

JSX è anche molto comodo anche per mostrare il valore delle variabili, perché in JS dobbiamo alternare stringhe a codice. Noi mettiamo tra parentesi graffe un'espressione, che può essere una variabile oppure un'espressione o una singola funzione. Non posso metterci codice arbitrario, solo una funzione.

```
//Mostro il valore della variabile
<div>
  <span>{this.state.textToShow}</span>
</div>
```

```
//Mostro il risultato dell'espressione
<div>
  <span>{2+4}</span>
</div>
```

```
//Mostro il risultato della funzione
<div>
  <span>{sum(5,6)}</span>
</div>
```

JSX può contenere tag annidati, ma al primo livello deve contenere un solo tag. Questo esempio non andrebbe bene:

```
<p>ciao </p><p>mondo</p>
```

Dobbiamo mettere tutto tra div.

Tecnicamente sarebbe possibile usare React anche senza JSX. In react potremmo indicare in JSX come comporre i componenti nella vista. Ma JSX è ampiamente utilizzato in React perchè rende la struttura molto più compatta e leggibile.

9.3.3 Hello Word in react

Ci sono due comandi per creare un progetto react e farlo partire:

- npm init react-app my-app
- npm start

Una volta partito fa partire un server, questo perchè il nostro browser non è in grado di interpretare i file jsx.

Capiamo come funziona un progetto react (cosa che non serve praticamente, ma teoricamente è interessante). Nel progetto lui crea una serie di cartelle, i sorgenti della nostra applicazione sono nella cartella src, public contiene le risorse (es. icone), un file index.html.

Il file index.html contiene nel body un div vuoto:

```
<body>
  <div id="root"></div>
</body>
```

Viene creato questo div vuoto ha un id, poi il file index.js fa questo:

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
```

```

    <App />
  </React.StrictMode>
);

```

Prende l'elemento con l'id e ci mette dentro un tag App, che non è html ma JSX. Quindi noi dobbiamo modificare App.

Nel momento in cui salviamo esiste una cosa che si chiama live reload in modo che senza dover fare nulla vediamo le modifiche apportate.

9.3.4 Struttura di un componente

In generale un component è una classe che estende React.Component e che ha un metodo render(), che definisce cosa deve mostrare il component Il codice di esempio di un component:

```

import React, {Component} from 'react';

class App extends React.Component {
  render() {
    return <div>Ciao Mondo</div>;
  }
}
export default App;

```

Abbiamo un import e ci serve esportare la classe App. Poi abbiamo una classe che estende react component, deve avere uno stato ed un component. L'ultima riga serve per esportare il modulo. Non è concettualmente interessante. Per convenzione i component si mettono nella cartella "components" dentro "src". Come suggerimento, quando creiamo il file di un component, usiamo l'estensione jsx.

9.3.5 Lo stato dei componenti

Come già visto ogni componente ha una funzione render() per mostrare il contenuto del componente stesso. Può avere una variabile state per salvare dei dati associati al componente. Nell'esempio vediamo come mostrare, in JSX, i valori dello stato.

```

import React, {Component} from 'react';

class App extends React.Component {
  state = {
    textToShow : "ciao"
  }
  render() {
    return <div>
      <span>{this.state.textToShow}</span>
    </div>;
  }
}

```

```

    }
  }
  export default App;

```

9.3.6 Visualizzazione condizionale JSX

Consideriamo questo problema: se una certa variabile dello stato è vuota, mostriamo un'interfaccia grafica, altrimenti ne mostriamo un'altra. All'interno di JSX non possiamo inserire costrutti di selezione. Però possiamo creare metodi che effettuino il controllo (fuori dal `render()`) e ritornino quanto serve.

```

import React, {Component} from 'react';

class App extends React.Component {
  state = {
    textToShow : "ciao"
  }
  renderText() {
    if (this.state.textToShow === "") {
      return <h3>{this.state.textToShow}</p>
    }
    return <p>{this.state.textToShow}</p>
  }
  render() {
    return <div>
      <h1>Ti dico una cosa importante</h1>
      {this.renderText()}
    </div>;
  }
}
export default App;

```

Quando abbiamo una piccola selezione possiamo usare un operatore ternario in JSX:

```
(condizione1)?(faiQuesto):(faiQuest'altro);
```

Che tradotto sarebbe, se(?) `condizione1` è vera allora `faiQuesto` altrimenti(:) `faiQuest'altro`.

9.3.7 Visualizzazione iterativa JSX

Potremmo voler mostrare un elemento grafico per ogni elemento di un array. Anche in questo caso, non possiamo inserire un costrutto di iterazione nel JSX. Però JSX può contenere un array di oggetti.

Prima soluzione: Come nel caso dei costruttori di selezione, possiamo creare una funzione esterna che genwri gli elementi del virtual DOM che ci servono:

```
import React, {Component} from 'react';

class App extends React.Component {
  state = {
    textToShow : ["Andrea", "Bruno", "Carlo"]
  }
  createList() {
    let contactsGUI = []
    for (let contact of this.sate.contacts) {
      contactsGUI.push(<li>{contacts}</li>)
    }
    return contactsGUI
  }
  render() {
    return <div>
      <ul>{this.createList()}</ul>
    </div>;
  }
}
export default App;
```

Questa soluzione funziona, però genera un warning, ogni elemento del virtual DOM deve essere identificato univocamente, dobbiamo quindi aggiungere una key al li.

Seconda soluzione: creiamo l'array di elementi di virtual DOM in una sola espressione usando il metodo map:

```
import React, {Component} from 'react';

class App extends React.Component {
  state = {
    textToShow : ["Andrea", "Bruno", "Carlo"]
  }

  render() {
    return <div>
      <ul>{this.state.contacts.map(contact =>
        <li key={contact}{contact}</li>)}</ul>
    </div>;
  }
}
export default App;
```

Terza soluzione: spesso le righe di una lista sono elementi complessi, potrebbe essere complicato e soggetto ad errori scrivere nella funzione di ma come mostrare

il dato. Una soluzione è delegare ad una funzione la creazione del layout della riga (ancora meglio sarebbe creare un component che rappresenti un elemento della lista, lo vedremo più avanti):

```
render() {
  return <div>
    <ul>{this.state.contacts.map(contact =>
      this.contactRow(contact))}</ul>
  </div>;
}

contactRow(contact) {
  return <li key={contact}>
    <h1>{contact}</h1>
    <p>E' il mio amico con il nome di {contact.length} caratteri</p>
  }
}
```

9.3.8 Gestione degli eventi

Possiamo definire, per gli oggetti del virtual DOM, gli stessi eventi che si possono definire sugli elementi del DOM. (onClick). Per convenzione, i metodi che vengono usati per rispondere ad eventi iniziando con handle (handleDelete se c'è il pulsante delete). Attenzione, come normale in javascript, quando specifichiamo quale metodo gestisce l'evento, indichiamo la funzione, non la invochiamo (cioè la specifichiamo senza parentesi).

Esempio:

```
handleDelete() {
  console.log("Delete Button pressed")
}

render() {
  return <div>
    <button on Click={this.handleDelete}>delete</button>
    <ul>{this.state.contacts.map(contact =>
      this.contactRow(contact))}</ul>
  </div>
}
```

C'è un problema che emerge nella gestione degli eventi. Guardiamo questo codice:

```
handleDelete() {
  console.log("Delete Button pressed " + this.state.contacts.lenght)
}
```

Sembrerebbe facile capire il comportamento, invece non funziona! genera un'eccezione al click sul pulsante: this non è definito. E' un problema di binding: quando viene chiamata la funzione handleDelete lo scope non è quello atteso (cioè l'oggetto). Soluzione 1: facciamo un binding del metodo nel costruttore:

```

constructor() {
  super()
  this.handleDelete = this.handleDelete.bind(this)
}

handleDelete() {
  console.log("Delete Button pressed " + this.state.contacts.length)
}

```

Soluzione 2, wrapping: usando la arrow notation (nel render) passiamo una funzione che richiama la funzione di callback su this. In pratica creiamo una nuova funzione che quando viene richiamata usa lo scope corretto (quello dell'oggetto).

```

handleDelete() {
  console.log("Delete Button pressed " + this.state.contacts.length)
}

render() {
  return <div>
    <button onClick={() => this.handleDelete()}>delete</button>
    <ul>{this.state.contacts.map(contact =>
      this.contactRow(contact))}</ul>
  </div>
}

```

Soluzione 3: definiamo la funzione di callback tramite arrow function. Questo garantisce il binding all'oggetto. E' una sintassi chiamata sperimentale che al momento funziona ed è compatta:

```

handleDelete = () => {
  console.log("Delete Button pressed " + this.state.contacts.length)
  this.setState({contact : this.state.contacts.pop()})
}

render() {
  return <div>
    <button onClick={this.handleDelete}>delete</button>
    <ul>{this.state.contacts.map(contact =>
      this.contactRow(contact))}</ul>
  </div>
}

```

9.3.9 Aggiornamento dello stato

La classe Component (da cui ereditano tutti i component) definisce il metodo setState che prende in input un oggetto e fa due cose:

- un merge tra lo stato corrente (attributo state) e l'oggetto passato. Merge: unisce l'oggetto passato all'attributo state, aggiungendo attributi o sovrascrivendone il valore se ci sono già.
- Ridisegna l'interfaccia grafica

Esempio:

```
state = {
  contacts : ["Andrea", "Bruno", "Carlo"]
}

handleDelete = () =>{
  console.log("Delete Button pressed " + this.state.contacts.lenght)
  this.state.contacts.pop()
  this.setState(this.state)
}
```

Quando lo stato viene aggiornato, React richiama la funzione render.

9.3.10 Gestione degli eventi con parametri

Succede spesso di voler gestire eventi in cui abbiamo bisogno di gestire un parametro (ad esempio un tocco su un elemento in una lista). Usando la soluzione del wrapping possiamo aggiungere un parametro alla funzione in JSX.

```
handleDelete(contact) {
  console.log("Deleting " + contact)
  let newContacts = this.state.contacts.filter(c => c !== contact)
  this.setState({ contacts : newContacts })
}

render() {
  return <div>
    <ul>{this.state.contacts.map(contact => this.contactRow(contact))}</ul>
  </div>
}

contactRow(contact) {
  return <li key={contact}>
    <h1> {contact} </h1>
    <button onClick={() => this.handleDelete(contact)}>delete</button>
    <p>E' il mio amico con il nome di {contact.length} caratteri</p>
  </li>
}
```

• Andrea

delete

E' il mio amico con il nome di 6 caratteri

• Bruno

delete

E' il mio amico con il nome di 5 caratteri

• Carlo

delete

E' il mio amico con il nome di 5 caratteri

9.4 Composizione di componenti

Il codice scritto fino ad adesso è brutto, perchè non sfrutta la caratteristica di react di modularità del codice. Noi fino ad adesso abbiamo un solo componente che fa tante cose.

Creiamo un componente il cui scopo è mostrare un singolo contatto. Di che dati avrà bisogno? usiamo un id ed un nome (manca per ora la gestione dell'evento di tocco sul pulsante).

```
import React, {Component} from 'react';

class Contact extends React.Component{
  state = {
    id:0,
    name: "Prova"
  }
  render() {
    return <li>
      <h1> {this.state.name} </h1>
      <button>delete</button>
      <p>E' il mio amico con il nome di {this.state.name.length}
        caratteri </p>
    </li>
  }
}
export default Contact;
```

Poi questo componente dobbiamo importarlo in App.js, quando lo importiamo creiamo nello state la lista di contatti. Poi in jsx possiamo usare il contatto, è come se creassimo un nuovo tag.

```
import React, {Component} from 'react';
import Contact from './Contact'

class App extends React.Component {
  state = {
    contacts : [{uid: 0, name:"Andrea", pversione:0},
                {uid:1, name:"Bruno", pversion:1},
                {uid:2, name:"Carlo", pversione:2}
    ]
  }
  render() {
    return <div>
      <ul>{this.state.contacts.map(contact => <Contact
        key={contact.uid} />)}</ul>
    </div>
  }
}
export default App;
```

Ma come facciamo a passare dati tra componenti? ogni componente ha una variabile `props` che contiene le coppie chiave:valore del componente padre. A questo punto potremmo settare nello state `uid` e `name` con le variabili passate nel `props`.

```
// Contenitore
return <div>
<ul>{this.state.contacts.map(contact => <Contact key={contact.id}
  data={contact}/>)}</ul>
</div>

// Contact
state = {
  id: this.props.data.uid,
  name: this.props.data.name
}

render(){
  console.log(" Props " + this.props.data.name)
  // altro codice omissso
}
```

Esiste un altro modo per passare i dati, nel caso volessimo passare dal padre al figlio un `jsx`, c'è un particolare attributo `props.children` che mi fa leggere i dati dentro tag `jsx`.

9.4.1 Gestione eventi

Nella gerarchia di componenti i dati passano dalla radice verso sotto. Gli eventi sono gestiti dal primo che sono in grado di gestirli. Se abbiamo un evento che riguarda la riga di un singolo contatto se possiamo lo gestiamo lì, nel nostro esempio non possiamo fare gestire la cancellazione di un elemento dall'elemento stesso, è chi ha la lista di utenti che deve cancellarlo. Quindi generalmente gli eventi tornano indietro, passano dalle foglie alla radice. Devo quindi fare in modo che gli eventi tornino indietro, lo facciamo usando i riferimenti a funzione. Nel padre definisco la funzione di cancellazione (`handleDelete`) e passo con le `props` un riferimento di questa funzione al figlio.

```
handleDelete = (uid) => {
  console.log(uid)
  console.log(" Deleting " + uid)
  let newContacts = this.state.contacts.filter(c => c.uid !== uid)
  this.setState({ contacts : newContacts })
  console.log(this.state)
}
```

```
<button onClick={() => this.props.onDelete(this.state.id)}>delete</button>
```

9.4.2 Single source of truth

C'è un problema in questo codice, perchè andiamo a violare un principio di progettazione. Il principio della single source of thrut. Dobbiamo avere una singola sorgente dei dati, altrimenti capita che li modifichiamo da una parte e da un'altra rimangono uguali a prima. Il componente Contact ha uno stato che è inizializzato con i dati ricevuti dal componente padre (tramite props). Il componente Contact fa una copia dei dati passati come props e la usa per definire lo stato. Nell'esempio fatto fino ad ora questa cosa non crea problemi, ma immaginiamo di avere un pulsante anonimizza. Usiamo il metodo map per creare un nuovo array di oggetti e lo siamo per aggiornare lo stato, non funziona perchè non aggiorna lo stato.

```
handleMakeAnonymous = () => {
  let newContacts = this.state.contacts.map(c => {
    let res = {}
    res.id=c.id
    res.name = "anonymous"
    return res
  })
  this.setState({contacts : newContacts})
}
render() {
  return <div>
    <button onClick={() => this.handleMakeAnonymous()}> Make all
    anonymous</button>
  }
```

Dobbiamo fare in modo che i dati siano memorizzati solo dalla componente genitore (non da quella figlio), per ridisegnare il componente figlio, si fa riferimento a props. Quando il figlio ha un evento che deve modificare dei dati, si richiama una funzione del genitore, che li modifica nel suo state.

La soluzione è, al posto di avere una variabile locale nello stato del componente lo togliamo ed usiamo direttamente this.props.data.name:

```
class Contact extends React.Component {
  render() {
    return <li>
      <h1> {this.props.data.name} </h1>
      <button onClick={() => this.props.onDelete
        (this.props.data.id)}>delete</button>
      <p>E' il mio amico con il nome di
        {this.props.data.name.length} caratteri</p>
    </li>
  }
}
```

9.4.3 Componenti controllate

Nell'esempio precedente la componente Contact è rimasta senza uno stato (perchè tutti i dati vengono passati dal padre in modo da non duplicare i dati), questo è molto comune. Queste si chiamano componenti controllate, non gestisce eventi, ma li rimanda tutti al padre.

Si possono implementare con la sintassi delle stateless functional components, che è identica a prima ma un po' più compatta. Definiamo una costante X che è una funzione (definita con arrow notation) che ritorna quello che era ritornato dalla funzione render(). La funzione ha un parametro props. nel codice della funzione facciamo riferimento a props e non a this.props.

```
import React from 'react';

const Contact = (props) => {
  return <li>
    <hi> {props.data.name} </h1>
    <button onClick={() => props.onDelete(props.data.id)}>delete
    </button>
    <p>E' il mio amico con il nome di {props.data.name.length}
    caratteri </p>
  </li>
}
export default Contact;
```

Non è obbligatorio usare questa notazione, però è più compatta.

9.4.4 Sincronizzazione di componenti multiple

Nell'esempio precedente avevamo bisogno di sincronizzare due componenti in rapporto genitore-figlio. In generale potremmo aver bisogno di sincronizzare due o più componenti generiche (non solo padre e figlio), quindi dovremo continuare a passare tutte le info che ci servono con le props. In questi casi bisogna spostare lo stato nella componente antenato dei due o più componenti.

Un modo più comodo è definire un context, quando lo definiamo tutti i figli del componente hanno accesso ai dati con il context.

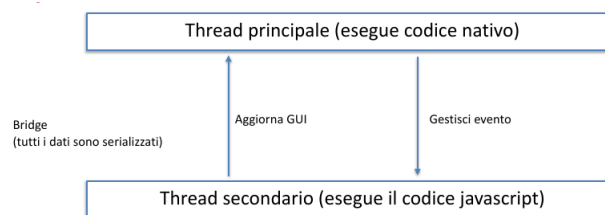
9.5 React Native

Iniziamo a vedere la parte di react native. Vedremo tra poco che quello che abbiamo imparato per react lo utilizzeremo in react native per realizzare.

E' un framework di sviluppo cross-platform di interfacce grafiche, questo perchè le interfacce grafiche sono il limite più grande per la programmazione ibrida. Nasce con l'idea di permettere lo sviluppo cross-platform senza dover dipendere da HTML.

9.5.1 Architettura del sistema

L'idea alla base è che abbiamo un thread (il main thread) che gestisce l'interfaccia grafica nativa usando degli oggetti di interfaccia nativi. Inoltre ha un thread secondario dove esegue la business logic dell'applicazione in JavaScript, che, tra le altre cose, crea le componenti, secondo l'architettura React. I due thread comunicano attraverso un canale di comunicazione chiamato bridge. A differenza di quello che succede in Java dove semplicemente chiamiamo un metodo. L'architettura è complicata, ma noi programmatori non lavoreremo mai sul thread principale ma solo su quello secondario.



Il DOM non c'è più, ma abbiamo il virtual DOM, perchè ereditiamo da React l'idea del virtual DOM, la differenza è che in react native il virtual DOM non è più rimappato sul DOM. Abbiamo oggetti di virtual dom che si rimappano su elementi nativi.

9.5.2 Come sono le performance

Ogni volta che avviene un evento sull'interfaccia grafica è necessario serializzare l'informazione e richiedere ad un altro thread di gestire l'evento. Se poi l'evento ha un effetto sull'interfaccia grafica (come spesso avviene), è necessario seguire il procedimento al contrario. Chiaramente il procedimento è più complesso di quando il codice esegue solo in modo nativo. Tuttavia, dal punto di vista dell'utente, questi ritardi sono solitamente impercettibili. Il fatto che l'interfaccia grafica esegue con codice nativo permette di avere una UX molto simile a quella di applicazioni scritte completamente in codice nativo. Nonostante l'architettura sia complicata le performance sono accettabili e si riesce a fornire una buona UX.

9.5.3 Uso di ReactNative

L'uso diretto di ReactNative avviene tramite una serie di tool chiamati ReactNative CLI. La gestione delle librerie e di tutte le dipendenze è particolarmente complicata. (eseguire un'app già sviluppata su una macchina diversa può richiedere ore di lavoro).

Da qualche anno c'è un sistema che sta sopra ReactNative, si chiama EXPO, è un tool che semplifica l'utilizzo di ReactNative, di fatto rende l'utilizzo molto semplice. Ci nasconde la gestione delle varie librerie e delle versioni mettendogli

un contenitore sopra. Ci semplifica la parte di deploy su dispositivi fisico e simulatore. Rimangono tutti i problemi sistemistici. Sicuramente è adatto per lo sviluppo di prototipi, ma secondo l'esperienza del prof si può anche sviluppare un'applicazione finale da pubblicare.

EXPO rende disponibile una serie di tool da linea di comando ed un client IOS ed Android per testare rapidamente le applicazioni su dispositivi fisico (app Expo su store) ed una serie di controlli tramite pagina web.

Anche qui abbiamo il live reload, quando un sorgente viene salvato, l'app cambia.

Possiamo leggere i messaggi di log dal terminale dove abbiamo fatto partire npm start e dalla finestra del browser che viene fatta partire da npm start. La creazione del progetto genera varie cartelle che per il momento non ci interessano e alcuni file, tra cui notiamo app.json che è un file di configurazione dell'app. e App.js che è la componente root dell'applicazione, è una functional component, quindi va modificata se vogliamo aggiungere lo stato.

Possiamo anche abilitare il remote debugging su chrome (lo vedremo più avanti).

9.5.4 Core component

Sono tag analoghi ad HTML ma hanno nomi diversi, sono forniti direttamente da ReactNative. Alcuni componenti sono specifici per iOS o Android, ad esempio il back button che esiste solo su Android.

Le componenti possono permetterci di fare anche operazioni non strettamente di interfaccia grafica, ad esempio vedremo delle componenti per gestire il ciclo di vita dell'applicazione.

Impariamo ad usare bene la [documentazione](#) perchè non ci spiegherà tutte le componenti, ma sono tutte lì.

Ricordiamo che come in React il programmatore può creare le sue componenti.

I componenti permettono di gestire elementi di interfaccia e API. I componenti grafici trasformano il virtual DOM in elementi grafici nativi (ci sono alcuni elementi nativi che sono specifici per una sola piattaforma e dunque funzioneranno solo su quella)

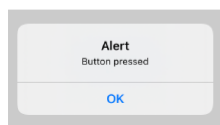
```
<View style={styles.container}>
  <Button title="Button example" onPress={() => alert("Button pressed")} />
</View>
```

iOS

Android

Button example

BUTTON EXAMPLE



9.5.5 Uso delle liste

Vediamo questo esempio per vedere come si usano le liste. Si usa la componente `FlatList`, si crea una costante `data` che contiene gli oggetti che vogliamo mostrare. Poi abbiamo un `renderItem` che è una funzione che prende in input un elemento dei dati e ritorna come dev'essere mostrato.

Facciamo attenzione che ogni core component che usiamo va importato!

9.6 Ciclo di Vita di un'applicazione ReactNative

Esistono una serie di componenti che non hanno un immediato effetto visivo sull'interfaccia grafica, ma sono usati principalmente per l'interazione con il OS, uno di questi componenti, `AppState` ci serve per gestire il ciclo di vita dell'applicazione. Ci dà accesso ad una serie di metodi che ci permettono di gestire il ciclo di vita, possiamo capire quale sia lo stato dell'applicazione (foreground o background). Abbiamo ad esempio `addEventListener` che ci permette di aggiungere una funzione di callback (funzione che passiamo ad una procedura e viene eseguita quando succede qualche cosa) quando c'è un cambiamento nello stato dell'applicazione. In React le applicazioni possono stare solo in background o in foreground, nell'esempio il `change` ci permette di definire un cambiamento (la funzione di callback) quando lo stato cambia.

Quando parliamo di ciclo di vita delle app ReactNative abbiamo due livelli:

- L'intera applicazione, componente `AppState`
- Ciclo di vita delle componenti

9.6.1 Ciclo vista componenti

Per ogni componenti abbiamo 4 fasi principali:

- **Mounting**: quando la componente viene importata nel virtual DOM. Un possibile uso del mounting è andare a scaricare dalla rete dei dati evitando di ricaricarli ogni volta che si aggiorna la componente.
- **Updating**: quando viene aggiornato lo state o cambiano le props
- **Unmounting**: quando la componente viene rimossa
- **Error-handling**: gestione degli errori

e per ogni fase abbiamo 1 o più metodi che possiamo sovrascrivere per personalizzare il comportamento della componente

Mounting

Quando la componente viene creata vengono chiamati in questo ordine i seguenti metodi:

- `constructor()`: del codice all'interno del costruttore. Tipicamente usato per inizializzare lo state. Può prendere props come parametro
- `getDerivedStateFromProps()`: usato molto raramente, ignoriamolo
- `render()`: unico metodo indispensabile, lo abbiamo già visto
- `componentDidMount()`: metodo che viene chiamato dopo che il componente è stato creato. Se è necessario fare una richiesta di rete, questo è un buon punto del codice. In questo caso nel primo render facciamo vedere una schermata di caricamento, poi chiamo il `componentDidMount()` e quando arrivano i dati aggiorniamo lo state e mostro un nuovo render con i dati che sono arrivati dalla rete.

Updating

Questi metodi vengono chiamati ogni volta che viene chiamato l'update.

- `getDerivedStateFromProps()`: ignoriamolo
- `shouldComponentUpdate()`: da usare quando, in rari casi, non vogliamo che un componente sia aggiornato. Per ottimizzare il codice
- `render()`: lo stesso metodo definito in fase di creazione
- `getSnapshotBeforeUpdate()`: usato raramente serve, ad esempio, per riposizionare la posizione corretta di scroll di una lista
- `componentDidUpdate()`: riceve in input le props precedenti (quelle prima dell'update) e quelle nuove, permette quindi di fare alcuni controlli come scaricare nuovamente dei dati dal server (se stiamo visualizzando lo stesso utente non richiediamo i dati, altrimenti sì)

Unmounting

Prima di togliere il componente dal virtual DOM ho bisogno di fare terminare l'azione del componente, ad esempio immaginiamo di avere un timer che fa qualcosa (lo faremo partire nel mounting) e per fermarlo lo faremo nel unmounting. Un caso più pratico è la gestione della posizione, faremo partire un metodo che periodicamente richiede la posizione e nell'unmounting possiamo interromperlo. Il metodo è `componentWillUnmount`.

Gestione delle eccezioni

Altri metodi per gestire le eccezioni, non credo che le useremo, ma comunque esistono due metodi per gestire le eccezioni:

- `static getDerivedStateFromError()`
- `componentDidCatch()`

9.7 Componenti funzionali e hook

Se programmiamo con le class component in realtà tutta questa parte non serve. In realtà da qualche anno React sta puntando forte sulle componenti funzionali, dicono in documentazione ufficiale che migliorano la leggibilità e testabilità del codice e migliorano le performance. Alla fine emerge che è una best practice, ma in realtà non è chiarissimo il motivo tecnico per cui bisogna fare questa cosa.

Come abbiamo già visto è possibile implementare alcune componenti tramite una singola funzione (che di fatto è la funzione `render`) senza stato, queste sono dette componenti funzionali. Per far funzionare queste componenti funzionali e per creare componenti funzionali con stato si sono inventati il concetto di HOOK, sono delle funzioni che ci risolvono dei problemi tipici delle componenti react. Per esempio di permettono di definire lo stato all'interno di una functional component, in particolar modo vedremo un hook per aggiungere lo stato ed uno per aggiungere il ciclo di vita dell'applicazione. Gli hook sono circa 15 e sono definiti dalla [documentazione](#). Sono funzioni che per convenzione iniziano con `use`.

9.7.1 useState

Nell'approccio a classe definiamo lo stato e poi lo aggiorniamo con il `setState`.

Se vogliamo farlo con un componente funzionale abbiamo la nostra funzione (example, notiamo che se avessimo dei props potremmo leggerli nei parametri della funzione), poi definisco con `useState` una variabile (associata ad un valore) ed una funzione che serve per aggiornare il valore della variabile. Facciamo caso che quando chiamiamo il `setCount` tutto il componente viene ridisegnato come se chiamassimo il `setState`.

Approccio "functional component"

```
import React, { useState } from 'react';

function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

Approccio "class component"

```
class Example extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }

  render() {
    return (
      <div>
        <p>You clicked {this.state.count} times</p>
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>
          Click me
        </button>
      </div>
    );
  }
}
```

Sergio Mascetti - Mobile Computing

15

E' un po' meno codice, ma poca roba, però che una cosa molto comoda, perdiamo il `this`, perchè non è più un oggetto ma una funzione, quindi `count` è una normale variabile dentro una funzione.

9.7.2 useEffect

Supponiamo di avere una componente class un po' complicata con parecchio codice nei vari metodi di gestione del ciclo di vita (oppure chiamare lo stesso codice in più funzioni del ciclo di vita), `useEffect` ci permette di gestire il ciclo di vita dell'applicazione anche nelle componenti funzionali (non potremmo sovrascrivere dei metodi in una funzione, è una funzione), ci permette di eseguire del codice dopo il `componentDidMount` e `componentDidUpdate`.

Nell'esempio vogliamo che dopo il click sul pulsante vogliamo aggiornare il titolo della pagina, notiamo che nello `useEffect` ha 1 parametro che è una fun-

Approccio “functional component”

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

zione (definite come arrow function) che non prende nulla in input ed esegue quel codice. La funzione parametro del `useEffect` viene chiamata dopo il `didMount` o il `didUpdate`.

E' più ordinato perchè se abbiamo tante cose da fare dopo il `didMount` usando le classi dobbiamo mettere tutto nella stessa funzione anche se fanno cose diverse, mentre in questo modo possiamo definire più `useEffect()` che fanno cose diverse (anche con diversi parametri).

Possiamo definire con dei trucchetti cosa fare quando c'è il `componentWillUnmount`, si ritorna una funzione che viene richiamata al `componentWillUnmount` (dentro la funzione del `useEffect` se ritorniamo una funzione quella funzione viene eseguita nel `WillUnmount`).

E' possibile anche raffinare il tutto, si può specificare come secondo parametro della funzione quali variabili vogliamo osservare, lo `useEffect` viene eseguito solo se quelle variabili sono state modificate. Nell'esempio di prima vogliamo fare una nuova richiesta al server solo se è cambiato l'utente che stiamo mostrando.

Di fatto gli hook sono dei modi per gestire rapidamente dei casi comuni che si possono gestire con un po' più di codice con i class component.

Nel secondo punto degli esercizi bisogna mettere a log quali metodi del ciclo di vita dell'applicazione e delle componenti.

9.8 Thread

In JavaScript un programmatore interagisce con un solo thread. Ma se abbiamo un solo thread come facciamo a fare le chiamate bloccanti? il programmatore interagisce con un solo thread e sotto probabilmente ci sono le chiamate bloccanti, però il programmatore non lo vede. Volendo esistono delle estensioni di JavaScript multi thread, ma non ci interessa. Siamo contenti sia single thread, perchè la gestione multi thread è un casino, però siccome è tutto basato su chiamate asincrone dobbiamo diventare dei draghi sulle chiamate asincrone, anche perchè in JS hanno un modo per essere implementate figo, vediamo 3 approcci:

- Callback
- Promise
- Async-await: è la cosa fida, ma bisogna usarle insieme alle altre cose

9.8.1 Callback

Storicamente in JS le chiamate asincrone sono gestite tramite callback. E' una funzione richiamata quando c'è un certo evento, quindi lo facevamo già quando facevamo un pulsante. Con delle chiamate di rete succede che quando la chiamata termina esegui questo codice. Nell'addEventListener mettiamo la funzione di callback che viene eseguita quando è finita la richiesta di rete.

```
const req = new XMLHttpRequest();
req.addEventListener("load", reqListener);
req.open("GET", "http://www.example.org/example.txt");
req.send();
```

Il problema di questa cosa è quando abbiamo nella funzione di callback un'altra chiamata bloccante, ad esempio chiediamo l'elenco degli utenti ad un server, quando arriva vogliamo vedere se abbiamo l'immagine del profilo, è una chiamata bloccante, scriviamo una callback che dice se i dati ci sono o meno, nella callback di risposta abbiamo un'altra callback per chiedere le nuove immagini dal server. Questa cosa qui si chiama callback hell, è un casino questo codice.

```
fs.readdir(source, function (err, files) {
  if (err) {
    console.log('Error finding files: ' + err)
  } else {
    files.forEach(function (filename, fileIndex) {
      console.log(filename)
      gm(source + filename).size(function (err, values) {
        if (err) {
          console.log('Error identifying file size: ' + err)
        } else {
          console.log(filename + ' : ' + values)
          aspect = values.width / values.height
          widths.forEach(function (width, widthIndex) {
            height = Math.round(width / aspect)
            console.log('resizing ' + filename + 'to ' + height + 'x' + height)
            this.resize(width, height).write(dest + 'w' + width + '_' + filename, function(err) {
              if (err) console.log('Error writing file: ' + err)
            })
          })
        }
      }).bind(this)
    })
  }
})
```

9.8.2 Promise

Per questo motivo è stato introdotto un livello di astrazione superiore, le promesse (promise). Una promessa è un oggetto che rappresenta un'operazione in fase di svolgimento, promette che quando l'operazione è finita esegue del codice. Una promessa ci dà un risultato strano, perchè non possiamo accederci come fosse un normale risultato di una qualsiasi funzione. Dobbiamo leggere il risultato quando è stato calcolato, lo facciamo con il metodo `then` della classe `Promise`. `Result` è una funzione che prende come input il risultato e fa qualcosa, in realtà ha anche un parametro `error` che mi dice come gestire i casi di errore.

```
p.then(result => {gestione risultato}, error => {gestione errore})
```

Vediamo l'esempio di `fetch`, che fa una richiesta di rete, `fetch` ritorna `p` che è una promise, sulla promise richiamo il metodo `then` che vuol dire "quando hai finito di fare quello che stai facendo, mi esegui questo codice e se c'è stato un errore fai qualcos'altro".

```
let p = fetch("https://picsum.photos/200")
p.then(result => console.log("prima foto scaricata"),
error => console.log("Ops, c'e' un errore"))
```

Solitamente non si sta mai a creare una variabile che si riferisce alla promise, ma ci attacco direttamente in `.then`.

```
fetch("https://picsum.photos/200")
.then(result => console.log("prima foto scaricata"),
error => console.log("Ops, c'e' un errore"))
```

Se anche fosse che per qualche motivo prima calcoliamo il risultato con la `fetch` e successivamente ci attacchiamo il `then` lo gestisce senza errori.

Quindi di fatto possiamo usare questo metodo sapendo che funziona.

Catch

Il `then` prende in input due parametri, il secondo è una funzione che dice cosa fare in caso di errore, in realtà non si fa così, si usa un `then` quando tutto va bene e si concatena un `CATCH` nel caso in cui ci sia un errore.

```
fetch("https://picsum.photos/200")
.then(result => console.log("prima foto scaricata"))
.catch(error => console.log("Ops, c'e' un errore"))
```

Vediamo un esempio per capire bene l'ordine di esecuzione:

```
console.log("Log 1")
fetch("https://picsum.photos/200")
.then(result => console.log("Log 2"))
console.log("Log 3")
```

Il senso di tutto questo è che facciamo partire la richiesta ma poi andiamo avanti, quindi stampa prima `log3` di `log2`.

Concatenazione di then

Sfruttando la stessa cosa detta del catch possiamo concatenare più then tra di loro, supponiamo di fare un primo download, poi abbiamo una fetch, nel risultato della fetch mettiamo un'altra fetch, noi ritorniamo una fetch, il then ritorna un'altra promise che però è legata alla fetch dopo.

```
console.log("Inizio download")
fetch("https://picsum.photos/200")
  .then(result => {
    console.log("Immagine 1 ricevuta")
    return fetch("https://picsum.photos/100")
  }).then(result => {
    console.log("Immagine 2 ricevuta")
  })
  .catch(error => console.log("Ops, c'è un errore"))
```

E' vero che si può fare così, ma se abbiamo una situazione in cui abbiamo una chiamata asincrona che al suo interno nel codice di gestione abbiamo una chiamata asincrona questo modo risulta incasinato.

9.8.3 Async

Questa è una parte molto delicata. Async è un modificatore di funzione, posso metterlo come parola riservata prima di definire una funzione, mi rende la funzione che sto creando asincrona. Cosa succede se definisco una funzione come asincrona, vuol dire che quando la chiamo il mio codice intanto che la chiama va avanti ad eseguire, non possiamo quindi usare un risultato di una funzione asincrona come se fosse qualsiasi risultato di una funzione sincrona. Quindi qualsiasi funzione async ritorna una promessa devo quindi usare un then per leggere il risultato, il risultato della promessa sarà quello che mettiamo nel return.

```
async prova() {
  console.log("Chiamata asincrona")
  return "ciao"
}

handlePress = () => {
  console.log("Sto per fare una chiamata asincrona")
  this.prova().then(result => console.log(result))
  console.log("dopo chiamata asincrona")
}
```

Definiamo una funzione async prova e gli facciamo ritornare ciao, quando nel codice stampo quella cosa e poi fa this.prova, che non ritorna ciao, ma ritorna una promise, alla quale posso mettere un .then. Prendo il result e stampo il risultato. Attenzione che stampa prima il secondo console.log. Notiamo che il log nella chiamata asincrona viene eseguito quando chiamiamo la funzione, non con il then.

Await

Lo usiamo prima di richiamare una funzione, lo possiamo usare soltanto nelle funzioni asincrone. Se ho definito una funzione async dentro posso metterci delle chiamate bloccanti, posso rimanere in attesa che una certa chiamata termini, await vuol dire proprio aspetta.

```
async prova() {
  console.log("Scarico la prima immagine")
  const res1 = await fetch("https://picsum.photos/200")
  console.log("Scarico la seconda immagine")
  const res2 = await fetch("https://picsum.photos/100")
  //do something with res1 and res2
  return
}

handlePress = () => {
  console.log("Sto per fare una chiamata asincrona")
  this.prova()
  .then(result => console.log("finito di scaricare"))
  .catch(error => console.log("something wrong"))
  console.log("dopo chiamata asincrona")
}
```

Immaginiamo di creare due chiamate di rete, definiamo prova come async, siccome è asincrona dentro posso chiamare le funzioni fetch usando davanti la parola riservata await, l'effetto è che rimane bloccato lì finché il fetch non termina, e restituisce IL RISULTATO DELLA PROMISE, posso farlo proprio perchè sono dentro una funzione async. Attenzione che quando chiamiamo la funzione async dobbiamo metterci il .then per leggere il risultato. Non abbiamo più nessuna funzione di callback perchè con await facciamo diventare la fetch una chiamata sincrona, perchè il thread si ferma lì ad aspettare che fetch abbia finito.

	Quando viene eseguita la riga successiva?	Che valore ha a?
a=f() con f sincrona	Quando f() termina	Quello indicato nel return di f
a=f() con f asincrona	Immediatamente, in parallelo a f()	Una promise che contiene il valore indicato nel return di f (ci accedo con .then)
a=await f() con f sincrona	Non ha senso!	Non ha senso!
a = await f() con f asincrona	Quando f() termina	Quello indicato nel return di f

9.8.4 Strumenti di comunicazione in React

Ne abbiamo tanti:

- Fetch per chiamate HTTP
- WebSocket
- Varie librerie di terze parti

ma vedremo Fetch, è una chiamata che estende altre chiamate, solitamente utilizzate nella comunicazione in JavaScript, ha due parametri:

- La URL
- Un oggetto con tutti gli altri parametri della richiesta

9.8.5 Fetch risposta

Esempio chiamata POST (non usiamo l'estensione php)

```
fetch('https://develop.evlab.di.unimi.it/mc/twittok/register',{
  method: 'POST',
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json'
  }
})
```

9.8.6 Fetch gestione risposta

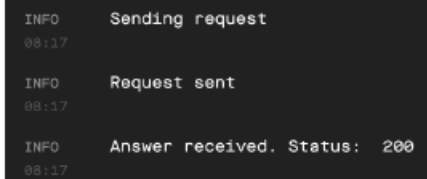
Fetch ritorna una Promise, il risultato della Promise è il risultato della chiamata di rete, ci accediamo con `.then()`. Con `catch()` gestiamo gli eventuali errori di comunicazione. Va sempre definito il catch, quindi come minimo mettiamo il catch che stampi a log che c'è stato un errore ed il tipo di errore. (se devo fare una sola chiamata conviene usare solo la promise, è quando ne ho tante che è comodo usare `async` e `await`).

```

const handlePress = () => {
  const URL = "https://ewserver.di.unimi.it/mobicomp/treest/register.php"

  console.log("Sending request")
  fetch(URL, {
    method: 'POST',
    headers: {
      Accept: 'application/json',
      'Content-Type': 'application/json'
    }
  }).then(result => {
    console.log("Answer received. Status: ", result.status)
  })
  console.log("Request sent")
}

```



```

INFO      Sending request
08:17

INFO      Request sent
08:17

INFO      Answer received. Status: 200
08:17

```

La funzione `fetch` fa partire la richiesta ma non può ritornare il risultato, perchè altrimenti sarebbe una chiamata bloccante. Ritorna quindi una promessa, che prima o poi darà un risultato (o un errore). Con il metodo `then()` diciamo cosa fare quando arriverà il risultato, ma il codice passato come argomento di `then()` viene eseguito solo quando riceviamo una risposta. Intanto l'esecuzione prosegue: per questo la scritta "Request sent" viene stampata prima di "Answer received", anche se "Answer received" sta "più in alto" nel codice.

Notiamo che quando si programma in JavaScript per React Native si usano molto le funzioni definite con arrow notation che vengono definite dove vanno usate (come nell'esempio sopra). Se queste funzioni contengono pochissime righe di codice, possiamo lasciarle in quella posizione, altrimenti è molto più ordinato estrarle in una funzione esterna a cui fare riferimento:

```

handleRegisterAnswer = (result) => {
  console.log("Answer received. Status: ", result.status)
  //a lot of other code here
}

handlePress = () => {
  const URL = "https://ewserver.di.unimi.it/mobicomp/treest/register.php"

  console.log("Sending request")
  fetch(URL, {
    method: 'POST',
    headers: {
      Accept: 'application/json',
      'Content-Type': 'application/json'
    }
  }).then(this.handleRegisterAnswer)
  console.log("Request sent")
}

```

9.8.7 Inviare dei dati

Dobbiamo usare il metodo `stringify` che trasforma un oggetto JS nell'equivalente string JSON.

```
fetch('https://ewserver.di.unimi.it/mobicomp/treest/getLines.php', {
  method: 'POST',
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    sid: "8pvADV3GiGgBXlIG" //usate un sid corretto, questo è inventato
  })
}).then(response => {
```

Bisogna fare attenzione ai casi di errore, perchè dal punti di vista teorico sono 2, ma dal punto di vista pratico abbiamo gli errori. In generale nella `fetch` ci sono dei casi di errori quando la richiesta non termina, ma se la richiesta termina ed il server ci manda un messaggio di errore quell'errore non va nel `catch` ma dobbiamo gestirlo nel `then` (ad esempio lo `status`). In ogni caso l'utente va notificato.

9.8.8 Estrazione del risultato

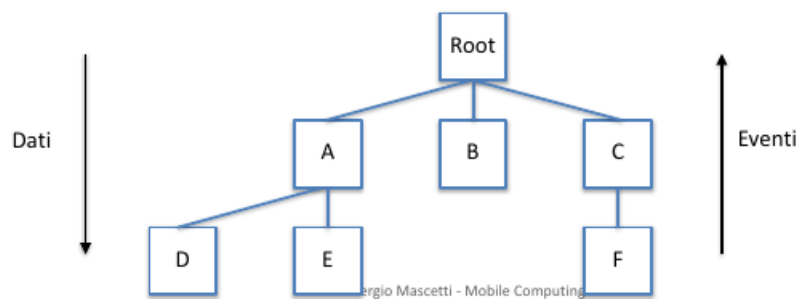
Se la richiesta è andata bene dobbiamo estrarre il risultato, si fa con un metodo asincrono perchè potrebbe metterci un po' di tempo, si chiama `json()`. Facciamo la `fetch`, quando ricevo il risultato controllo il risultato, se è 200 faccio `result.json()` che è una chiamata asincrona che ritorna una `promise`, se lo facciamo con il `async` e `wait` è più facile da gestire. Il secondo `.then()` è eseguito sulla `promise` ritornata da `.json()`.

```
fetch(URL, {
  method: 'POST',
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    sid: sid
  })
})
.then(result => {
  if (result.status == 200) {
    console.log("request OK")
    return result.json()
  } else {
    console.log("request KO")
    //TODO: Manage this case
  }
})
.then(unmarshalledObject => {
  console.log(unmarshalledObject)
}).catch(error => {
  console.log("error")
  //TODO: Manage this case
})
```

L'esercizio va suddiviso in passaggi intermedi, altrimenti non lo facciamo mai. (facciamo una register e facciamo una richiesta per prendere un sessionID). Il tag image di ReactNative se ci mettiamo dentro una roba base64 ce la mostra (prendiamo quello che restituisce il server ci mettiamo in testa il prefisso del base64).

9.9 Organizzazione del codice

Per il momento stiamo scrivendo codice senza organizzarlo troppo, o meglio organizziamo solo la suddivisione in componenti, suddividiamo il progetto in componenti dove ogni componente rappresenta una parte dell'interfaccia. Però è molto importante ad organizzare il codice al di là di queste componenti.



Quando abbiamo una gerarchia di componenti React i dati vengono passati dai genitori ai figli, i figli richiamano le funzioni di gestione degli eventi dei genitori. Non è detto che tutti i dati debbano essere gestiti dalla root e non è detto che tutti gli eventi debbano risalire alla root. Ci sono due regole:

- Non vogliamo duplicare il dato
- Chi genera un dato non lo può passare a quelli che non stanno nel suo sottoalbero (A può passare a D ed E e non a B)

Quindi di solito quando voglio capire quel'è il componente che deve creare un dato e passarlo alle componenti figlie devo capire chi ha bisogno di quel dato, va messo nel nodo in cui i figli del sottoalbero hanno bisogno di quel dato. Come passa i dati:

- Props
- Context

9.9.1 Context

A volte succede che vogliamo poter utilizzare un dato in tanti componenti del sottoalbero e non ho voglia di usare le props per passare iterativamente quel dato, in questi casi si può usare il context, root definisce una context e tutti

possono accederci, ma non è una cosa consigliata da fare sempre (nel nostro progetto un dato a cui tutti devono accedere è il session id, perchè in tutte le richieste verso il server va fatto).

Devo nel padre:

- Creare un contesto, con una variabile
- Indico un componente di tipo contesto appena creato
- Tutti i componenti che sono sotto possono accedere al dato (vanno indicati)

```
MyContextValue = createContext();
<MyContextValue.Provider value={user}>
//componenti figli qua
</UserContext.Provider>
```

Nei figli ho due modi diversi:

- Se è un functional component ho una sola funzione, ho un hook useContext
- Se è un class component devo sovrascrivere una variabile statica di classe e dopo ci acceco con this.context

```
//se sono functional component
const myData = useContext(MyContextvalue);
```

```
//se sono classi
static contextType = MyContextValue
//ci accedo
this.context
```

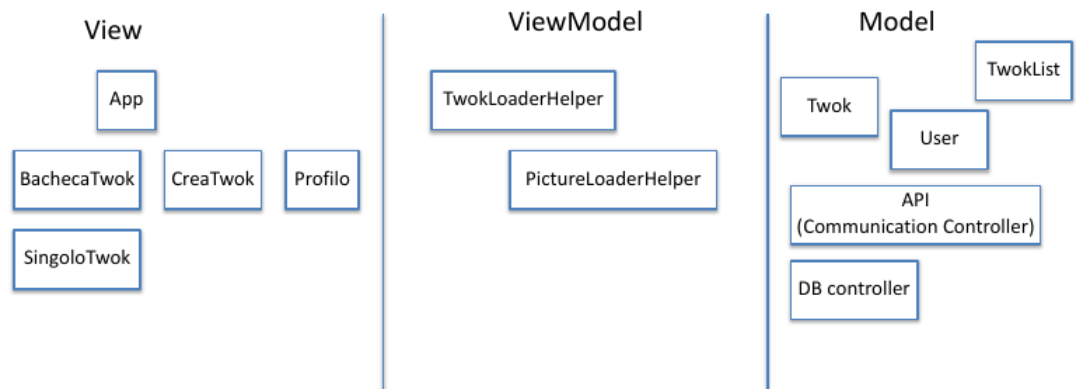
Fino ad ora ci siamo preoccupati dell'interfaccia utente (react nasce per questo) tuttavia negli esercizi che stiamo svolgendo c'è via via sempre più codice che non riguarda solo l'interfaccia:

- Le strutture dati
- La comunicazione con il server
- La memorizzazione persistente

questo codice dove lo mettiamo? L'idea è tenere le componenti più semplici possibili e se vogliamo eseguire delle operazioni sui dati vogliamo farlo nelle classi.

Quello che vogliamo evitare è che le componenti esplodano, altrimenti ci troviamo con componenti enormi difficili da debuggare, dobbiamo metterci nell'ottica che c'è uno strato che espone dei metodi alle componenti che possono richiamare con una riga di codice. Questo è molto vicino all'approccio MVVM (model view viewModel):

- View: componenti
- Model: dati (la classe che rappresenta i twok, la lista di twok), la definizione delle classi, delle strutture dati, le operazioni sui dati, come lettura e scrittura da rete e da memoria persistente.
- ViewModel: possiamo definire un layer del codice, cioè quel livello che prepara i dati che devono essere dati alla view. Tutto quello che non è model, ma sono dati che devono essere preparati per la view. La view-Model è comoda da debuggare perchè possiamo debuggurla con node da riga di comando.



9.9.2 Problema della navigazione, ReactNavigation

Un altro problema relativo all'organizzazione del codice è quello della suddivisione in pagine. Fino ad ora abbiamo implementato la navigazione attraverso una variabile dello stato e dei costrutti di selezione (`if(page == 1) {renderFirstPage()}.`). Esiste [ReactNavigation](#) che è una libreria che semplifica l'utilizzo di questa parte del codice. Lo studiamo da soli (ci vuole un po' di tempo ma ne vale la pena perchè ci fa risparmiare molto tempo nello sviluppo del progetto).

9.9.3 Organizzazione del communication controller

Precedentemente abbiamo visto come fare una richiesta di rete, possiamo mettere il codice direttamente nel componente (ma complica inutilmente la complessità). Identifichiamo tre blocchi concettuali di codice relativi alle richieste:

- inviare la richiesta
- gestire la risposta, le parti comuni come estrazione del risultato, controllo degli errori ...

- Gestione effettiva della logica della risposta, ad esempio mostrare i dati all'utente ...

Le prime due parti sono ampiamente generalizzabili. Cerchiamo di trovare un modo per rendere le prime due parti comuni.

Abbiamo due parti del nostro progetto dove possiamo implementare il codice di gestione delle richieste:

- I component
- Una classe di supporto, che chiameremo Communication Controller (CC)

Dobbiamo bilanciare varie esigenze:

- Semplificare e ridurre il codice nel component
- Evitare ripetizioni nel codice
- Limitare la complessità generale del codice

La prima soluzione è tenere tutto nel component e non implementiamo il communication component (sconsigliato), oppure facciamo le richieste e la gestione comune nel Communication controller e nel component gestiamo la logica applicativa delle risposte. Per implementare la seconda soluzione spostiamo nel CC la gestione degli errori e l'unmarshalling dei risultati. Inoltre usiamo `async-await` perchè viene incredibilmente semplice.

Il `communicationController` è un metodo di una classe, la gestione dell'errore con `throw` è comoda perchè passo l'errore fuori.

```
async getLines(sid) {
  const parameters = {sid: sid};
  const url = "https://ewserver.di.unimi.it/mobicomp/treest/getLines.php"
  let httpResponse = await fetch(url, {
    method: 'POST',
    headers: {
      Accept: 'application/json',
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(parameters)
  });
  const status = httpResponse.status;
  if (status == 200) {
    let deserializedObject = await httpResponse.json();
    return deserializedObject;
  } else {
    let error = new Error("Error message from the server. HTTP status: " + status);
    throw error;
  }
}
```

Di fatto se creo una classe con questo metodo (`getLines`), quando un componente richiama il metodo otterrà un oggetto che contiene le informazioni che servono, di fatto per ogni endpoint del server possiamo creare un metodo per avere il risultato in oggetto JS.

```

constructor() {
  super();
  const cc = new CommunicationController();
  cc.getLines("ABC") //usate un vero SID
  .then(result => {
    this.state.lines = result.lines
    this.setState(this.state)
  })
  .catch(error => {
    console.log("Error: " + error)
    //TODO gestire l'errore
  })
}

```

Solo in questo esempio la classe veniva usata nel costruttore, ma si può mettere in vari punti del codice, ad esempio alla pressione di un pulsante oppure al mounting della componente.

Ricordiamoci che la classe va esportata per poi usarla da una altra parte, possiamo anche generalizzare le richieste. Possiamo ulteriormente generalizzare il CC per evitare di avere codice duplicato. Definiamo una costante URL di base che è tutto il percorso tranne l'endpoint e poi definiamo un metodo che è una richiesta generica per il server, in cui passiamo l'endpoint ed i parametri.

```

export default class CommunicationController {
  static BASE_URL = "https://ewserver.di.unimi.it/mobicomp/treest/"

  static async treestRequest(endpoint, parameters) {
    console.log("sending request to: " + endpoint);
    const url = this.BASE_URL + endpoint + ".php";
    let httpResponse = await fetch(url, {
      method: 'POST',
      headers: {
        Accept: 'application/json',
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(parameters)
    });
    const status = httpResponse.status;
    if (status == 200) {
      let deserializedObject = await httpResponse.json();
      return deserializedObject;
    } else {
      let error = new Error("Error message from the server. HTTP status: " + status);
      throw error;
    }
  }
}

```

Questo metodo non lo vogliamo esporre alla componente, ma definiamo dei metodi specifici (sempre nella classe CC) per ogni chiamata che usano la richiesta generale.

Può anche essere che non sia il componente ad usare le richieste di rete, ma un'altra classe di viewModel che ha un metodo che usa i metodi del communicationController. Attenzione che nell'ultima versione essendo diventato statico il metodo non dobbiamo piu' creare un istanza dell'oggetto, ma basta richiamarlo

```

static async getLines(sid) {
  const endPoint = "getLines";
  const parameter = {sid: sid};
  return await CommunicationController.treestRequest(endPoint, parameter);
}

static async register() {
  const endPoint = "register";
  const parameter = {};
  return await CommunicationController.treestRequest(endPoint, parameter);
}
}

```

al volo.

Nell'esercizio 2 creiamo un script node per fare delle prove di richiesta dei dati. Nell'esercizio 3 bisogna fare la progettazione di tutte le componenti del progetto (fare attenzione che esistono delle pagine per cui si possono usare delle componenti in comune) hook reducer per gestire stati complessi con le componenti funzionali (useReducer).

9.10 Gli stili

In ReactNative si possono definire degli stili. La sintassi e alcune costanti sono simili a quanto avviene per i CSS, ma in realtà non sono CSS, ma normali oggetti JS. Quando creiamo un progetto nuovo ci crea una costante styles che è un modo in react native per creare gli stili, ci ricorda il CSS, abbiamo un oggetto JS i cui attributi ricordano gli attributi del CSS, ma non è esattamente CSS ha delle proprietà da guardare in documentazione. Per definire gli stili, passiamo come attributo degli elementi di ReactNative un oggetto con alcuni attributi, vedremo tra poco alcuni esempi di attributi. Ovviamente possiamo definire l'oggetto dove più comodo:

- Direttamente dove va usato
- Attraverso una variabile

Come dicevamo la classe App.js contiene questo codice:

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});

export default function App(){
  return (

```

```

    <View style={styles.container}/>
  )
}

```

Cosa succede se sbagliamo a definire uno stile? (ad esempio se scriviamo male il nome di un attributo) il comportamento non sarà quello atteso, ce ne accorgiamo a tempo di esecuzione. Per fare emergere i problemi tempo di compilazione, possiamo usare il metodo `create` della classe `StyleSheet`: crea una serie di oggetti da usare come stili e ne verifica la correttezza sintattica a tempo di compilazione.

Quando creiamo una `View` ci assegniamo lo stile. Per il progetto dobbiamo creare un'interfaccia che sia un minimo personalizzata. Guardiamo alcuni aspetti relativi all'organizzazione degli elementi sullo schermo. In Android c'è un oggetto molto espressivo che permette di posizionare alcuni elementi, in React native usa un approccio a layout delle componenti, abbiamo una componente che può contenere 1 o altre che a loro volta possono contenerne altre. Per una lista completa vedere la [documentazione](#)

9.10.1 Dimensioni

In questo esempio abbiamo una view che ne contiene altre. Tipicamente si usa una view radice che contiene tutto ed al suo interno dividiamo i vari elementi in view che a loro volta possono avere al loro interno altre view.

Width ed height in percentuale si riferiscono alla view padre.

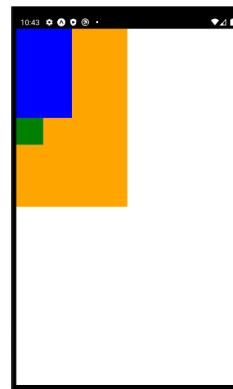
Le dimensioni di un oggetto di interfaccia si possono indicare:

- In dip:
- In percentuale rispetto al padre:
 - Con una percentuale esplicita
 - Usando flex

```

export default function App() {
  return (
    <View style={{
      backgroundColor: "orange",
      width: "50%",
      height: "50%"
    }}>
      <View style={{
        backgroundColor: "blue",
        width: "50%",
        height: "50%"
      }}></View>
      <View style={{
        backgroundColor: "green",
        width: 50,
        height: 50
      }}></View>
    </View>
  );
}

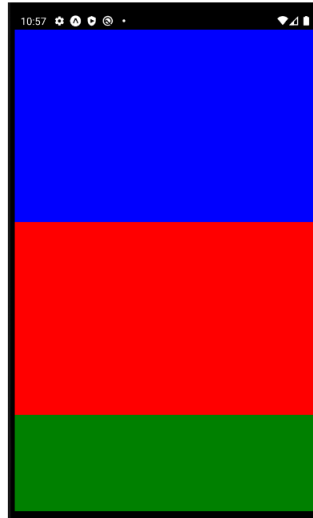
```



Flex

Abbiamo una view esterna (completamente coperta dagli altri, quella arancione) che ne contiene altre tre, siccome non abbiamo indicato niente dispone gli oggetti in verticale. Ognuno degli elementi occuperà una dimensione definita dal flex, si dividono lo spazio del padre rispetto al valore di flex.

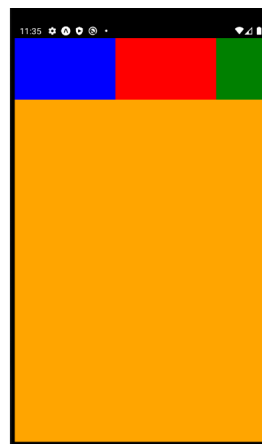
```
export default function App() {
  return (
    <View style={{
      backgroundColor: "orange",
      flex:1
    }}>
      <View style={{
        backgroundColor: "blue",
        flex: 2
      }}/>
      <View style={{
        backgroundColor: "red",
        flex: 2
      }}/>
      <View style={{
        backgroundColor: "green",
        flex: 1
      }}/>
    </View>
  );
}
```



Spesso consiglia quando dobbiamo prendere dimestichezza con queste divisioni è usare elementi con colore di background.

Possiamo anche organizzare gli elementi in orizzontale cambiando direzione dell'asse principale, lo dico nella view con `flexDirection:'row'`. Con flex in questo caso indichiamo la larghezza.

```
export default function App() {
  return (
    <View style={{
      backgroundColor: "orange",
      flex:1,
      flexDirection:"row",
    }}>
      <View style={{
        backgroundColor: "blue",
        flex: 2,
        height: 100
      }}/>
      <View style={{
        backgroundColor: "red",
        flex: 2,
        height: 100
      }}/>
      <View style={{
        backgroundColor: "green",
        flex: 1,
        height: 100
      }}/>
    </View>
  );
}
```



9.10.2 Distribuzione del contenuto

Se abbiamo degli elementi che non occupano tutto lo spazio abbiamo dei modi per organizzare il contenuto in maniera ordinata, se non diciamo niente ci organizza da sinistra verso destra (o dall'alto verso il basso dipende dall'asse principale).

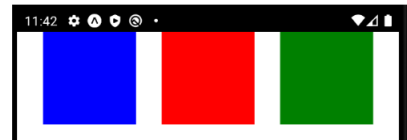
MyTutorial

```
export default function App() {
  return (
    <View style={{
      flex:1,
      flexDirection:"row",
    }}>
      <View style={{
        backgroundColor: "blue",
        width: 100,
        height: 100
      }}/>
      <View style={{
        backgroundColor: "red",
        width: 100,
        height: 100
      }}/>
      <View style={{
        backgroundColor: "green",
        width: 100,
        height: 100
      }}/>
    </View>
  );
}
```



```
<View style={{
  flex:1,
  flexDirection:"row",
  justifyContent: "space-evenly"
}}>
```

justifyContent si riferisce all'asse principale



```
<View style={{
  flex:1,
  flexDirection:"row",
  justifyContent: "space-between"
}}>
```



Possiamo anche allinearli rispetto all'asse secondario si usa `alignItems`. Possiamo anche usare `flexWrap` per andare a capo se si esaurisce lo spazio sull'asse principale. Vediamo in [documentazione](#) anche `flexGrow` e `flexShrink`.

(è sempre meglio tenere separata la costante dello stile).

9.10.3 Posizioni

Si possono definire posizioni relative o assolute:

- Relative: alla posizione decisa dal padre rispetto ai parametri flex forniti
- Assolute: rispetto al padre, ignorando il resto



```
left: 30,
top: 30,
```



```
left: 30,
top: 30,
position: "absolute"
```



9.11 Memorizzazione persistente

Ci sono negli ambienti di programmazione mobile 3 modi per salvare in maniera persistente l'informazione (in realtà c'è anche il file, ma non ne parliamo):

- Una tecnica che permette di salvare le cose con coppie chiave-valore, comodo ma utilizzabile per piccole quantità di dati, ad esempio il SID, per vedere se siamo al primo avvio dell'app si usa questa cosa
- Utilizzo del database, potrebbe sembrare simile a quello fatto per PHP
- C'è una tecnica in più che vedremo solo in Android, quando facciamo le query e leggiamo un dato, leggiamo riga per riga una serie di attributi, se dobbiamo creare un oggetto dobbiamo prendere questi attributi e popolare un oggetto (marshalling ed unmarshalling dei dati per il DB), esistono alcuni DB che offrono un livello di astrazione superiore in cui definiamo le query in una certa sintassi ed abbiamo regalato il fatto che possiamo fare un insert di un oggetto nel DB. Addirittura in XCode possiamo definire visualmente la struttura dati e lui ci crea il DB e gli oggetti definiti per il DB.

9.11.1 Memorizzazione coppie chiave-valore

Si usa una libreria [AsyncStorage](#) (non fa parte di RN) che va installata ed importarla quando la usiamo. Abbiamo due metodi:

- `AsyncStorage.getItem`: indichiamo la chiave e ci restituisce il valore
- `AsyncStorage.setItem`: passiamo la chiave ed il valore, se la chiave esiste già la chiave sostituisce il valore, altrimenti crea una nuova coppia chiave-valore.

Attenzione che sono chiamate asincrone, dunque ritornano una promise. Si può gestire con il `.then()`, meglio se con `async` e `await`.

```
async checkFirstRun() {  
  const firstRun = await AsyncStorage.getItem("first-run")  
  if (firstRun) {  
    console.log("Second run");  
  } else {  
    console.log("first run");  
    await AsyncStorage.setItem("first-run", "false");  
  }  
}
```

In questo esempio non è quello che dobbiamo fare nel progetto, perchè tendenzialmente vogliamo ritornare qualcosa se non siamo al primo avvio. Prima

cerchiamo nel local storage se c'è la chiave first run, se c'è significa che non siamo al primo avvio e stampiamo second run. Altrimenti se la variabile non è settata stampo e la setto.

9.11.2 Memorizzazione su DB

Esistono molte tecnologie che permettono la memorizzazione persistente:

- Real (tecnologia basata su MongoDB)
- Firebase: tecnologia di Google pensata per la memorizzazione persistente in locale e nel cloud
- SQLite: libreria analoghe a quelle SQL

Esiste una differenza sostanzialmente tra quello che facevamo con PHP e quello che faremo ora con SQLite. Quando abbiamo un server che esegue codice PHP, il DBMS è un processo differente che può girare anche su un'altra macchina. Mentre su mobile c'è un'altra architettura, non abbiamo un DBMS, abbiamo una libreria che ci espone tramite normali metodi dei metodi analoghi a quelli che ci espone un normale DBMS ma siamo sullo stesso processo.

La libreria prende i dati che gli diamo e gli scrive un file organizzato che contiene le informazioni del DB.

Come prima dobbiamo installare la [libreria](#) e poi importarla.

C'è una chiamata (non bloccante) per accedere ad un DB:

```
const db = SQLite.openDatabase("myDB")
```

Abbiamo una classe SQLite con un metodo statico, gli passiamo una stringa con il nome del DB e ci torna un oggetto di tipo WebSQLDatabase.

Per fare le query dobbiamo fare delle transazioni, creiamo una transaction e ci mettiamo dentro una funzione chiamando un metodo executeSql che esegue la query definita.

```
const query = "CREATE TABLE IF NOT EXISTS PROVA (value TEXT NOT NULL);"
db.transaction(tx => {
  tx.executeSql(query);
})
```

Quando lavoriamo su un web server non creiamo le tabelle da codice, ma lo facciamo a mano sul server, ma in mobile non lo abbiamo un DB, quindi quello che si fa è al primo avvio eseguire una query per creare le relazioni che ci servono. Le query vanno ovviamente testate prima di essere inserite nel codice. Esistono dei tool online che permettono di eseguire delle query SQLite e vedere il risultato.

Eseguire delle query

Il metodo executeSQL in realtà ha quattro parametri:

- La query da eseguire (una stringa)
- Un array di valori da sostituire ai placeholder
- La callback per eseguire codice quando la chiamata al DB termina
- La callback per gestire gli errori nella chiamata al DB

C'è un modo per trasformare il database in promise, c'è sulle slide dell'anno scorso ed ha detto che è un po' incasinato.

Placeholder

E' buona norma di programmazione quando abbiamo un codice SQL non concatenare stringhe a nomi di variabili. Questo perchè è vulnerabile ad attacchi di SQLInjection. Si usano dei placeholder (carattere ?) all'interno della query, poi ho un array di stringhe che vengono sostituiti ai placeholder. Si fa questo perchè controlla in automatico che quello che sostituisce il placeholder non sia un attacco SQL injection.

```
const query = "select * from userse where name = ?"
const placeholderValues = [searchName]
executeSQL(query, placeholderValues)
```

Lettura dei valori risultanti da una query

Se vogliamo usare i risultati di una query, dobbiamo implementare la callback di executeSQL. La variabile result contiene le righe risultanti dalla select. Ci consiglia vivamente di gestire gli errori, come minimo di loggarli.

```
tx.executeSql('select * from SALUTI', [],
(tx, result) => {console.log(JSON.stringify(result.rows))},
(tx, error) => {console.log(error)}}
);
```

Struttura del codice

Dobbiamo usare una transaction, che ha come primo parametro la funzione che deve essere seguita in una transazione, dentro questa funzione ci sono una o piu' executeSQL, ciascuna con almeno una funzione di callback. La transaction ha anche due e ulteriori funzioni di callback, per gestire eventuali errori e il caso in cui tutto sia andato bene. E' molto facile fare confusione. Spostiamo questa complessita' in una classe esterna che implementi le chiamate di gestione del DB, facciamo un progetto con i pulsanti che ognuno ci chiama le funzioni della classe che creiamo.

le chiamate:

- Dato un uid e l'id trovare se ce l'ho l'immagine dell'utente

- Scrivere l'immagine, tripla nome utente ipVersion ed immagine

```
export default class StorageManager {

  constructor() {
    this.db = SQLite.openDatabase("myDB")
  }

  getGreetingById(id, onSuccess, onError) {
    const transaction = (tx) => {
      //vedi prossima slide
    }

    const error = (e) => {onError(e)};
    this.db.transaction(transaction, error);
  }
}
```

Noi possiamo fare due metodi di lettura e scrittura, possiamo anche metterci metodi che non usano il DB ma per verificare il sid.

La classe StoreManager crea il db nel costruttore, poi ha dei metodi, ma non posso restituire direttamente l'immagine, quindi l'unica cosa che posso fare è che chi richiama questo metodo passa una funzione che viene eseguita quando il metodo finisce. Possiamo avere un errore a livello di transazione o di query.

```
const transaction = (tx) => {
  let query = "SELECT value from SALUTI where id = ?";
  tx.executeSql(query, [id],
    (tx, queryResult) => { //gestione risultato
      let result = null;
      if (queryResult.rows.length > 0) {
        onSuccess(queryResult.rows._array[0].value)
      } else {
        onSuccess("boh")
      }
    }
  ),
  (tx, error) => { //gestione errore query
    onError(error)
  });
}
```

Nella transazione definiamo la stringa della query, poi eseguiamo la query sostituendo il placeholder, poi diciamo che quando arriva il risultato dobbiamo chiamare il metodo onSuccess, quello che facciamo è prendere il risultato e passandolo al metodo chiamante.

Il componente creerà un'istanza dello stored manager poi chiama il metodo passandogli i parametri e le funzioni per gestire il risultato e l'errore.

Nell'esercizio facciamo un progetto con un pulsante per chiamare l'init, un pulsante con due campi di testo per userid e pversion che li manda alla libreria e ritorna il risultato se c'è. Un pulsante con tre campi di testo per mandare i dati al DB. Inoltre facciamo dei metodi per capire se siamo al primo avvio, e

```
let sm = new StorageManager();
sm.getGreetingById(3,
  result => console.log("risultato", result),
  error => console.log("error", error)
);
```

nel caso in cui lo siamo salviamo delle informazioni per capire poi che non lo sapremo più.

9.12 Posizione e mappe

Expo semplifica l'uso delle librerie di calcolo della posizione e di uso delle mappe. Come vedremo, la configurazione è minima, anche meno di quanto è necessario per Android.

9.12.1 Acquisizione della posizione

Dobbiamo utilizzare una [libreria](#) esterna, che va installata e poi importata. Prima di poter acquisire la posizione dell'utente è necessario ottenere i permessi da parte dell'utente. Questo richiede di interagire con il OS, si chiama una funzione di OS che richiede all'utente di poter usare la posizione.

Permessi

La libreria Location include anche alcuni metodi per acquisire i permessi per l'uso della posizione. In android nativo è un pasticcio, perchè sono tutte chiamate asincrone (l'OS deve chiedere all'utente) in JS diventa semplice perchè possiamo usare le chiamate asincrone con `async-await`.

Creiamo una funzione `async` e dentro ci mettiamo le `await`. Vogliamo controllare se ci sono i permessi per l'uso della posizione, e se ci sono usiamo la posizione. Partiamo assumendo che la posizione non possiamo usarla, poi usiamo un metodo che ci dice se l'utente ha già in precedenza dato il permesso per utilizzare la posizione. Se l'attributo `status` è settato a `granted` possiamo usare la posizione. Se invece non me l'ha data devo chiedergli il permesso. Alla fine dobbiamo anche gestire il caso in cui l'utente non ci da l'autorizzazione.

```

async locationPermissionAsync() {
  let canUseLocation = false;
  const grantedPermission = await Location.getForegroundPermissionsAsync()
  if (grantedPermission.status === "granted") {
    canUseLocation = true;
  } else {
    const permissionResponse = await Location.requestForegroundPermissionsAsync()
    if (permissionResponse.status === "granted") {
      canUseLocation = true;
    }
  }

  if (canUseLocation) {
    //use the location now
  }
}

```

9.12.2 Acquisire la posizione

Quando abbiamo verificato di avere i permessi, possiamo effettivamente acquisire la posizione. Abbiamo 3 modi:

- Prendere l'ultima posizione nota, non facciamo neanche partire l'antenna GNSS (o il servizio di calcolo della posizione) ma usiamo l'ultima posizione che è stata acquisita, vuol dire che se un'altra applicazione ha usato il calcolo della posizione abbiamo la posizione aggiornata, altrimenti no. Questo si fa per risparmiare batteria.
- Calcolare la posizione una volta sola, come nel progetto (twok)
- Calcolare la posizione periodicamente, come un navigatore. In questo caso si definiscono una serie di parametri e si danno ad un servizio di OS che periodicamente richiama un metodo di callback quando la posizione è cambiata.

Da programmatori non abbiamo il controllo di quello che viene attivato (quale hardware utilizzato per calcolare la posizione), possiamo richiedere un certo livello di precisione, allora il OS decide sulla base della nostra richiesta cosa fare per calcolare la posizione. Perché bisogna bilanciare il consumo energetico e la precisione. In tutti e tre i casi otteniamo un riferimento ad un [LocationObject](#).

9.12.3 Mappe

Per mostrare le mappe esiste una [libreria](#) inclusa in Expo. Anche in questo caso è necessario installare la libreria e ricordarsi di importare le classi (importare anche il Marker). In Android è più complicato perché con le API di Google alcuni servizi sono a pagamento, servizi che se includiamo nell'applicazione vanno pagati, ci riconoscono perché ci danno una chiave che viene passata a google

ogni volta che l'app fa una richiesta e noi paghiamo. Su Expo tutto questo non serve, perchè nel momento in cui creiamo un App di prova ha lui una chiave che usa, però quando vogliamo pubblicare l'app dobbiamo fare una serie di passaggi in più.

Per mostrare la mappa e' sufficiente includere l'oggetto MapView all'interno della gerarchia di componenti:

```
render = function() {  
  return <View style={styles.container}>  
    <MapView style={styles.map}/>  
  </View>  
};
```

Per specificare la dimensione della mappa inseriamo la mappa in una view, rendiamola mappa grande come la view stessa:

```
const styles = StyleSheet.create({  
  map: {  
    position: 'absolute',  
    top: 0,  
    right: 0,  
    bottom: 0,  
    left: 0  
  },  
});
```

```
const styles = StyleSheet.create({  
  map: {  
    ...StyleSheet.absoluteFillObject  
  },  
});
```

Ovviamente con la mappa vogliamo interagire, gestendo gli eventi e disegnando sulla mappa.

Disegnare diventa facile perchè possiamo aggiungere un componente che rappresenta quello che ci interessa, come ad esempio un marker. Se abbiamo più oggetti da disegnare possiamo usare la funzione map (ocio che è una cosa comune da fare durante l'esame, tipo scarica 50 twok e mostrali sulla mappa, esiste un esempio in documentazione).

```
render = function() {
  return <View style={styles.container}>
    <MapView
      style={styles.map}
      onRegionChange={this.handleRegionChanged}
      initialRegion={{
        latitude: 49.5,
        longitude: 0,
        latitudeDelta: 1,
        longitudeDelta: 1,
      }}>
      <Marker
        coordinate={{latitude: 49.5, longitude: 0}}
        title="prova"
        description = "descrizione"
      />
    </MapView>
  </View>
};
```

9.13 Debug e test

Il logger è uno dei due strumenti principali per effettuare il debugging del codice. Di fatto il logging è una forma di documentazione, è anche molto importante nella programmazione ad eventi perchè grazie ai log si riesce a capire con quale ordine avvengono gli eventi. Esiste la componente LogBox che quando facciamo il release dell'app sopprime tutti i log, il logbox permette anche di sopprimere alcuni warning se sappiamo che non ci interessano.

9.13.1 Uso del debug da chrome

Il debugger ci aiuta a convivere con il codice, in react native si abilita con un menù quando expo è in funzione (m da terminale), ci apre un pannello su browser e selezionando il sorgente possiamo inserire un breakpoint. E' comodo perchè a questo punto possiamo interrompere l'esecuzione del codice. Si possono usare gli strumenti di debug direttamente in Visual Studio Code, bisogna fare qualche set up installando qualche plugin, [video](#) di spiegazione.

Il debugging in JS e' particolarmente comodo perche' ci permette di capire che valore e che tipo hanno le variabili durante l'esecuzione del codice. Una delle pi' comuni fonti di errore e' dovuta a variabili che hanno un valore inatteso (undefined), spesso il problema si manifesta lontano da dove e' stato generato e mettere dei breakpoint nei punti corretti del codice e osservare il valore delle variabili ci aiuta a risolvere in fretta questi errori.

9.13.2 Unit Test

Ci consiglia anche di sviluppare degli unit test. Bisogna trovare delle situazioni in cui lo unit test si sviluppa in poco tempo ma è comodo. In react native c'è un tool [jest](#) che permette di creare dei test con pochissime righe di codice. Ha

senso crearle per le parti che non sono di interfaccia. Noi abbiamo un assert (expect) che è quello che ci aspettiamo.

```
test("myFirstTest", ()=> {
  expect(1+1).toBe(2);
});
```

Per la parte dove vengono fatte chiamate asincrone bisogna cambiare il testing aspettando che la chiamata termina. Si definisce un argomento per la funzione di test e si richiama quella funzione quando il test e' terminato (l'import serve per usare fetch).

```
import "isomorphic-fetch"

test("prova", done => {
  CommunicationController.register()
    .then(result=> {
      done();
    })
    .catch(error => {
      fail(error);
      done();
    });
});
```

9.13.3 Testing interfaccia

Mi salvo l'interfaccia come una stringa e successivamente controllo con snapshot successivi che coincidano. Probabilmente per il nostro prototipo questo non ha senso. [Documentazione](#).

Chapter 10

Android

Quando facciamo scriviamo un programma in Java utilizziamo tutti i tool di Java, ma in realtà stiamo utilizzando una serie di classi che vengono fornite assieme alla libreria Java Development Kit (JDK), librerie che si chiamano Java Standard Edition (SE), in Android abbiamo accesso a queste librerie con qualche eccezione, ad esempio non abbiamo accesso ad alcune classi per la crittografia. Inoltre abbiamo bisogno di un'altra libreria SDK che contiene tutte le API per interagire con il sistema operativo Android, queste SDK includono un numero di versione.

Per programmare utilizzeremo Android Studio che rende disponibile un editor per il codice, strumenti per la gestione delle risorse, strumenti per la creazione WYSIWYG per le interfacce grafiche, poi tutti strumenti per la compilazione ed il deploy su dispositivo fisico (gradle) abbiamo anche strumenti di testing e debugging.

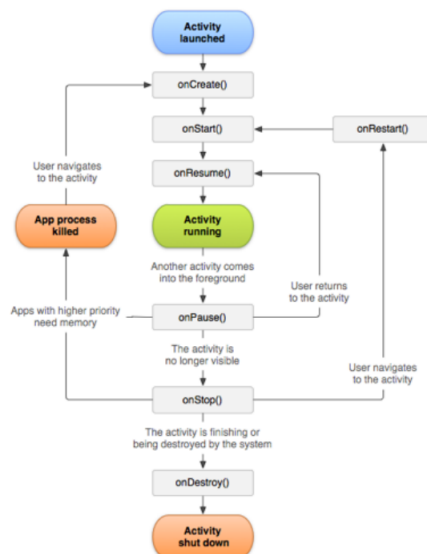
Nelle prossime lezioni impareremo ad utilizzare alcune classi della SDK ed a risolvere alcuni problemi comuni, impareremo ad imparare a fare da soli quello che ci serve guardando la documentazione. Per tutto il resto dovremo imparare ad usare la documentazione:

- [Documentazione](#) ufficiale di Android
- [Documentazione](#) ufficiale di Android (API)

10.1 Activity

E' la prima classe che vediamo. Può rappresentare una schermata, una parte di una schermata o più schermate. Per il momento pensiamo l'activity come la componente di una schermata. In termini MVC e' una classe di tipo View-Controller. Permette di definire l'interfaccia grafica, vedremo il metodo `setContentView(View)`

Permette di gestire gli eventi del ciclo di vita: quando l'applicazione parte chiama 3 metodi(`onCreate`, `onStart`, `onResume`) uno dopo l'altro e poi sta nello stato run. Poi l'activity sta eseguendo ed intanto non può essere terminata. Ci sono delle situazioni in cui vengono richiamati altri metodi quando l'applicazione passa da foreground a background. Prima viene chiamato il metodo `onPause`, significa che un'altra activity è entrata in foreground ma la nostra non è ancora scomparsa (come una notifica), è importante perchè così possiamo gestire cosa succede all'applicazione quando arrivano le notifiche. Quando l'applicazione non è più visibile viene chiamato il metodo `onPause`. Dopo `onStop` ed `onPause` l'applicazione può essere terminata, dobbiamo quindi salvare i dati che avevamo.



Avevamo i tre metodi alla partenza perchè in base a dove ci siamo fermati possiamo ripartire da uno o dall'altro. Questo ci può aiutare a decidere quali azioni fare in quale momento.

In android studio possiamo creare un nuovo progetto con un template che includa un'activity. Crea un progetto che contiene, oltre a vari altri file che vedremo nei seguenti, il file "MainActivity.java". Troviamo il metodo `onCreate` già definito. Possiamo definire gli altri metodi di ciclo di vita in modo analogo. Super significa che non vogliamo cambiare il metodo della classe padre, ma vogliamo usare quello aggiungendo altri comportamenti.

R rappresenta tutte le risorse statiche del sistema, setContentView prende un xml (R) e lo fa diventare degli elementi dinamici.

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}
```

10.2 Context

E' una classe che rappresenta un'interfaccia verso l'ambiente che rappresenta un'applicazione, activity estende context, di conseguenza quando creiamo un'activity abbiamo anche un context (esempio di polimorfismo). Quindi solitamente abbiamo tanti metodi che vogliono come parametro un oggetto di tipo Context, se siamo all'interno di una Activity e vogliamo richiamare questi metodi basta usare this, altrimenti dobbiamo recuperare un riferimento a context.

Il fatto che ci siano piu' context puo' generare confusione. Non focalizziamoci troppo su questa questione, usiamo il contesto dell'activity corrente, quando possibile. Quando non e' possibile (perche' il contesto sopravvive al passaggio tra piu' activity) usiamo il contesto dell'applicazione (metodo getApplicationContext() dall'activity)

10.3 Android Manifest

Quando creiamo un progetto android ci crea un manifest, che rappresenta tutta una serie di informazioni relative all'applicazione stessa, quando creiamo un Activity va dichiarata nel manifest, possiamo dire ad Android studio di creare un Activity e lui ce la mette direttamente nel manifest

10.4 Passaggio tra Activity

Sono state progettate per fare in modo che non ci siano riferimenti comuni agli oggetti, perchè vogliamo essere in grado di eliminare dalla memoria delle activity e non tenerle aperte. Questo rende controintuitivo come passare da un Activity all'altra.

Per passare da una activity all'altra usiamo l'Intent, che e' la descrizione di un'operazione che deve essere svolta. Svolge il ruolo di collante tra le activity. Esistono due tipi di Intent:

- Espliciti: Specificano quale classe deve essere eseguita, mostra quell'activity e permettono di passare da un activity all'altra

- **Impliciti:** descrivono l'azione che dev'essere compiuta (ad esempio aprire una pagina del browser) senza specificare quale activity se ne debba occupare. Il SO poi sceglie l'activity piu' adatta a portare a termine il task tra tutte quelle disponibili (anche di altre app)

Un problema potrebbe essere passare dei dati da un activity all'altra, si prendono i dati si copiano in un intent e si passano per copia.

Notiamo nel secondo esercizio che nel codice creiamo degli oggetti Java a cui passiamo un id di un oggetto statico XML, successivamente usiamo il riferimento all'oggetto java per modificare l'oggetto statico.

10.5 Gestione degli eventi

Non basta quello che abbiamo visto, perchè la definizione statica di un metodo ci permette di gestire solo l'evento di click, ma potremmo aver bisogno di gestire molti eventi diversi. L'altro motivo è che quando la struttura dell'applicazione si complica (fragment) il metodo statico non funziona più. In React Native non ci eravamo neanche posti il problema, vedremo come arrivare ad avere lo stesso comportamento in Java ma facendo un ragionamento molto lungo perchè dovremo trattare le lambda expression, le classi anonime ed il pattern observer.

Immaginiamo di dover implementare un button, dobbiamo fare un pulsante che generi un evento che faccia qualcosa ma non sa ancora cosa. Si usa il pattern observer, abbiamo un componente che ha un evento ed un altro componente che deve ricevere notifiche di quando questo evento avviene, esistono quindi tre componenti:

- Una classe che rappresenta il subject
- Una o più classi che rappresentano l'observer
- Un'interfaccia (o classe astratta) implementata da tutti gli observer che descrive quali metodi il subject può chiamare sugli observer. Le interfacce nascono perchè una classe in Java può ereditare una sola classe, quindi l'interfaccia è un modo per definire che un certo oggetto ha delle caratteristiche.

Notiamo anche che in Java è possibile definire le variabili di tipo classe o interfaccia, ma non posso creare un istanza di interfaccia, posso però creare un istanza di una classe che implementa l'interfaccia (polimorfismo). Noi possiamo anche avere un metodo che ritorna un oggetto di tipo interfaccia che non ci interessa chi la implementa, ma sappiamo che ha i metodi dell'interfaccia.

Tornando ai pulsanti, sono stati implementati in modo da esporre metodi che accettano un interfaccia, l'oggetto che implementa l'interfaccia è quello che ci serve per gestire gli eventi.

Una prima soluzione e' avere una classe di gestione dell'evento che implementa OnClickListener:

```
public class MyClickListener implements OnClickListener {  
  
    private TextView mTextView;  
    public MyClickListener(TextView textView) {  
        mTextView = textView;  
    }  
    @Override  
    public void onClick(View view) {  
        mTextView.setText("pippo!");  
    }  
}
```

Poi creiamo un'istanza di questa classe nell'activity e passo questa istanza al metodo setOnClickListener:

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        TextView myTextView = findViewById(R.id.pippo);  
        MyClickListener myClickListener =  
            new MyClickListener(myTextView);  
  
        Button myButton = findViewById(R.id.button);  
        myButton.setOnClickListener(myClickListener);  
    }  
}
```

Nella gestione dell'evento (onClick), abbiamo bisogno di modificare una proprieta' di una TextView. Non possiamo pero' passare il riferimento alla TextView nel metodo onClick, perche' non possiamo cambiare la signature di questo metodo, che prende in input solo un parametro di tipo view, che rappresenta l'oggetto su cui e' avvenuto il click (un bottone, in questo esempio). Dobbiamo quindi passare il riferimento alla TextView nel costruttore, e associarlo ad una variabile di classe, che possiamo dunque utilizzare nel metodo onClick.

Se dobbiamo gestire gli eventi di più pulsanti nella stessa activity possiamo definire una classe listener diversa per ogni evento oppure definire una sola classe e capire quale evento è stato generato grazie al parametro View passato a onClick():

```
public void onClick(View v) {
    switch(v.getId())
    {
        case R.id.button_a_id:
            // handle button A click;
            break;
        case R.id.button_b_id:
            // handle button B click;
            break;
        default:
            throw new RuntimeException("Unknow button ID");
    }
}
```

Una seconda soluzione è fare in modo che l'activity stessa implementi View.OnClickListener. Noi quindi abbiamo una classe (Activity) che implementa l'interfaccia clickListener, che passa se stessa con il this al bottone che REGISTRA l'activity come observer, quando poi il button avrà qualcosa da segnalare chiamerà il metodo onClick dell'activity (che ricordiamo sa esistere perchè l'activity implementa l'interfaccia).

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener {

    private TextView mTextView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mTextView = (TextView)findViewById(R.id.pippo);

        Button myButton = (Button)findViewById(R.id.button);
        myButton.setOnClickListener(this);
    }

    @Override
    public void onClick(View view) {
        mTextView.setText("pippo!");
    }
}
```

Ci sono due situazioni in cui si può realizzare:

- Quando chi genera l'evento lo manda ad un solo listener, l'observer si registra con il metodo addObserver
- Lo manda a più classi in ascolto, l'observer si registra con il metodo addObserver

Una terza soluzione e' creare una classe anonima all'interno dell'activity per ogni evento da gestire. La classe anonima e' una classe che mi serve una volta sola (per farne un'istanza) e che definisco nella stessa parte del codice dove la uso per crearne l'istanza. Nella soluzione con le classi anonime gli diciamo di creare un oggetto che implementa l'interfaccia ed implementiamo il metodo dell'interfaccia. Occhio che si può mettere tutto compatto.

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        TextView mTextView = findViewById(R.id.pippo);

        Button myButton = findViewById(R.id.button);
        myButton.setOnClickListener(new View.OnClickListener() {
            public void onClick(View v) {
                mTextView.setText("pippo!");
            }
        });
    }
}
```

Classe Anon

Esiste una versione ancora più compatta che si implementa con le lambda expression, si può fare quando abbiamo un'interfaccia con un solo metodo. Stiamo creando un oggetto che implementa una classe di cui non sappiamo il nome che è istanza di `OnClickListener` il cui metodo ha un parametro che si chiama `view` che fa il codice scritto. Regola sintattica, i parametri vanno indicati tra parentesi tonde, eccezione se ce ne sono 1, se ce ne sono 0 vanno messi.

```
myButton.setOnClickListener(v -> {
    Log.d(TAG, "Button Clicked 1");
    mTextView.setText("Pippo");
});
```

10.6 View in Android

E' un argomento concettualmente semplice ma richiede di approfondire un po' di tecnologia da soli.

La classe `View` non va confusa con la `view` dei paradigmi di programmazione come MVC. Una `view` in android è una classe che rappresenta una zona rettangolare dello schermo, tutti gli elementi di interfaccia in Android estendono la classe `view`, gestisce internamente alcuni eventi generati dall'utente (ad esempio l'effetto grafico del pulsante che viene premuto), in generale abbiamo visto che cosa deve fare il pulsante va gestito nell'app (controller).

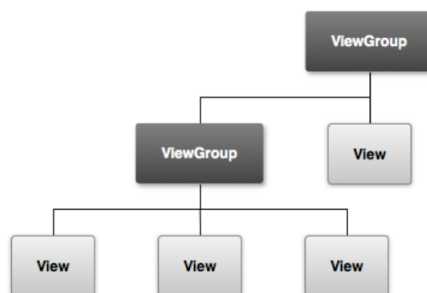
Gli oggetti di view hanno qualche decina di metodi ma due particolarmente rilevanti:

- **OnDraw**: è un metodo che viene richiamato quando l'oggetto viene disegnato, è qui che definiamo effettivamente l'aspetto che vogliamo dare all'oggetto. Quando i programmatori usiamo oggetti di interfaccia definiti da altri non dobbiamo definire questo metodo, ma se dobbiamo definire l'interazione con un oggetto di interfaccia nostro dobbiamo farlo e comunque non dobbiamo richiamarlo, perchè è gestito dallo stack che gestisce l'interfaccia.
- **onTouchEvent**: è un metodo per gestire i tocchi. Per ogni dito che sta toccando sullo schermo ci dice le coordinate e tutta un'altra serie di dati su quel tocco

Esistono tutta un'altra serie di metodi più comodi per gestire i tocchi più specifici (come lo swipe...)

10.6.1 ViewGroup

In Android è definita la classe **ViewGroup** che estende **View** (teniamo distinti il concetto di estendere una classe e comporre vari elementi di interfaccia). Quindi per il principio del polimorfismo è una **View**, serve perchè può contenere altre **View** (o **ViewGroup**, dato che è una **View**), questo ci permette di creare una gerarchia di contenimento di **View**, andando a definire un albero che rappresenta l'interfaccia grafica.



10.6.2 Personalizzazione di View

L'SDK di Android mette a disposizione view predefinite che rappresentano i più comuni elementi di interfaccia. Quando usiamo un oggetto di interfaccia può capitare di volerlo personalizzare. Tipicamente gli oggetti di **View** rendono disponibili a chi li usa dei metodi per personalizzarli, già il **Button** ha dei metodi per cambiare il pulsante (testo, font, background...).

Un altro modo, più complicato e da fare solo in casi particolari, è riscrivendo il metodo `onDraw`, ma è come se andassimo a creare un nuovo oggetto di interfaccia.

Inoltre possiamo comporre più oggetti grafici per crearne uno, ad esempio `Image + TextView`, creiamo un `ViewGroup` che rappresenta i due e dentro ci mettiamo gli oggetti che ci interessano.

10.6.3 Layout

Sono delle classi che estendono `ViewGroup` che definiscono come devono essere mostrati i `viewGroup` all'interno. Quello che troviamo di default è il `constraint layout`. Potremmo usare il `linearLayout` che dispone gli oggetti uno dietro l'altro in orizzontale o in verticale (righe o colonne). `ConstraintLayout` è una classe molto espressiva perchè permette di organizzare gli elementi in posizione relativa (rispetto al padre, rispetto alla view...), nasce l'esigenza di fare il `ConstraintLayout` perchè prima esistevano dei layout semplici, il principio è che quando avevamo i layout così semplici ottenevamo un'interfaccia complessa componendo tanti layout diversi, questo può diventare scomodo quando abbiamo delle pagine complicate ed abbiamo una gerarchia con tanti livelli, il `ConstraintLayout` essendo molto espressivo ci permette di appiattire la gerarchia ed averla anche di un solo livello. (lui consiglia di guardarci un tutorial per il `constraint layout` ed usare quello).

Linear Layout



Constraint Layout



10.6.4 Stili e temi

L'idea è la stessa del CSS, quello che faccio è definire una singola volta uno stile e poi lo applico a tutte le View che voglio.

```
<TextView
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:textColor="#00FF00"
    android:typeface="monospace"
    android:text="@string/hello" />
```



```
<TextView
    style="@style/CodeFont"
    android:text="@string/hello" />
```

C'è un altro aspetto interessante relativo ai layout, quando definiamo un layout abbiamo la possibilità di dire in quali condizioni deve essere utilizzato quel layout, è utile ad esempio se vogliamo cambiare l'interfaccia tra smartphone e tablet.

10.6.5 Creazione delle View

Quello che abbiamo visto fino ad ora è definire una view staticamente, significa che creiamo del codice XML e quando l'applicazione esegue carica la definizione statica dell'interfaccia in oggetti dinamici e vengono usati questi. Abbiamo due modi:

- WYSISYG
- Editing a mano dell'XML, solitamente va fatto perchè con l'interfaccia grafica non si riesce a fare tutto

Ovviamente possiamo anche da codice definire la stessa cosa, possiamo scrivere l'interfaccia direttamente da codice, ma ovviamente è più comodo definire staticamente, ci sono però dei casi in cui non è possibile, capita che ci siano delle interfacce che vengono costruite dinamicamente, come nell'esempio:



10.6.6 Creazione statica della navigazione

E' possibile creare in maniera statica la navigazione tra le varie schermate, questo si realizza tramite una libreria navigation (la documentazione è parecchio lunga, quello che è comodo è leggere la parte di overview e poi fare un progetto di prova).

10.6.7 Risorse e codice

Quando lavoriamo nel codice (pulsante disabilitati) chiamiamo il metodo sull'oggetto dinamico, ho quindi bisogno un modo per prendere un riferimento all'oggetto dinamico, abbiamo quindi una serie di metodi per riprendere l'oggetto dinamico che mi rappresenta un interfaccia statica (fino ad ora abbiamo usato il findViewById, che ritorna l'oggetto dinamico che è stato utilizzato per rappresentare l'oggetto statico. Questo succede perchè Android Studio genera in automatico una classe, chiamata R, che al suo interno contiene varie costanti:

- con accesso pubblico
- il nome della costante è uguale al nome usato per la risorsa
- il valore della costante è quello dell'ID della risorsa

In questo modo quando da codice vogliamo riferirci ad una risorsa usiamo normalmente una variabile, che è meno soggetto ad errori. L'idea generale è questa, poi per vari tipi di risorse ci sono accorgimenti specifici.

Ad esempio per ottenere un riferimento ad un layout si usa il metodo della classe activity setContentView che accetta come parametro l'ID della risorsa che rappresenta il layout da mostrare:

```
setContentView(R.layout.activity_main)
```

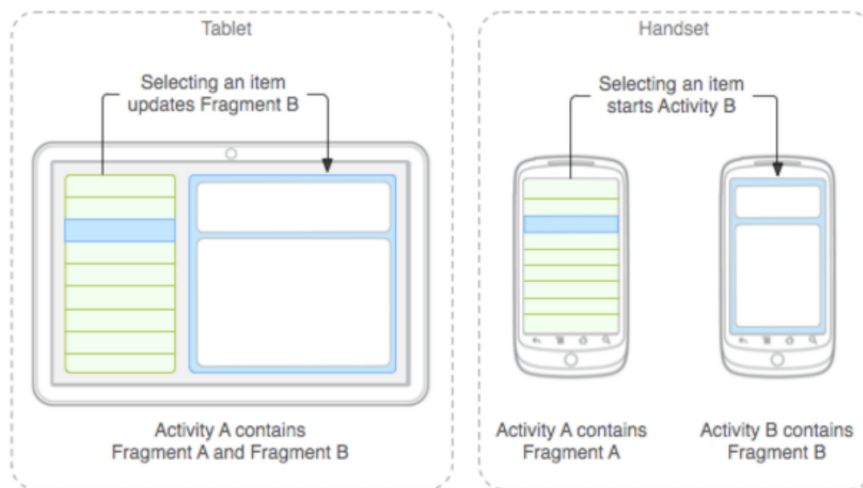
10.6.8 View Binding

Recentemente è stata introdotta una modalità più rapida chiamata [ViewBinding](#), l'idea è dopo aver abilitato la funzionalità, attraverso una variabile si accede direttamente agli elementi di interfaccia. Il vantaggio è che non c'è rischio di ottenere un riferimento nullo, consiglio: se creiamo un progetto con template "basic activity" viene automaticamente impostato un ViewBinding.

E' buona norma fare un file statico per le scritte da mettere dentro l'interfaccia, tipo scritta con chiave e nel codice indichiamo la chiave, così è più comodo riutilizzare il codice.

10.7 Fragment

Sono una classe importante della programmazione Android perchè rappresentano delle componenti di ViewController, storicamente i fragment non esistevano e si faceva tutto con l'activity, sono stati introdotti per dare una maggiore modularità alle interfacce grafiche, in particolare li hanno introdotti quando sono nati i primi tablet android.



In realtà ora si sono evoluti e si usano per mostrare anche le interfacce, dobbiamo avere una sola activity per l'applicazione e tanti fragment che mostrano le varie schermate. Questo viene molto naturale usando navigation, perchè nasce con i fragment.

10.7.1 Ciclo di vita

Analogamente alla Activity anche i Fragment hanno un ciclo di vita. Il modo di gestire gli eventi è analogo così come il nome di alcune eventi. I metodi tipicamente più usati sono:

- `onCreate` e `onPause`: analoghi ai metodi di Activity
- `onCreateView`: deve implementare i metodi per costruire l'interfaccia grafica, ritorna la View che il contiene

Facciamo attenzione quando li usiamo perchè si sono evoluti molto nel tempo, quindi se guardiamo documentazione vecchia rischiamo di fare dei pasticci (no

i 2 anni). Comunque in [documentazione](#) troviamo tutti i dettagli di tutti gli stati.

10.7.2 Creazione del Fragment

Estendo la classe `Fragment` e quando lo creo passo al costruttore del padre il `Layout` che voglio. Questo è creato ma non ancora mostrato

```
public class Fragment1 extends Fragment {
    public Fragment1() {
        super(R.layout.fragment1);
    }
}
```

10.7.3 Mostrare un Fragment

Definendo nell'XML che deve essere mostrato oppure da codice.

Da XML si crea un oggetto `fragmentContentView` (a sua volta collocato nel `layout` di un'Activity) che contiene un `fragment`.

```
<androidx.fragment.app.FragmentContainerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:name="com.example.fragmentexample.Fragment1" />
```

I `fragment` vengono solitamente usati per essere aggiunti e rimossi in modo programmatico. Supponiamo di aver creato il `FragmentContainerView` ma di non averci ancora inserito nessun `fragment`. Da qualche parte nell'Activity (es. nel metodo `onStart`) possiamo programmaticamente inserire un `fragment`. Con il metodo dell'activity `getSupportFragmentManager()` con `.add` gli passiamo il `fragment`, occhio che non definiamo qui cosa contiene il `fragment`, ma gli diciamo solo dove deve essere mostrato il `fragment`.

```
getSupportFragmentManager().beginTransaction()
    .setReorderingAllowed(true)
    .add(R.id.fragment_container_view, Fragment1.class, null)
    .commit();
```

Supponiamo ora di voler sostituire un `fragment` in un `container`, il codice è molto simile a prima:

```
getSupportFragmentManager().beginTransaction()
    .setReorderingAllowed(true)
    .replace(R.id.fragment_container_view, Fragment2.class, null)
    .commit();
```

10.7.4 Passaggio parametri

In modo simile a quanto succede con l'Activity, creiamo un oggetto bundle e con il metodo put creiamo una copia chiave-valore che poi passiamo come args, per il momento non gli passiamo un oggetto per non creare oggetti condivisi tra più fragment, ma vedremo come si fa.

```
Bundle args = new Bundle();
args.putInt("someInt", 5);

getSupportFragmentManager().beginTransaction()
    .setReorderingAllowed(true)
    .replace(R.id.fragment_container_view, Fragment2.class, args)
    .commit();
```

Nel metodo onCreate del fragment richiamiamo il metodo getArguments per riprendere il bundle e leggiamo i valori inseriti nel bundle (nota: è cattiva norma di programmazione inserire stringhe nel codice ad esempio "someInt", sarebbe meglio definire delle costanti ed usare quelle).

```
@Override
public void onCreate(@Nullable Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Bundle b = getArguments();
    if (b != null) {
        int i = b.getInt("someInt");
        Log.d(TAG, "Argument is: " + i);
    }
}
```

Sergio Mascetti - Mobile Computing

Es1: creiamo un nuovo fragment ce lo fa in automatico Android Studio, poi in activity sull'interfaccia grafica posso metterci un fragment container view mettendoci il fragment che mi interessa

Nel terzo esercizio ci sono delle schermate in cui schiacciando su un elemento della lista passiamo ad un'altra schermata, facciamo solo con dei pulsanti.

10.8 Model-View-Presenter

Riprendiamo l'esercizio dei pulsanti che modificano il numero, la soluzione che avevamo fatto è sbagliata, perchè stavamo utilizzando come memoria della nostra applicazione lo stato della view, potremmo invece che leggere il valore dal campo di testo metterlo in una variabile di classe. Questo però rimane molto brutto, perchè abbiamo un'unica componente che funziona da view, model e controller, in Android nativo non è solo un problema di pulizia del codice ma proprio non funziona. Se l'utente ad esempio ruota il device lui distrugge l'activity e poi la ricrea, creando una nuova istanza della classe e quindi il total viene riportato a 0. In android quindi dato che l'activity può essere distrutta e ricreata non deve in nessun modo essere legata ai dati.

Vedremo tre modi per risolvere questo problema:

- Instance state: salvare i dati su memoria persistente
- Pattern singleton model
- View-Model: modo nuovo che useremo

10.8.1 Memoria persistente

Il metodo `onSaveInstanceState` di un'activity viene richiamato quando è necessario salvarne lo stato. Un parametro di questo metodo è un oggetto di tipo `Bundle` (coppie chiave-valore). Quindi quello che facciamo è scrivere il codice per salvare il valore che mi interessa e nel metodo `onCreate` posso ripristinare i valori.

Ricordiamoci di chiamare il metodo `super.onSaveInstanceState` come ultima chiamata del metodo. Notiamo che nell'esempio creiamo una costante a livello di classe per la stringa che uso come chiave.

```
@Override
protected void onSaveInstanceState(@NonNull Bundle outState) {
    outState.putInt(BUNDLE_KEY_THE_NUMBER, theNumber);
    super.onSaveInstanceState(outState);
}
```

Per rileggere i dati prima devo verificare se esiste il bundle.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    if (savedInstanceState != null) {
        theNumber = savedInstanceState.getInt(BUNDLE_KEY_THE_NUMBER);
    }
    numberTextView = findViewById(R.id.numberTextView);
    refreshUi();
    //altro codice omissso...
}
```

Il metodo `onCreate` rischia di diventare spesso molto complicato, c'è un metodo apposito che viene richiamato solo quando il `Bundle` è non-nullo:

```
@Override
protected void onRestoreInstanceState(@NonNull Bundle savedInstanceState) {
    super.onRestoreInstanceState(savedInstanceState);
    theNumber = savedInstanceState.getInt(BUNDLE_KEY_THE_NUMBER);
    refreshUi();
}
```

In realtà però è che ci risolve solo dei casi molto particolari, perchè nel bundle ci mettiamo solo pochi dati non strutturati.

10.8.2 Singleton Model

L'idea del pattern singleton è con pochissime righe di codice scrivere una classe di cui gli altri non possono fare un'istanza, solo la classe stessa può, il tutto è organizzato in modo che quella classe possa fare un'unica istanza e passi quell'istanza agli altri per riferimento.

```
public class MyModel {
    private static MyModel instance = null;

    //Il costruttore private impedisce l'istanza di oggetti da parte di classi esterne
    private MyModel() {}

    // Metodo della classe impiegato per accedere al singleton
    public static synchronized MyModel getInstance() {
        if (instance == null) {
            instance = new MyModel();
        }
        return instance;
    }
}

//quando mi serve il model uso
MyModel m = MyModel.getInstance();
```

Quello che faccio nella classi di model Singleton è mettere privato il costruttore in modo che nessuno possa creare istanze di quell'oggetto, poi creo un metodo statico che ritorna l'istanza dell'oggetto (se non c'è la crea prima).

Nel nostro esempio:

```
public class MyModel {
    private static MyModel theInstance;

    private MyModel() { };

    public static synchronized MyModel getInstance() {
        if (theInstance == null) {
            theInstance = new MyModel();
        }
        return theInstance;
    }

    private int theNumber = 0;
    public int getTheNumber() {return theNumber;}
    public void setTheNumber(int theNumber) {this.theNumber=theNumber;}
}
```

Siccome abbiamo creato il model come Singleton, quando l'activity parte ne prende l'istanza (se c'è già, altrimenti il Singleton ne crea una nuova). Poi l'activity legge il valore del model e lo mostra nella view. A questo punto non

importa se l'activity viene distrutta e ricreata: il model permane (finanto che l'app rimane in memoria) e memorizza i dati.

La differenza è che quando faccio il model così mi permane in altre activity, è brutto perchè nel singleton tutti possono accedere (dal metodo statico) e farci casino.

10.8.3 View-Model

Possiamo interpretarlo come accezione di MVC o come viene suggerito adesso in documentazione ufficiale. Il viewModel è in un certo senso simile al Singleton, possiamo definire una classe view model che è una classe particolare che si chiama life cycle aware, creiamo una classe che sopravviva ai cambi di stato (quando l'activity viene distrutta e poi ricreata), l'altra cosa carina con il view model è di utilizzare al suo interno una classe live data, live data è una classe che rappresenta un dato (che può essere quello che vogliamo) sulla quale possiamo registrare un observer, possiamo definire quando quel dato cambia fai qualcosa, un oggetto di tipo MutableLiveData ha questi metodi principali:

- getValue: ritorna il valore contenuto
- observe: aggiunge un observer
- setValue: modifica il valore contenuto e notifica gli observer
- postValue: come setValue, ma si può richiamare da un thread secondario (ne parliamo più avanti)

Il ViewModel:

```
public class MyViewModel extends ViewModel {
    private MutableLiveData<Integer> grandTotalLiveData = null;

    public MutableLiveData<Integer> getGrandTotalLiveData() {
        if (grandTotalLiveData == null) {
            grandTotalLiveData = new MutableLiveData<Integer>();
            grandTotalLiveData.setValue(0);
        }
        return grandTotalLiveData;
    }
}
```

Quando abbiamo un oggetto viewModel non dobbiamo costruirlo con il new, ma c'è un modo con cui posso creare un istanza del view model, in pratica usiamo un new ViewModelProvider, che prende in input la classe e fa da solo quello che faceva prima il singleton.

Con il `mutableLiveData` possiamo dire di ridisegnare l'interfaccia quando cambia quel valore (nel nostro esempio setta il testo quando la variabile nel model cambia).

Poi definiamo l'evento sul tap del pulsante.

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    numberTextView = findViewById(R.id.numberTextView);

    //Ottengo un riferimento al ViewModel
    MyViewModel model = new ViewModelProvider(this).get(MyViewModel.class);

    //Definisco l'observer: quando cambia il valore del LiveData, viene eseguito questo codice
    model.getGrandTotalLiveData().observe(this, theValue -> {
        numberTextView.setText(String.valueOf(theValue));
    });

    //Definisco l'evento di tap su un pulsante
    findViewById(R.id.plus1).setOnClickListener(v -> {
        model.getGrandTotalLiveData().setValue(model.getGrandTotalLiveData().getValue() + 1);
    });

    //... e così via per gli altri pulsanti
}
```

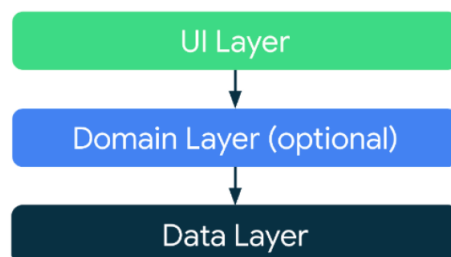
Questa cosa è comoda perchè possiamo scrivere una sola volta il codice che aggiorna l'interfaccia grafica, il pulsante modifica il valore e poi con l'observer modifica l'interfaccia.

10.8.4 Struttura dell'applicazione

Usando i `ViewModel` possiamo creare un pattern view model controller. Però Android tende a non consigliare l'uso di un paradigma MVC ne MVP, ma hanno una loro proposta di progettazione architetturale che ricorda MVVM.

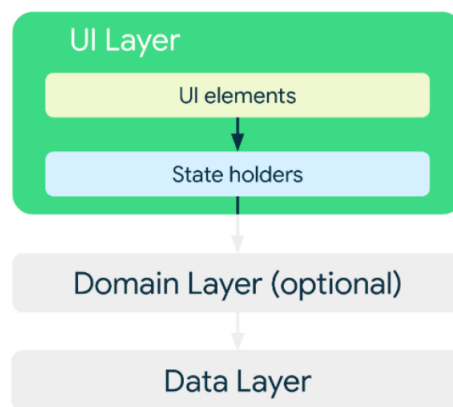
Quello che suggeriscono di fare è avere 3 layer, di cui uno opzionale:

- UI layer: interfaccia
- Domain Layer (optional): se abbiamo delle funzioni in comune di più UI layer
- Data layer: dati



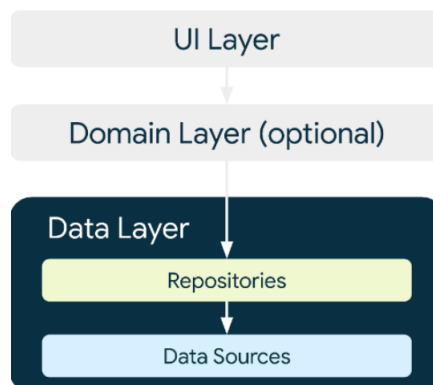
Lo UI layer ha due sotto livelli:

- Gli elementi di UI (fragment e view) e la parte di gestione degli eventi. Il View-Controller in MVVM
- "state holder": mantiene lo stato, è implementato dal ViewModel, assomiglia molto al viewModel di MVVM, prepara i dati in modo che quando sono nella view non dobbiamo fare tanta elaborazione dei dati, ma richi- amiamo quei metodi



Il data layer:

- Repositories: le strutture dati del dominio applicativo
- Data source: dove memorizziamo i dati in maniera persistente (DB, rete...)



10.8.5 Organizzazione del model

Tipicamente le componenti del model sono organizzate in una o più classi. Una classe di ViewModel che si interfaccia con il view e il controller. Il ViewModel a sua volta gestisce uno o più riferimenti a tante classi che rappresentano i dati del problema in oggetto.

Le classi specifiche per la rappresentazione dei dati del problema in oggetto le deve creare il programmatore, non reinventiamo l'acqua calda ma andiamo a vedere le strutture dati per organizzare il model nel [collection framework](#).

10.9 Liste ed Adapter

L'adapter è il modo in android nativo per mostrare le liste. Mentre in React Native per fare una lista basta pochissimo, in Java per Android bisogna scrivere un po' di componenti.

Quando abbiamo visto come creare le interfacce grafiche abbiamo visto che quando ne abbiamo la possibilità definiamo degli elementi statici e questo possiamo farlo quando sappiamo a priori quali elementi avremo, non sempre però sappiamo gli elementi dell'interfaccia che abbiamo. Ci sono dei casi molto comuni in cui vogliamo generare una lista di elementi e non sappiamo a priori quali e quanti saranno, le liste sono molto comuni per le applicazioni mobili. In HTML lo faremmo con un costrutto di iterazione e per ogni dato nel model creiamo un oggetto di View, va bene ma c'è un problema, se abbiamo tantissimi elementi di model creiamo tantissimi elementi di view, ed occupiamo un sacco di memoria, quello che si fa in Android è usare un trucco, utilizzare delle recycle View.

Gli strumenti che utilizzano queste recycle view sono appunto gli adapter, che sono elementi del controller che permettono di definire view dinamiche rispetto ai dati da mostrare. Collegano la view con il model, definiscono quali elementi devono essere mostrati, possono definire come devono essere mostrati gli elementi.

10.9.1 Recycle View

Supponiamo di avere una rubrica di 10000 contatti, nella vista dell'applicazione ne mostriamo 10, possiamo evitare di creare 10000 elementi di view ma con un trucco prendiamo gli elementi che scompaiono dalla vista li spostiamo e gli cambiamo il contenuto in modo da avere un numero limitato di oggetti di interfaccia. Questa cosa non dobbiamo implementarla noi, ma c'è una classe in Android che lo fa al posto nostro.

La RecyclerView gestisce un sacco di cose (anche le tabelle), quindi facciamo attenzione in documentazione, se troviamo robe più vecchie di 2 anni fidiamoci poco.

Componenti

Per implementare questa roba qua abbiamo bisogno di un po' di elementi:

- Oggetto RecyclerView: mi rappresenta l'intera lista è di tipo view, non dobbiamo fare altro che metterlo nell'interfaccia
- Elemento di view-controller che rappresenta un elemento della cella:
 - Per la parte di view si crea un generico layout che rappresenta un singolo elemento della lista
 - Una classe T che estende ViewHolder che gestisce il comportamento della singola cella, RecyclerView.ViewHolder
- Adapter: una classe che estende RecyclerView.Adapter<T> dove T è la classe definita per il ViewHolder

Per approfondire guardare la [documentazione](#).

Tutto questo ci serve per mettere in comunicazione i dati di un model nella view. Non c'è nessuna classe predisposta per il model, ma la facciamo noi, in questa classe dobbiamo avere un metodo che restituisce l'elemento in base alla posizione ed il numero di elementi.

Una difficoltà nelle liste è che abbiamo tanti passaggi da fare prima di verificare che tutto funziona, quindi l'approccio che suggerisce è fare una lista semplicissima e poi complicarla.

10.9.2 Esempio

Costruiamo un app che contenga una lista in vari passaggi. Mostriamo una lista di nomi ed andiamo a gestire l'evento di tocco su un elemento della lista. Cominciamo a fare una classe di Model (in realtà avremmo anche bisogno di una classe che rappresenta un singolo contatto). Questo esempio è fatto con il singleto, sarebbe buona norma realizzarlo con il ViewModel.

```
public class Model {  
  
    private static Model theInstance = null;  
  
    private ArrayList<String> contacts = null;  
    public static synchronized Model getInstance() {  
        if (theInstance == null) {  
            theInstance = new Model();  
        }  
        return theInstance;  
    }  
  
    private Model() {  
        contacts = new ArrayList<String>();  
    }  
}
```

```
public String get(int index) {  
    return contacts.get(index);  
}  
  
public int getSize() {  
    return contacts.size();  
}  
  
//creiamo dei dati di test  
public void initWithFakeData() {  
    for (int i=0; i<100;i++) {  
        contacts.add("Entry "+i);  
    }  
}
```

Successivamente creiamo un nuovo layout e settiamo da XML l'altezza del layout come `wrap_content`, altrimenti un elemento della cella sarà grande come tutto lo schermo. (se vogliamo avere una cella più complessa facciamo qui il layout come vogliamo).

```
<?xml version="1.0" encoding="utf-8" ?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="TextView"
        android:textSize="60sp" />

</LinearLayout>
```

Poi creiamo una classe `ViewHolder` che estende `RecyclerView.ViewHolder` (è una classe astratta, significa che ha alcuni metodi già implementati ed altri che dobbiamo fare noi) che mi serve per aggiornare il contenuto della singola cella. Dobbiamo creare il costruttore e per comodità creiamo un metodo `update` che serve per aggiornare il contenuto della cella. L'unico problema è che non abbiamo in questa classe il riferimento alla view, quindi lo prendiamo nel costruttore.

```
public class ViewHolder extends RecyclerView.ViewHolder {
    private TextView myTextView;

    public ViewHolder(View itemView) {
        super(itemView);
        myTextView = itemView.findViewById(R.id.textView);
    }

    public void updateContent(String text) {
        myTextView.setText(text);
    }
}
```

Ora andiamo a fare l'Adapter, è una classe che estende `RecyclerView.Adapter<ViewHolder>`, anche questa è una classe astratta e dobbiamo implementarne i metodi:

- `getItemCount()`: per disegnare la lista deve sapere quanti elementi ha, dobbiamo quindi ritornare questa informazione
- `onCreateViewHolder`, quando viene creata la celletta
- `onBindViewHolder()`: quando alla cella è associato un oggetto (testo nel nostro caso). Notiamo molto chiaramente che l'adapter mette in comunicazione il model con la view, ho bisogno di un elemento della lista, lo va a prendere dal model e lo passa alla view

Abbiamo anche bisogno del costruttore per farci passare il model, poi come sempre prendo la variabile che mi è stata passata e la assegno ad una variabile che ha visibilità di classe.

Dobbiamo usare un oggetto Inflater a cui passiamo una definizione statica di un oggetto di interfaccia e lui torna un oggetto dinamico, il problema di Inflate è che per crearlo mi serve un riferimento a Context, quindi ancora una volta dal costruttore ci facciamo passare il Context.

```
public class MyAdapter extends RecyclerView.Adapter<ViewHolder> {

    private LayoutInflater mInflater;

    public MyAdapter(Context context) {
        this.mInflater = LayoutInflater.from(context);
    }

    @Override
    public ViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
        View view = mInflater.inflate(R.layout.single_row, parent, false);
        return new ViewHolder(view);
    }

    @Override
    public int getItemCount() {
        return Model.getInstance().getSize();
    }

    @Override
    public void onBindViewHolder(ViewHolder holder, int position) {
        String text = Model.getInstance().get(position);
        holder.updateContent(text);
    }

}
```

Ora abbiamo bisogno di creare dentro il layout dell'activity la RecyclerView e dirgli come usarla. Innanzitutto inseriamo una RecyclerView all'interno del layout:

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Nel MainActivity prendiamo il riferimento alla recyclerView creata in XML, definiamo il layoutManager (tipicamente è un LinearLayoutManager, ma potrebbe

essere diverse per esempio se vogliamo dare delle griglie al posto delle liste). Creiamo l'adapter e lo associamo alla recyclerView. (in un fragment esistono dei metodi per prendere i riferimenti di view e context della main view).

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Model.getInstance().initWithFakeData();

        RecyclerView recyclerView = findViewById(R.id.recyclerView);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        MyAdapter adapter = new MyAdapter(this);
        recyclerView.setAdapter(adapter);
    }
}
```

10.9.3 La gestione degli eventi

Per come abbiamo strutturato il codice, la gestione degli eventi è molto semplice: abbiamo una classe (ViewHolder) che rappresenta il view-controller per una singola cella. Andiamo a definire lì il comportamento.

Facciamo implementare a ViewHolder l'interfaccia View.OnClickListener (o con le lambda).

```
public class ViewHolder extends RecyclerView.ViewHolder implements View.OnClickListener {
    private TextView myTextView;

    public ViewHolder(View itemView) {
        super(itemView);
        myTextView = itemView.findViewById(R.id.textView);
        itemView.setOnClickListener(this);
    }

    public void setText(String text) {
        myTextView.setText(text);
    }

    @Override
    public void onClick(View view) {
        Log.d("ViewHolder", "OnClick on Element: " + myTextView.getText().toString() + " with position: " + getAdapterPosition());
    }
}
```

A volte l'evento può essere gestito localmente dal ViewHolder, altre volte quando abbiamo una lista ci interessa sapere su che elemento ha cliccato, con il metodo `getAdapterPosition` ci dice in che posizione avviene il tocco.

Può capitare che il comportamento sul tocco del pulsante non possa essere gestito dalla cella, ma debba risalire a livello di fragment o activity, l'idea è di passare un riferimento dall'activity al viewHolder che lo può richiamare. E' però l'adapter che crea il ViewHolder, quindi passo all'adapter un riferimento all'activity che a sua volta lo passa al ViewHolder che può richiamare il metodo dell'activity.

Creiamo il metodo di gestione:

```
public void onClick(View view, int position){
    Log.d(" MainActivity", "Tap evento on item:" + position);
}
```

Passiamo il riferimento all'Activity modificando i costruttori dell'adapter e del viewHolder per fare in modo che possa essere passato un riferimento all'activity:

```
public MyAdapter(MainActivity mainActivity) {
    this.mInflater = LayoutInflater.from(mainActivity);
    this.mainActivity = mainActivity;
}

public ViewHolder(View itemView, MainActivity mainActivity) {
    super(itemView);
    myTextView = itemView.findViewById(R.id.textview);
    itemView.setOnClickListener(this);
    this.mainActivity = mainActivity;
}
```

A questo punto abbiamo fatto in modo che il viewHolder abbia un riferimento alla Activity, basta richiamare un metodo al momento necessario:

```
@Override
public void onClick(View view) {
    Log.d("ViewHolder", "OnClick on Element: " + myTextView.getText().toString() + " with position: "
+ getAdapterPosition());
    mainActivity.onClick(view, getAdapterPosition());
}
```

In realtà questa soluzione è bruttissima, perchè non abbiamo bisogno in ViewHolder di avere tutti i metodi dell'activity, ma solo 1, quindi creiamo un interfaccia e passiamo quella.

Definizione dell'interfaccia:

```
public interface OnRecyclerViewClickListener{
    public void onRecyclerViewClickListener(View v, int position);
}
```

Modifichiamo l'activity:

```
public class MainActivity extends AppCompatActivity implements OnRecyclerViewClickListener {  
  
    @Override  
    public void onRecyclerViewClick(View v, int position) {  
        Log.d("MainActivity", "List clicked at position: " + position);  
    }  
}
```

```
MyAdapter adapter = new MyAdapter(this, this);
```

Modifichiamo l'adapter:

```
private OnRecyclerViewClickListener mRecyclerViewClickListener;  
  
public MyAdapter(Context context, OnRecyclerViewClickListener  
recyclerViewClickListener) {  
    this.mInflater = LayoutInflater.from(context);  
    this.mRecyclerViewClickListener = recyclerViewClickListener;  
}  
  
@Override  
public ViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {  
    View view = mInflater.inflate(R.layout.single_row, parent, false);  
    return new ViewHolder(view, mRecyclerViewClickListener);  
}
```

Modifichiamo il viewHolder:

```
private OnRecyclerViewClickListener mRecyclerViewClickListener;

public ViewHolder(View itemView, OnRecyclerViewClickListener recyclerViewClickListener) {
    super(itemView);
    myTextView = itemView.findViewById(R.id.textview);
    itemView.setOnClickListener(this);
    mRecyclerViewClickListener = recyclerViewClickListener;
}

@Override
public void onClick(View v) {
    mRecyclerViewClickListener.onRecyclerViewClick(v, getAdapterPosition());
}
```

Volendo potremmo anche usare una lambda expression dall'activity per passare una classe che implementa solo l'interfaccia.

Nella pausa dobbiamo finire l'esercizio possiamo cominciare a popolare le varie pagine della navigazione, usando `twok hardcoded` (mettiamo invece delle classi di comunicazione con il server una chiamata che ritorna un `twok hardcoded` che ci siamo inventati noi).

10.10 Comunicazione di rete

Prima di cominciare ricordiamoci di quando avevamo parlato di come si serializza e deserializza un oggetto JSON, avevamo fatto la stessa cosa in JavaScript, ma non ce ne siamo neanche accorti perchè era quasi immediato. In Java ci sono due modi per fare la serializzazione e de-serializzazione:

- Farla a mano: facciamo un metodo dell'oggetto che ci serve per trasformarlo in oggetto JSON, si usa una classe `JSONObject` per costruire il JSON e si inseriscono tutti gli elementi che ci interessano
- Usare delle librerie che da sole prendono un oggetto e fanno la conversione da e verso JSON. Questa cosa non sapremmo farla noi, c'è tutta una branca di un linguaggio di programmazione (reflection) che può processare le caratteristiche di altre classi

Detto questo cominciamo. Prima di vedere le tecniche per la comunicazione di rete guardiamo una cosa sulla progettazione dell'app, è il modo con cui ci avviciniamo alla progettazione per la comunicazione. Abbiamo detto che quando facciamo la comunicazione la facciamo su un thread secondario, quindi concettualmente sarebbe possibile che mentre facciamo una comunicazione di rete

l'utente fa altre cose ad esempio in un applicazione per la mail, in cui mentre un utente guarda una mail può arrivarne un'altra. Molto spesso questa cosa non si può fare perchè porta dei problemi abbastanza seri di sincronizzazione, è vero che non vogliamo che l'interfaccia utente si blocchi ma nella stragrande maggioranza dei casi facciamo aspettare che il thread finisca di fare la comunicazione. In generale dove non serve avere una comunicazione veramente asincrona facciamo aspettare l'utente.

Come si fa in Java ad implementare la comunicazione di rete, il primo modo che vediamo ma non utilizziamo è usando le classi standard di Java per la comunicazione. Esistono delle classi che rappresentano una richiesta HTTP, possiamo aggiungere un input e leggere l'output. E' difficile perchè in questo caso dobbiamo gestire i thread, perchè sappiamo che non si fanno comunicazioni di rete con il main thread.

Le soluzioni che vediamo sono basate su librerie, quella più usata è Volley che tramite una callback permette di specificare una classe che risponda ad una determinata interfaccia che faccia qualcosa quando c'è l'evento di risposta del server. Ultimamente si usa un'altra libreria più nuova che risolve tutta una serie di problemi di Volley.

In tutti questi approcci dobbiamo mettere nel manifest una riga per specificare che dobbiamo usare la comunicazione di rete.

Vediamo un esempio di comunicazione con le classi standard di Java:

```
String url = "http://localhost:8081/myService/pippo";
URL requestURL = new URL(url);
URLConnection conn =
    (URLConnection)requestURL.openConnection();

conn.setRequestMethod("POST");
conn.setDoOutput(true);
conn.setDoInput(true);
DataOutputStream wr = new DataOutputStream(
    conn.getOutputStream());
wr.writeBytes("text to send");
wr.flush();
wr.close();
```

```
InputStream is = conn.getInputStream();
BufferedReader rd = new BufferedReader(
    new InputStreamReader(is));
String line;
response = new StringBuffer();
while((line = rd.readLine())!=null){
    response.append(line);
    response.append( '\r' );
}
rd.close();
```

La cosa veramente scomoda è che `getInputStream()` è una chiamata bloccante e tutto va gestito all'interno di un thread secondario. Va usato solamente se le librerie di comunicazione non ci permettono di fare quello che vogliamo.

10.10.1 Volley

E' una libreria che semplifica la scrittura del codice ed ottimizza le prestazioni. E' comodo perchè permette di fare delle richieste normali senza fare fatica e permette di avere servizi aggiuntivi:

- Una coda di richieste
- Caching di alcuni dati
- Annullare una richiesta

Vediamo qual'è l'idea generale. Gestisce una coda di richieste, nel momento in cui aggiungiamo una richiesta alla coda lui la manda, per ogni richiesta specifichiamo una serie di parametri, ed il codice da eseguire quando la richiesta termina e quando la richiesta va in errore. E' importante questa parte perchè la passiamo con una funzione di callback, avremo un'interfaccia che implementiamo come vogliamo (classe esterna, classe anonima o lambda expression), è comodo perchè è la libreria che gestisce in automatico la chiamata, non abbiamo bisogno di gestire i thread. E' una libreria esterna, dovrebbe importarla automaticamente l'ambiente di sviluppo che modifica il gradle.

Esempio:

Inizialmente creiamo una request queue a cui dobbiamo passare il context (essendo in un activity possiamo passare this), successivamente creiamo la richiesta (in questo caso di tipo StringRequest) e specifichiamo il metodo, l'url e due parametri, due classi anonime che implementano delle interfacce per gestire la risposta e l'errore. Ricordiamo di aggiungere i permessi per la comunicazione (se la facciamo partire una volta con l'errore va disinstallata e reinstallata l'app).

```
String url = "https://ewserver.di.unimi.it/mobicomp/esercizi/sample.txt";

RequestQueue queue = Volley.newRequestQueue(this);
StringRequest request = new StringRequest(
    Request.Method.GET,
    url,
    new Response.Listener<String>() {
        @Override
        public void onResponse(String response) {
            Log.d("TESTTEST", "Received: " + response);
        }
    },
    new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Log.d("TESTTEST", "Ops: " + error.toString());
        }
    }
);
queue.add(request);
```

Questo è il terzo parametro del costruttore di StringRequest

Questo è il quarto parametro

La versione più compatta è scrivere le classi anonime come lambda expression:

```
response -> Log.d("TEST", "Recived: " + response);
error -> Log.d("TEST", "Ops: " + error.toString());
```

La cosa scomoda di questa libreria è gestire i JSON, serve del codice apposito che serializza e deserializza, e questo codice è specifico per ogni chiamata, non può essere generalizzato (facendolo a mano o usando librerie come GSON).

Esempio richiesta JSON:

```
RequestQueue queue = Volley.newRequestQueue(this);
final String url = "https://ewserver.di.unimi.it/mobicomp/exercises/getstudent.php";
final JSONObject jsonBody = new JSONObject();
try {
    jsonBody.put("sid", 111);
} catch (JSONException e) {
    e.printStackTrace();
}

JsonObjectRequest request = new JsonObjectRequest(
    url,
    jsonBody,
    new Response.Listener<JSONObject>() {
        @Override
        public void onResponse(JSONObject response) {
            Log.d("Volley", "Correct: " + response.toString());
        }
    }, new Response.ErrorListener() {
        @Override
        public void onErrorResponse(VolleyError error) {
            Log.d("Volley", "Error: " + error.toString());
        }
    }
);
Log.d("Volley", "Sending request");
queue.add(request);
```

10.10.2 Retrofit

E' una libreria ancora a più alto livello che semplifica alcune cose, con volley avevamo tutta una parte di codice di com'è fatta la richiesta. E' un po' strano, perchè siccome serializza e de-serializza tutto in automatico (per i JSON usa GSON) la cosa più comoda è avere per ogni roba che rappresenta l'input o l'output per ogni richiesta di rete abbiamo una classe, che è costruita con tutti gli attributi che servono per le richieste.

Questa classe deve avere alcune caratteristiche, un costruttore vuoto i parametri attesi dal server con gli stessi nomi e dello stesso tipo e per ciascun parametro un metodo get e set con nome del metodo derivato dal parametro (name getName e setName).

Esempio

L'obiettivo è implementare la register. Creiamo una classe che rappresenta il sid, con getter e setter.

```

public class Sid{
    private String sid;

    public String getSid() {return sid;}
    public void setSid(String sid) {this.sid = sid;}
}

```

Creiamo un interfaccia che ci definisce quali chiamate vogliamo fare al server ed indichiamo tramite annotazioni dei parametri di tali richieste, sarà poi retrofit a implementare automaticamente ciascun metodo. Le chiamate devono tornare un oggetto di tipo "Call" che rappresenta la richiesta e che rappresenta il tipo ritornato dalla chiamata tramite generics. (ricordare che retrofit va importata)

```

public interface ApiInterface {
    @POST("register")
    Call<Profile> getProfile(@Body Sid sid);
}

```

A questo punto dobbiamo istanziare retrofit, che è l'unico codice boilerplate che trasforma l'interfaccia in un vero oggetto in grado di fare le chiamate al server. Il codice si può ripetere dove serve oppure, meglio, includerlo dentro una classe con un metodo statico.

```

String BASE_URL = "https://develop.ewlab.di.unimi.it/mc/twittok/";
Retrofit retrofit = new Retrofit.Builder()
    .baseUrl(BASE_URL)
    .addConverterFactory(GsonConverterFactory.create())
    .build();

ApiInterface registerAPI = retrofit.create(ApiInterface.class);

```

A questo punto non serve altro che effettuare la chiamata e gestire la risposta tramite una callback. Possiamo usare una classe anonima, ad esempio, anche se a volte è più conveniente fare in modo che l'activity (o il fragment) implementi l'interfaccia. Notiamo che stiamo usando un'interfaccia con due metodi, non possiamo quindi usare le lambda expression.

10.11 Testing in Android

Fare del testing per la comunicazione è molto comodo. In ogni progetto vengono create delle classi di testing, è quindi sufficiente scrivere il codice di test di una delle due classi ed eseguire. Una delle classi si chiama "UnitTest" e serve per testare tutte quelle parti di codice che sono indipendenti dal contesto di Android (context) come alcune classi del model.

Le chiamate di rete hanno due caratteristiche peculiari per quanto riguarda il testo: richiedono un riferimento al context (volley), si usa l'altra classe InstrumentedTest che permette di avere un riferimento al context. Sono asincrone,

```

Sid sid = new Sid();
sid.setSid("metti il tuo sid");
Call<Profile> call = registerAPI.getProfile(sid);
call.enqueue(new Callback<Profile>() {
    @Override
    public void onResponse(Call<Profile> call, Response<Profile> response) {
        Log.d(TAG, ""+response.body().getUid());
    }

    @Override
    public void onFailure(Call<Profile> call, Throwable t) {
        t.printStackTrace();
    }
});

```

dunque se le testiamo senza accorgimenti, il test termina prima che la chiamata di callback sia eseguita. Il test sembra funzionare correttamente, anche quando la chiamata effettivamente funziona. Ci sono due soluzioni.

Soluzione 1

Metodo che va bene sia per volley che per retrofit. Usiamo una specie di semaforo, va messo in un try-catch. All'inizio del test:

```
final CountDownLatch signal = new CountDownLatch(1);
```

Alla fine del test (tra try-catch):

```
signal.await();
```

Nella chiamata di callback:

```
signal.countDown();
```

Soluzione 2

Possiamo usarlo solo con retrofit, perchè ha una chiamata bloccante che non possiamo usarla nel codice Andoird, ma nel testing si, la chiamata execute, da usare al posto di enqueue(). Va anch'esso messo nel try-catch:

```

Response<Profile> resp = call.execute();
Profile prof = resp.body();
Log.d("TEST", ""+prof.getUid());

```

10.12 Memorizzazione persistente

Ci sono tre modi per implementare la memorizzazione persistente in Android, li accenniamo tutti e ne vediamo in dettaglio 2.

- Shared preferences
- File
- Databse:
 - (De)serializzazione manuale
 - (De)serializzazione automatica

10.12.1 Shared Preferences

Sono coppie chiave valore, è reso disponibile dalla classe `Context`, che rende disponibile ad esempio il metodo `getSharedPreferences()`. E' anche possibile crear rapidamente l'interfaccia grafica per modificare le preferenze (modificare le shared preferences). Lettura dei dati:

```
SharedPreferences settings = getSharedPreferences(PREFS_NAME, 0);
boolean silent = settings.getBoolean("silentMode", false);
```

Scrittura dei dati:

```
SharedPreferences.Editor editor = settings.edit()
editor.putBoolean("silentMode", mSilentMode);
editor.commit();
```

Negli esempi il secondo parametro è il valore di default che viene ritornato se non è settato il valore. Queste operazioni tecnicamente sarebbero da scrivere nel thread secondario ma ci gestisce tutto lui non dobbiamo preoccuparci di farlo.

10.12.2 File

L'uso dei file è analogo a Java classico, sono disponibili le medesime classi per la lettura/scrittura. L'accesso (apertura) dei file è differete, i file possono essere:

- Interni ("internal storage"): solo ad uso dell'applicazione. Cancellati quando l'applicazione viene cancellata.
- Esterni ("external storage"): accessibili all'utentee da altre app

10.12.3 Database

Il modo più semplice per usare i Databse in Android è usare `SQLite`, che ricordiamo essere un po' diverso rispetto ad un DBMS classico, è una libreria con cui interagiamo con dei metodi per fare delle query.

Per costruirlo abbiamo la classe `SQLiteOpenHelper` implementando due metodi:

- `onCreate`: abbiamo la query di scrittura di DB
- `onUpdate`: viene richiamata quando vogliamo aggiornare il database

```

class DatabaseHelper extends SQLiteOpenHelper {

    DatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE " + NotePad.Notes.TABLE_NAME + " ("
            + NotePad.Notes._ID + " INTEGER PRIMARY KEY, "
            + NotePad.Notes.COLUMN_NAME_TITLE + " TEXT, "
            + NotePad.Notes.COLUMN_NAME_NOTE + " TEXT, "
            + NotePad.Notes.COLUMN_NAME_CREATE_DATE + " INTEGER, "
            + NotePad.Notes.COLUMN_NAME_MODIFICATION_DATE + " INTEGER"
            + ");");
    }
}

```

Li richiamo quando apriamo il database.

Quando abbiamo definito questa classe possiamo aprire il db in lettura/scrittura. Ed utilizzare i metodi:

- insert()
- query() e rawQuery()
- delete()
- update()

Il problema è che scriviamo un linguaggio formale (SQL), come stringa in un altro linguaggio di programmazione, come `ReactNative`, in quanto gli errori sono visibili solo a runtime. Un altro aspetto noioso da fare è la serializzazione e de-serializzazione dei dati. Diventa complesso quando ho insiemi complessi di oggetti, in relazione tra di loro.

Esistono delle soluzioni che ci semplificano la scrittura del codice SQL, e che ci permettono di fare la serializzazione e de-serializzazione automatica. Un sistema molto avanzato su `Ios` è `cordata`, che permette di disegnare graficamente la struttura dati (schema ER) e da lì crea dinamicamente il DB e gli oggetti dinamici che rappresentano i dati del DB. In `Android` c'è `Room` che permette di salvare direttamente gli oggetti in db e prenderli come oggetti.

10.12.4 Room

(Vanno messi degli import in `Gradle`). [Documentazione](#). Abbiamo una classe che rappresenta le entità (utente), una classe che rappresenta le operazioni sulle entità (quali sono le query). Una classe (astratta) che rappresenta il DB. Usiamo

le funzionalità di Room per ottenere una istanza di una classe che estende la nostra classe astratta. Fatto tutto questo abbiamo la possibilità di richiamare le nostre operazioni su DB andando a leggere e a scrivere degli oggetti, non ci dobbiamo preoccupare della de-serializzazione.

Entità

Definiamo una o più classi che rappresentano le entità che devono essere memorizzate nel DB. Sono definiti gli attributi in una classe Java. Tramite annotazioni indichiamo le associazioni agli attributi del DB dove devono essere memorizzati.

```
@Entity
public class User {
    @PrimaryKey
    public int id;

    public String firstName;
    public String lastName;
}
```

Data access object (DAO)

E' un interfaccia che definisce quali operazioni si possono fare sui dati, sotto forma di metodi java, ognuno associato all'operazione SQL da effettuare sul DB. (la notazione User... significa che possiamo passargli un utente, degli utenti separati da virgola o un array di user e lui si fa andare bene tutto).

In documentazione c'è scritto come fare per fare delle query con il where.

```
@Dao
public interface UserDao {
    @Insert
    void insertAll(User... users);

    @Delete
    void delete(User user);

    @Query("SELECT * FROM user")
    List<User> getAll();
}
```

Creazione dei DB

Partendo dalle entità e dal DAO così creati, si crea prima una classe astratta e poi si usano le funzionalità di Room per creare un oggetto concreto.

```
@Database(entities = {User.class}, version = 1)
public abstract class UserRepository extends RoomDatabase {
    public abstract UserDao userDao();
}
```

Ovviamente si arrabbia se facciamo le chiamate a DB nel main thread, perchè sono bloccanti.

```
UserRepository db = Room.databaseBuilder(getApplicationContext(),
    UserRepository.class, "database-name").build();

//.....Some code here.....
db.userDao().insertAll(user);
```

(c'è un'annotazione che dice che quando vede una chiave se c'è già aggiornamela)

10.13 Programmazione multithread in Android

Dobbiamo gestire il fatto che le chiamate al DB non possono essere fatte nel main thread. Quindi possiamo creare un flusso di esecuzione parallelo al main thread. Funzionano come in Java tradizionale:

- Estendendo la classe Thread, oppure
- Implementando l'interfaccia Runnable

```
public class MyThread extends Thread {

    public void run() {
        System.out.println("Hello from a thread!");
    }
}
```

```
...
//altra parte del codice dove devo far partire il nuovo thread
MyThread myThread = new MyThread();
myThread.start();
```

//NOTA: richiamo il metodo start() che genera un nuovo thread ed esegue al suo interno il codice del metodo run().
 //Se chiamassi direttamente il metodo run(), eseguirei lo stesso codice, ma nello stesso thread del chiamante.

Possiamo farlo creando una classe `MyThread` che la cui istanza prende in input un'istanza di un'interfaccia `Runnable`, che possiamo fare o creando una classe che implementa `Runnable` oppure farlo con una classe anonima.

```
public class MyRunnable implements Runnable {

    public void run() {
        System.out.println("Hello from a thread!");
    }
}

...
//altra parte del codice dove devo far partire il nuovo thread
MyRunnable myRunnable = new MyRunnable();
Thread myThread = new Thread(myRunnable);
myThread.start();

//Oppure in forma più compatta (identica a quella sopra)
(new Thread(new MyRunnable())).start();

//anche in questo caso possiamo usare una classe anonima per avere
codice più compatto
//scriviamo questo codice direttamente dove dobbiamo far partire il
thread

(new Thread(new Runnable(){
    public void run() {
        System.out.println("Hello from a thread!");
    }
})).start();
```

Siccome l'interfaccia `Runnable` definisce un solo metodo, possiamo usare una lambda expression:

```
new Thread(() -> {
    Log.d("TEST", "In another thread");
}).start();
```

All'interno di un thread secondario non possiamo modificare l'interfaccia grafica. Possiamo chiedere al thread principale di farlo, ogni oggetto di View ha un metodo post che permette di associare dentro un runnable a cui può essere associato un thread secondario.

Possiamo anche usare il metodo postValue di un oggetto LiveData, in modo che modifichiamo i dati e poi dato che abbiamo messo un observer su quei dati ci cambia da solo l'interfaccia.

```
public void onClick(View v) {
    new Thread(new Runnable() {
        public void run() {
            final Bitmap bitmap = loadImageFromNetwork("http://example.com/image.png");
            mImageView.post(new Runnable() {
                public void run() {
                    mImageView.setImageBitmap(bitmap);
                }
            });
        }
    }).start();
}
```

Un altro modo è importare Room per avere delle chiamate asincrone, che tutto sommato non ci fa guadagnare tanto tempo. Ci sono due librerie (documentate malissimo).

10.14 Posizione

La parte di acquisizione della posizione è semplice, è difficile acquisire i permessi per prendere la posizione. In realtà l'utilizzo della posizione si è semplificato rispetto a qualche anno fa.

Abbiamo due set di API:

- Android Location: sono rese disponibili direttamente dal SO. L'uso diretto di queste API è sconsigliato perchè sono difficili da gestire
- FusedLocation API: è una funzionalità di Google Play Services di più alto livello che si appoggiano a loro volta sulle API Android Location. Bisogna gestire la presenza/assenza di google play services (per gli scopi del progetto assumiamo che ci siano)

I Google Play Services sono un servizio in funzione sul sistema Android. L'app può richiedere di svolgere alcune funzioni.

Ci sono 3 modi per acquisire la posizione:

- Acquisire l'ultima posizione nota
- Calcolare la posizione corrente
- Registrarsi per ricevere aggiornamenti di posizione

10.14.1 Ricevere l'ultima posizione nota

Chiediamo al SO di ritornare l'ultima posizione che ha calcolato, ma non ne calcola una nuova.

```
private FusedLocationProviderClient fusedLocationClient;

protected void onCreate(Bundle savedInstanceState) {
    //Altro codice
    fusedLocationClient = LocationServices.getFusedLocationProviderClient(this);
}

//da qualche parte nel codice (dopo capiamo meglio in quale parte del codice)
fusedLocationClient.getLastLocation()
    .addOnSuccessListener(this, new OnSuccessListener<Location>() {
        @Override
        public void onSuccess(Location location) {
            // Got last known location. In some rare situations this can be null.
            if (location != null) {
                Log.d("Location", "Last known location:" + location.toString());
            } else {
                Log.d("Location", "Last Known location not available");
            }
        }
    });
```

Nell'onCreate creiamo un'istanza di un oggetto tramite il quale possiamo prendere la posizione. Poi abbiamo una funzione per prendere la posizione, dato che è una chiamata asincrona abbiamo un metodo che si registra come observer e quando ha finito istanziando una classe anonima (o lambda expression) fa quello che deve. Possiamo chiamare il metodo addOnSuccessListener perchè getLastLocation ritorna un oggetto Task. FusedLocationProviderClient espone anche il metodo getCurrentLocation, che calcola la posizione corrente e la ritorna, sempre tramite un Task. La gestione del risultato è identica a quanto avviene per getLastLocation. In questo caso però la posizione ritorna viene effettivamente calcolata.

Possiamo registrarci per ricevere aggiornamenti di posizione, qua bisogna bilanciare bene l'uso della batteria con la precisione. Quando ci si registra per ricevere aggiornamenti periodici della posizione si possono calibrare vari parametri per bilanciare le due cose, ad esempio dammi la posizione massima, quando mi sono spostato di 100mt... Tre componenti principali:

- Un oggetto `LocationRequest` che serve per specificare i parametri delle richieste di posizione
- Una funzione di callback che viene chiamata quando la posizione è disponibile
- Un oggetto che rappresenta il fornitore della posizione

Metodo di callback che viene richiamato quando c'è una nuova posizione:

```
//attributo di classe
private LocationCallback locationCallback;

//nel onCreate
locationCallback = new LocationCallback() {
    @Override
    public void onLocationResult(LocationResult locationResult) {
        if (locationResult == null) {
            return;
        }
        for (Location location : locationResult.getLocations()) {
            Log.d("Location", "New Location received: " + location.toString());
        }
    }
};

//Andate a vedere in documentazione quali attributi ha la classe Location
```

10.14.2 Autorizzazione dell'utente

Per usare la posizione devo avere l'autorizzazione dell'utente. Tecnicamente quello che vale è che dobbiamo mettere i permessi nel manifest:

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

ma dobbiamo anche chiederli all'utente, questo è difficile perchè non sono banali if then per richiedere la posizione, ma sono chiamate asincrone che fanno cose solo quando altre chiamate asincrone hanno finito. Non si può richiedere la posizione prima che l'utente abbia fornito l'autorizzazione. Supponiamo di voler far partire il calcolo della posizione all'avvio dell'app. All'avvio verifichiamo se abbiamo l'autorizzazione per l'uso della posizione, se ce l'ho faccio partire le richieste di posizione, altrimenti chiedo all'utente di darmi l'autorizzazione. Quando l'utente ha risposto, verifico se mi ha dato l'autorizzazione. Se non l'ha

fatto, gli segnalo che c'è un problema, se l'ha fatto, faccio partire le richieste di posizione.

Verificare se l'utente ha già concesso i permessi: facciamo una funzione `ContextCompat.checkSelfPermission` che torna true o false se abbiamo i permessi

```
if (ContextCompat.checkSelfPermission(this,
    Manifest.permission.ACCESS_FINE_LOCATION)
    != PackageManager.PERMISSION_GRANTED) {
    //I permessi non sono stati (ancora) concessi
} else {
    //I permessi sono stati concessi
}
```

Richiedere i permessi:

```
ActivityCompat.requestPermissions(this,
    new String[]{Manifest.permission.ACCESS_FINE_LOCATION},
    MY_PERMISSIONS_REQUEST_ACCESS_FINE_LOCATION);
```

richiede al sistema operativo di chiedere i permessi (la prima riga), l'ultimo parametro è una costante intera che ci serve quando abbiamo l'evento di accettazione/rifiuto dell'utente come detto prima la richiesta dei permessi è asincrona, quindi da qualche parte deve esserci una chiamata di callback:

```
@Override
public void onRequestPermissionsResult(int requestCode,
    String[] permissions, int[] grantResults) {
    switch (requestCode) {
        case MY_PERMISSIONS_REQUEST_ACCESS_FINE_LOCATION: {
            // If request is cancelled, the result arrays are empty.
            if (grantResults.length > 0
                && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                // permission was granted
                Log.d("Location", "Now the permission is granted");
            } else {
                Log.d("Location", "Permission still not granted");
            }
            return;
        }
    }
}
```

è `onRequestPermissionsResult` che è un metodo dell'activity, potrei avere più richieste di permessi, allora si sono inventati il `requestCode`, che quando facciamo una richiesta gli passiamo (è un intero), quando risponde dice qual'è l'intero a cui stava rispondendo.

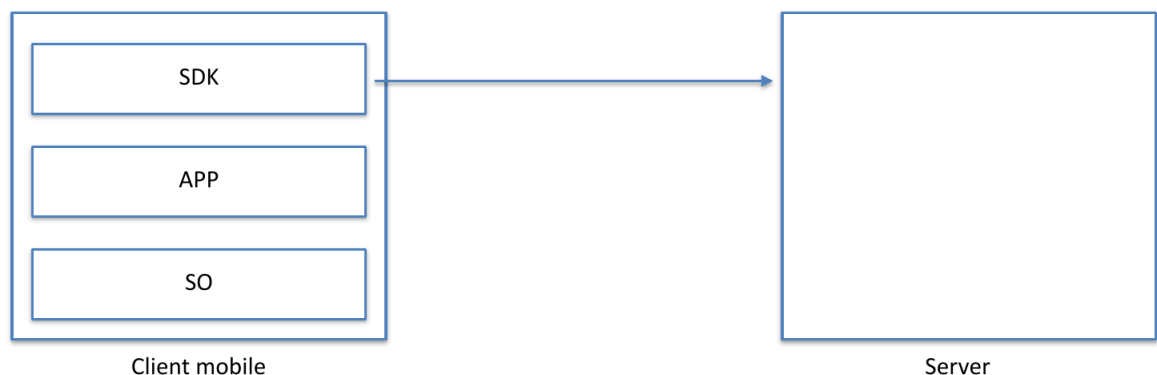
Attenzione all'ordine! Solitamente la struttura è questa:

- Se ho i permessi: faccio qualcosa (ad esempio faccio partire il calcolo della posizione)
- Se non ho i permessi li richiedo
 - Quando ricevo la risposta
 - * Se l'utente ha concesso i permessi: faccio qualcosa (ad esempio faccio partire il calcolo della posizione)
 - * Altrimenti gestisco questo caso

Nota: il codice di "faccio qualcosa" non va scritto due volte, mettiamolo in un metodo che richiamiamo due volte.

10.14.3 Uso delle mappe

Quando abbiamo un servizio di mappe l'architettura è client-server, un server espone al client dei servizi che permettono di scaricare mappe. Tuttavia l'uso delle mappe lato client è particolarmente complesso, dunque viene rilasciata una libreria che semplifica l'uso delle mappe.



I costi della posizione sono tutt'altro che irrisori, dipende dal servizio. C'è un sistema per cui ci registriamo al servizio di google maps che ci restituisce un token ed ogni volta che facciamo una richiesta gli mandiamo la chiave.

Alla chiave vengono applicate delle restrizioni, tipicamente alla chiave associamo l'identificativo univoco dell'applicazione che pubblichiamo.

Possiamo fare gli overlay, disegnare un marker sopra la mappa.

Possiamo mostrare la mappa centrata sulla posizione dell'utente.

ES1: creiamo un'app che mostri una mappa facendo centrare la mappa sulla posizione di un twok che prendiamo come da rete.

ES3: non facciamo la registrazione per ricevere aggiornamenti sulla posizione, ma usiamo `LastLocation` e `GetCurrentLocation`.

Fine del corso! METTIAMOCI NELL'OTTICA DI CONTINUARE AD IMPARARE, ANCHE NEL PERCORSO LAVORATIVO NON FACCIAMOCI FREGARE CONTINUANDO A FARE QUELLO CHE SAPPIAMO FARE.